MAKANAPP / BACKEND ENGINEERING

راهنمای جامع پروژه و بک‌اند ماکان

شناخت محصول، معماری، جریان اجرا و توسعه

نسخه بررسی: `main @ 701cba0`

تاریخ بازبینی: `2026-10-29`

> کتاب فارسی برای توسعه‌دهنده بک‌اند؛ بر مبنای کد واقعی، SQL Server و قراردادهای فعلی. این فایل بدون اینترنت قابل مطالعه است. برای جست‌وجو از Ctrl+F استفاده کنید.

فهرست کتاب

- وضعیت جاری و تغییرات مؤثر بر مطالعه
- وضعیت جاری Messenger: P0، P1، P2 و مرز P3
  - جریان‌های گذرا و پیام‌های پایدار
  - پیام زمان‌بندی‌شده، Quiz و Checklist
  - مرز رسانه و زیرساخت
  - مسیر مطالعه کد جدید
- تکمیل عملیاتی GAP-01 — ۱۵-۱۰-۲۰۲۶
- فهرست مطالب
- قابلیت‌ها و معماری
- از کجا شروع کنیم و پروژه چه مسئله‌ای حل می‌کند؟
  - نتیجه‌ای که خواننده باید بعد از مطالعه داشته باشد
  - برنامه مطالعه و اولین کار عملی
- واژه‌نامه مدل و شناسه‌ها
  - مثال چندسازمانی
- نقشه کد و مسئولیت لایه‌ها
  - از Controller به implementation
  - جای درست یک تغییر
- Dependency Injection، startup و مسیر هر درخواست
  - طول عمر سرویس‌ها
  - ترتیب pipeline
  - نشست این پروژه چگونه کار می‌کند؟
  - مجوز به صورت مرحله‌ای
  - خطا چگونه به HTTP تبدیل می‌شود؟
- راه‌اندازی محلی
  - پیش‌نیاز و بررسی اولیه
  - انتخاب دیتابیس توسعه
  - اعمال migration و اجرا
- تنظیمات
- قرارداد عملیاتی جاری: PH و انتشار دو محیط
  - تنظیم OTP و Host
  - worker، تراکنش و بازیابی
  - retention و observability
- قرارداد مشترک HTTP
  - استفاده در PowerShell و Postman
- ورود و پروفایل
  - دریافت OTP و نشست
  - تکمیل پروفایل و خروج
- پروفایل شخصی، تاریخ تولد و عکس
  - DOB خوداظهاری
  - انتخاب، تعویض و برداشتن عکس
  - Personal Messaging بدون نقش مدرسه
- فضای کاری و سرپرست
  - شناسه‌ها را با هم اشتباه نگیرید
  - انتخاب فضای شخصی یا سازمانی
  - دعوت و فرزند
- آموزش و حضور و غیاب
  - مجوزهای اصلی
  - ساخت کلاس
  - جلسه و برنامه تکرارشونده
  - ثبت و اصلاح حضور و غیاب
- مدیریت سازمان، کلاس، دعوت والد و جزوه
  - Directory و roster
  - دعوت والد و رضایت صریح
  - Lesson / Handout
- تکلیف و ارزیابی
  - ساخت و انتشار
  - پاسخ و تحویل دانش‌آموز
  - ارزیابی و انتشار نمره
- آزمون و انتشار نمره
  - طراحی و نسخه‌بندی آزمون
  - شروع، ذخیره پاسخ و نهایی‌سازی
  - تصحیح و انتشار نمره آزمون
- فایل‌ها
- پیام‌رسانی و همگام‌سازی
  - آغاز گفت‌وگو و مجوز ارتباط
  - Privacy، Contacts و Discovery
  - گروه، کانال و پیام
  - رفتار و state گفتگو در STEP 14C
  - تاریخچه، Delta و رسید خواندن
  - SignalR
  - جست‌وجو، مسدودسازی و گزارش
  - تکمیل Messenger در STEP 14E
- خطاها و تلاش مجدد
  - قراردادهای retry

- داشبوردهای نقش‌محور
- گزارش‌های تفصیلی نقش‌محور
- AI v1: جریان کد و وضعیت واقعی
- نقشه persistence و مرز تراکنش‌ها
  - سه لایه تضمین درستی داده
  - revision، rowversion و operationId سه چیز متفاوت‌اند
  - نمونه‌های واقعی تراکنش و locking
  - migration و محیط توسعه
- شش مسیر واقعی برای مطالعه کد از ابتدا تا انتها
  - ۱. درخواست و تأیید OTP
  - ۲. آخرین ظرفیت کلاس
  - ۳. تحویل تکلیف و انتشار نمره
  - ۴. پاسخ آزمون تا نتیجه رسمی
  - ۵. ارسال پیام، Delta و SignalR
  - ۶. یک گزارش نقش‌محور
- روش توسعه و عیب‌یابی برای عضو جدید تیم
  - یک قابلیت کوچک را چطور اضافه یا اصلاح کنیم؟
  - تست‌ها چه چیزی را ثابت می‌کنند؟
  - مسیر عیب‌یابی یک درخواست ناموفق
  - مرز میزبانی و موارد لازم برای تحویل عملیاتی
- تست و عیب‌یابی
  - اجرای validation
  - عیب‌یابی سریع
- محدودیت‌ها و مسیر مطالعه
  - مرز قابلیت موجود
  - دیتابیس و منابع کد
- نگهداری همین راهنما و مرجع
- شناسنامه تاریخی تحویل ۲۰۲۶-۱۰-۱۲ و نتیجه همان اجرا
- خروجی غیرهم‌زمان گزارش‌ها — STEP 11C
- اعلان‌های داخلی — STEP 12A
- تنظیمات و کانال‌های ارسال اعلان — STEP 12B
- اشتراک، دسترسی تجاری و ظرفیت — STEP 13A
- سفارش و پرداخت — STEP 13B
- تعدیل پیام و ایمنی کودک — STEP 14D
- Backoffice پلتفرم — STEP 15E
  - قرارداد IAM و Audit
  - Organization، User و Provisioning
  - Education، Messaging Metadata و PlatformSafety
  - Commerce، Finance و Operations
  - صفحه‌بندی، خطا و رفتار حساس
- تکمیل Messenger P1 و رجیستری Push
- سخت‌سازی Production — STEP 16
  - نشست و refresh token
  - Health و رفتار میزبانی
  - افزودهٔ Messenger P2 — Typing و Presence
  - افزودهٔ Messenger P2 — سازمان‌دهی شخصی
  - افزودهٔ Messenger P2 — مدیریت Group/Channel و عضویت
  - افزودهٔ Messenger P2 — Topics
  - افزودهٔ Messenger P2 — Scheduled / Silent
  - افزودهٔ Messenger P2 — Search و current-state hardening
  - افزودهٔ Messenger P2 — Poll / Quiz و پذیرش
  - افزودهٔ Messenger P2 — Checklist v1

راهنمای جامع پروژه و استفاده از بک‌اند و API ماکان

این راهنما برای تحویل پروژه به توسعه‌دهنده بک‌اند است. بازبینی `2026-10-29` از `main@701cba0`؛ **332 عملیات HTTP در 59 Controller و 291 مسیر؛ 570 ،107 schema جدول و 52 migration**. منبع جاری شامل حذف MFA، پروفایل شخصی و تکمیل مدیریت مدرسه است؛ گزارش‌های مرحله‌ای شواهد همان اجرا هستند و پذیرش سرور محسوب نمی‌شوند. شاخص پروژه.

وضعیت جاری و تغییرات مؤثر بر مطالعه

DOC-CONTRACT-01 / B08 در کد P0 اصلاح شد: گزینه‌های HTTP JSON با MVC هماهنگ و `StringEnumSchemaTransformer`، type حذف‌شدهٔ Enum در نسخهٔ جاری .NET 9 را تکمیل می‌کند. nullable، literal و Flags واقعی حفظ می‌شوند؛ JSON صحیح Controller/SignalR به عدد تبدیل نشده است. تست قرارداد، serializer واقعی nullable، OpenAPI، MVC/Hub، ترکیب Flags و رد نام/bit ناشناخته را بررسی می‌کند. ثبت نتیجه.

Messenger P0: تأیید اولیهٔ User شخصی با UsersManage و نشست معتبر و شاهد، self messaging-status، append-only discovery دو Adult بدون شرط مدرسه، عدم خروج شماره در Username discovery و بررسی محدودیت actor در ساخت Direct افزوده/اصلاح شدند. Realtime علاوه بر نشست/حساب/بافت جاری، visibility دقیق event را از Delta می‌گیرد؛ FromJoin و state خصوصی قابل bypass با audience hint نیستند. پذیرش دوحسابی شامل پاسخ POST گم‌شده پس از retry، commit همزمان، reply و reconnect فقط محلی اجرا می‌شود؛ REAL SERVER PASS ادعا نمی‌شود. ممیزی و تست‌ها.

- Identity شامل access یک‌ساعته، refresh چرخشی با عمر مطلق ۳۰روزه و مدیریت نشست است. OTP ملی‌پیامک provider واقعی دارد، اما موفقیت تست خودکار تحویل SMS سرور را اثبات نمی‌کند.
- onboarding مدرسه و Guardian در GAP-01 API عملیاتی دارند؛ دعوت اتصال حساب نیازمند پذیرش احرازشدهٔ صاحب حساب است و merge خودکار Person/User نداریم.
- Reports/export، اعلان داخلی/Commerce/order/payment lifecycle، outbox/preferences و Backoffice/IAM API دارند. Web Push و FCM در P1 پیاده شده‌اند و به config/credential خارجی نیاز دارند؛ پذیرش دستگاه واقعی pending است. SMS عمومی اعلان و gateway پرداخت واقعی همچنان unavailable هستند؛ OTP ورود با SMS اعلان یکی نیست.
- تصمیم ۲۰۲۶-۱۰-۲۴ مالک، MFA/Step-Up و key ring صرفاً مرتبط را از کد فعال حذف کرده است؛ قرارداد جاری Platform.
- سیاست مصوب GAP-05 جای پیش‌فرض‌های قدیمی Messenger را می‌گیرد: Unknown مانند Minor محافظت می‌شود؛ غریبه حق Direct ندارد؛ هر Send رابطه/Block/membership جاری و Content Safety را دوباره می‌سنجد. Internal Makan موتور قاعده‌محور واقعی ولی محدود است؛ رسانه unsupported به ReviewRequired می‌رود. Guardian و School Manager مجوز مرور Direct خصوصی ندارند. قرارداد کامل.
- ۱۲ API Backend AI با قرارداد context، Service JWT، v1 مجاز و response grounding وجود دارند؛ پیش‌فرض غیرفعال است. Service Auth واقعی قبلاً PASS شده، Parent Child Summary مانع Provider Timeout دارد و Full-12 واقعی Deferred است؛ RAG source lifecycle نیز deferred باقی مانده. وضعیت AI.

- انتشار IIS اکنون دو profile مستقل TestIis و ProductionIis دارد: Test با Mock مجاز false/true و Production با OTP واقعی true/false. انتخاب قبلی Production/HTTP با استثنا، پذیرش TLS نیست. فایل‌های خصوصی ignored و نمونه‌ها بدون secret هستند؛ راهنمای انتشار دو بسته، IIS و آمادگی.

برای شروع، بخش راه‌اندازی و ورود را بخوانید؛ سپس فضای کاری مناسب را انتخاب کنید و جریان موردنیاز را دنبال کنید. مرجع کامل Endpointها و قراردادها همراه این راهنماست و مسیر، پارامتر، بدنه، نوع پاسخ و فیلدهای تمام APIهای Controllerها را نشان می‌دهد.

وضعیت جاری Messenger: P0، P1، P2 و مرز P3

این بخش وضعیت کد ثبت‌شده تا `main@5885d11` در `2026-10-23` است. P0/P1/P2 تا Checklist موجودند؛ جدول‌های MISSING آغاز ممیزی P2 و تعویق اولیه Checklist تاریخچه‌اند. hardening worker و رفع deadlock retry مسدودسازی سیاست محصول را تغییر نداده‌اند؛ وضعیت Messenger و Handoffهای P0/P1/P2 مرجع‌اند.

حوزه	وضعیت جاری و مرجع
سن و شروع ارتباط	تأیید اولیهٔ رسمی سن، وضعیت امن self و Direct دو حساب Adult طبق GAP-05 در P0 موجودند؛ OTP/DOB خوداظهاری اثبات سن نیست
Peer Read	read-state مجاز Direct در P1؛ Group/Channel رسید per-member ندارند و Hidden علت خصوصی را افشا نمی‌کند
Web Push / FCM	transport واقعی و رجیستری خصوصی دستگاه در P1؛ `P1_PUSH_RUNTIME_PENDING` باقی است؛ SMS عمومی اعلان provider واقعی ندارد
Typing / Presence	lease و TTL واقعی با subscription و privacy جاری در P2؛ state گذرا در حافظه و تک‌instance، نه تاریخچه دائمی یا توزیع‌شده
Folder / Saved Labels	سازمان‌دهی خصوصی کاربر و قرارداد نسخه‌دار در P2؛ انتخاب پوشه/برچسب اجازه خواندن تازه نمی‌سازد
Invite / Join / Restriction	لینک دعوت، درخواست عضویت و محدودیت عضو طبق نقش و current policy در P2؛ token دعوت credential محرمانه است
Topics	lifecycle موضوع و topicId پیام در P2؛ موضوع حذف‌شده یا خارج scope انتخاب مجاز نیست
Scheduled / Silent	قصد ارسال پایدار، لغو و worker با بازبینی مجوز و safety هنگام dispatch؛ Scheduled به معنی Message/Sent نیست؛ Silent همان pipeline پیام با سیاست اعلان خودش است
Poll / Quiz	MessageKind.Poll و endpointهای مستقل Messenger؛ anonymous/multiple-choice و quiz یک‌بار پاسخ طبق قرارداد؛ کلید quiz قبل موعد انتشار عمومی افشا نمی‌شود

حوزه	وضعیت جاری و مرجع
Checklist	MessageKind.Checklist=8؛ حالت Collaborative و Personal مستقل از Poll و Assessment؛ تعریف مشترک و وضعیت شخصی قرارداد جدا دارند
Search	فیلترهای hasAttachment/hasLinks/topicId و current authorization قبل paging؛ false نیز فیلتر است؛ link search واکشی URL نیست
Voice / Video / Live	policy رسانه مصوب است، ولی `P3_S_INFRA_REQUIRED` ؛ LiveKit/TURN واقعی اجرا یا پذیرفته نشده‌اند؛ endpoint یا SDK تماس محصول از این مصوبه نتیجه نمی‌شود

جریان‌های گذرا و پیام‌های پایدار

Hub `/hubs/messaging` اکنون شش invoke عمومی دارد: `SubscribeConversation` ، `UnsubscribeConversation` ، `SetTyping` ، `Heartbeat` ، `SubscribePresence` و `UnsubscribePresence` . قراردادهای client شامل `ConversationChanged` ، `TypingChanged` و `PresenceChanged` است. `ConversationChanged` برای تغییر پایدار به read/Delta هدایت می‌کند؛ دو event دیگر فقط hint دارای expiry هستند. ارتباط Direct مجاز و privacy دوطرفه شرط Presence است؛ `Hidden` با timestamp خالی وضعیت private مقصد را افشا نمی‌کند.

پیش‌فرض‌های فعلی `MessagingEphemeralOptions` : typing TTL شش ثانیه، refresh سه ثانیه، heartbeat ده ثانیه، connection lease چهل‌وپنج ثانیه، Recently پنج دقیقه و نگهداری Last Seen یک روز. budget و سقف اتصال/subscription از همان options اعتبارسنجی می‌شوند. reconnect همهٔ subscriptionهای قبلی را برای اتصال جدید معتبر نمی‌کند؛ ثبت دوباره و بازیابی Delta لازم است. Group عضویت دائمی اجازهٔ send/read نمی‌سازد؛ نقش، Block ،account/session، رابطه و visibility دوباره بررسی می‌شوند.

پیام زمان‌بندی‌شده، Quiz و Checklist

ایجاد scheduled intent از `POST /api/v1/conversations/{id}/scheduled-messages` انجام می‌شود. حالت‌ها `Scheduled/Dispatched/Cancelled/Failed` هستند؛ `Dispatched` بعد از commit پیام واقعی و Outbox حاصل می‌شود. `clientMessageId` و payload/زمان یکسان همان intent را resolve می‌کنند؛ شناسهٔ قبلی با درخواست متفاوت conflict می‌دهد. ارسال فوری، رزرو شناسهٔ intent را دور نمی‌زند. logout به‌تنهایی لغو قصد پایدار نیست؛ مجوز mutation نشست فوراً سلب و مجوز حساب/رابطه در dispatch بازبینی می‌شود.

Poll/Quiz و Checklist APIهای Messenger و داده‌های همان scope هستند؛ از endpointها و tableهای Assessment برای آن‌ها استفاده نشود. history/search نمای خلاصه پیام را می‌دهند؛ جزئیات تعامل از endpoint مجاز خودش می‌آید. `expectedVersion` تعریف یا وضعیت رأی/تکمیل باید از response واقعی گرفته شود؛ retry نسخه قدیمی مجوز overwrite نیست. تغییر رأی/تکمیل کار، ارسال پیام یا unread/Push تازهٔ عمومی محسوب نمی‌شود مگر قرارداد همان event آن را صریح تعیین کند.

مرز رسانه و زیرساخت

policy رسانه P3 و گزارش نیاز زیرساخت S مرجع‌اند. وضعیت جاری `LIVEKIT_REJOIN_SECURITY_GATE=NOT_TESTED` و `P3_PRODUCTION_MEDIA_APPROVAL=NOT_GRANTED` است. نصب CLI، موفقیت regression یا تأیید محصول، اثبات revocation روی SFU یا عبور TURN نیست. نیاز محیط S شامل Linux lab مجاز غیرTURN، DNS/TLS، Docker runtime، Production احرازشده، دسترسی شبکه و دو client synthetic است؛ این به‌روزرسانی deployment، بازکردن پورت یا تغییر firewall انجام نمی‌دهد.

مسیر مطالعه کد جدید

`MessagingEphemeralService → IMessagingEphemeralState/InMemoryMessagingEphemeralState` برای lease و privacy؛ `ScheduledMessageService/Processor → EfMessagingStore.Scheduling/ScheduledDispatch` برای intent و ارسال؛ `ConversationPollService` و `ConversationChecklistService` برای تعامل؛ `NotificationPushDependencyInjection`، `WebPushNotificationProvider` و `FcmPushNotificationProvider` برای transport. نام/فیلد/route دقیق هر HTTP در مرجع تولیدشده و قواعد کامل در Handoffهای P0، P1 و P2 آمده است.

تکمیل عملیاتی ۲۰۲۶-۱۰-۱۵ — GAP-01

مسیر ساخت و مدیریت پرونده فراگیر و رابطه سرپرستی اکنون API دارد؛ توضیح کامل قراردادها، نمونه‌ها، خطاها، migration و تست‌ها در سند GAP-01 است. این بخش وضعیت قدیمی fixture-only را جایگزین می‌کند؛ HTML/PDF که در تحویل GAP-01 بازتولید نشده بودند، در همگام‌سازی جاری از منبع تازه بازسازی می‌شوند.

جریان مدرسه: ورود و workspace با نقش فعال Manager → `POST /api/v1/organization/people` → جست‌وجو/دریافت پرونده → ثبت‌نام اختیاری از Academic موجود. ایجاد پرونده حساب، عضویت، نقش، Enrollment یا TeacherAssignment نمی‌سازد. ویرایش نام فقط برای Person بدون حساب و دارای دقیقاً یک ارجاع سازمانی مجاز است؛ هر دو rowversion پرونده و Person لازم‌اند.

اتصال حساب: ساخت دعوت از `/api/v1/organization/people/{id}/invitations` برای User موجود → نمایش دعوت به همان کاربر → ورود و تکمیل پروفایل توسط صاحب حساب → پذیرش صریح با `{"confirmPersonLink":true}` در مسیر accept موجود. فقط مرجع PersonId پرونده متصل می‌شود؛ هیچ Person/User ادغام یا حذف نمی‌شود. Membership/Student Role از جریان قبلی دعوت می‌آید؛ ثبت‌نام همچنان مستقل است. پذیرش دعوت عادی قبلی بدون بدنه سازگار باقی مانده است.

سرپرستی: مدیر برای Parent دارای عضویت/نقش معتبر همان سازمان، از `/api/v1/organization/guardian-relations` رابطه Pending می‌سازد؛ سپس با rowversion فعال می‌کند. Parent خوداظهاریِ دسترسی ندارد. Parent از children/select موجود استفاده می‌کند؛ بعد از end/revoke، درخواست بعدی dashboard/report با انتخاب قبلی رد می‌شود و تاریخچه آموزشی حذف نمی‌شود. School Manager جای Platform Admin نیست و این قابلیت دسترسی به محتوای خصوصی پیام‌رسان ایجاد نمی‌کند.

فهرست مطالب

- قابلیت‌ها و معماری
- شروع مطالعه و هدف پروژه

- واژه‌نامه مدل و شناسه‌ها
- نقشه کد و مسئولیت لایه‌ها
- startup، DI و pipeline
- راه‌اندازی محلی
- تنظیمات
- عملیات و بازیابی
- قرارداد مشترک HTTP
- ورود و پروفایل
- پروفایل، عکس و ارتباط شخصی
- پروفایل، عکس و ارتباط شخصی
- فضای کاری و سرپرست
- آموزش و حضور و غیاب
- مدیریت مدرسه و جزوه
- مدیریت مدرسه و جزوه
- تکلیف و ارزیابی
- آزمون و انتشار نمره
- فایل‌ها
- پیام‌رسانی و همگام‌سازی
- خطاها و تلاش مجدد
- داشبوردهای نقش‌محور
- گزارش‌های تفصیلی
- استقرار و secretهای Production
- قرارداد اتصال React و React Native
- AI v1 و وضعیت واقعی
- persistence و تراکنش
- شش مسیر واقعی مطالعه کد
- روش توسعه و عیب‌یابی
- تست و عیب‌یابی
- نگهداری و ساخت HTML/PDF
- نتیجه اعتبارسنجی
- محدودیت‌ها و مسیر مطالعه

قابلیت‌ها و معماری

بخش	قابلیت موجود
Identity	درخواست و تأیید OTP، نشست Bearer، خروج، مشاهده و تکمیل پروفایل
Organization	فهرست و انتخاب فضای کاری، مشاهده و پذیرش یا رد دعوت
Guardian	فهرست فرزندان مجاز، انتخاب فرزند و مشاهده رابطه
Academic	دوره، درس، کلاس، ثبت‌نام، انتساب معلم، برنامه تکرارشونده، جلسه و حضور و غیاب
Assignment	پیش‌نویس و انتشار تکلیف، پاسخ متنی و فایل، تحویل نهایی، ارزیابی و انتشار یا اصلاح نمره
Exam	نسخه‌بندی آزمون، سؤال تستی و تشریحی، شروع تلاش، ذخیره پاسخ، انتقال حق نوشتن، نهایی‌سازی، تصحیح و انتشار نمره
Storage	بارگذاری، metadata، دانلود مجاز و حذف فایل
Messaging	Contact، Privacy و Discovery، گفت‌وگوی مستقیم، گروه، کانال، مدیریت اعضا، پیام و رسانه، جست‌وجو، Delta، Report، Block و SignalR
Reports	چهار داشبورد، شانزده گزارش تفصیلی و export خصوصی CSV با job پایدار؛ visibility نمره و scope نقش
Notification	اعلان داخلی، preferences، durable outbox و دستگاه؛ Web Push/VAPID و FCM قابل تنظیم، پذیرش واقعی pending؛ SMS عمومی unavailable
Commerce	Plan/Subscription/Entitlement/Quota و Order/Payment lifecycle؛ بدون gateway مالی واقعی
Platform	metadata operations، provisioning، IAM و moderation محدود به case؛ OTP، نشست معتبر و مجوز جاری Platform
AI Backend	دوازده API v1؛ Service Auth واقعی قبلی PASS؛ Parent Provider Full-12، Timeout و RAG lifecycle deferred

پروژه یک Modular Monolith با Onion Architecture است. همه قابلیت‌ها در یک میزبان API اجرا می‌شوند و SQL Server مرجع داده‌های تراکنشی است.

پروژه	مسئولیت
`MakanApp.Domain`	مدل و قانون دامنه، بدون EF Core یا HTTP
`MakanApp.Application`	سرویس کاربردی، قرارداد ورودی و خروجی، اعتبارسنجی و مجوز منبع

پروژه	مسئولیت
`MakanApp.Infrastructure`	پیاده‌سازی portها، SQL Server ،EF Core 9، فایل و adapterهای فنی
`MakanApp.Api`	Controller، احراز هویت، SignalR ،ProblemDetails و ترکیب DI
`tests`	تست‌های Integration ،Unit و Architecture

```
HTTP -> Controller -> Application Service -> Domain rules / Application port
                                          -> Infrastructure -> SQL Server
```

وابستگی کامپایل از بیرون به داخل است: `Api -> Application` ، `Api -> Infrastructure` ، `Infrastructure ->` `Application/Domain` و `Application -> Domain` . مسیر اجرای درخواست با جهت وابستگی یکسان نیست؛ Application از interface استفاده می‌کند و DI پیاده‌سازی Infrastructure را به آن می‌دهد.

از کجا شروع کنیم و پروژه چه مسئله‌ای حل می‌کند؟

MakanApp یک backend مشترک برای ارتباط و فعالیت آموزشی در سازمان است. یک شخص می‌تواند حساب ورود داشته باشد، عضو چند سازمان باشد، در هر سازمان چند نقش داشته باشد و در نشست جاری فقط یک فضای کاری و نقش را انتخاب کند. مدرسه/آموزشگاه، کلاس، حضور، تکلیف و آزمون داده رسمی آموزشی‌اند؛ پیام‌رسان مسیر ارتباط افراد است. داشبورد و گزارش، وضعیت همین داده رسمی را برای مخاطب مجاز خلاصه می‌کنند.

Commerce، Notification، Reports/export، Messenger، School و Platform Operations اکنون API اجرایی دارند؛ Copilot نیز قراردادهای Backend متصل به port سرویس AI دارد و آماده‌بودن سرویس بیرونی جداست. Knowledge ingestion/RAG source lifecycle کامل نشده است. مدیر مدرسه `Manager` با ادمین پلتفرم یک مفهوم نیست؛ bootstrap اولین PlatformOwner خارج HTTP و با مرز عملیاتی کنترل‌شده است.

نتیجه‌ای که خواننده باید بعد از مطالعه داشته باشد

1. بتواند solution را بسازد، دیتابیس توسعه را آگاهانه انتخاب کند و API را اجرا کند.
2. فرق حساب ورود، شخص، پرونده سازمانی، عضویت، نقش، ثبت‌نام و رابطه والد را بداند.
3. از یک endpoint به Controller، Application service، port، implementation و configuration برسد.
4. بداند کدام تغییر به Infrastructure، Application، Domain یا Api تعلق دارد.
5. خطای ورودی، نداشتن مجوز، تضاد وضعیت، تضاد نسخه و خرابی provider را تفکیک کند.
6. یک تغییر کوچک را با تست مناسب، migration بررسی‌شده و قرارداد HTTP روشن تحویل دهد.

برنامه مطالعه و اولین کار عملی

ترتیب	مطالعه/عمل	نشانه فهم درست
۱	همین بخش، معماری و واژه‌نامه	می‌توانید مدل یک Student و Parent در دو سازمان را توضیح دهید
۲	راه‌اندازی و تنظیمات	host بالا می‌آید و OpenAPI از همان build خوانده می‌شود
۳	ورود، workspace و child context	context از نشست معتبر سرور می‌آید؛ role ورودی مجوز مستقل نیست
۴	یک جریان Academic یا Assignment	از HTTP تا ذخیره و برگشت DTO را در کد دنبال می‌کنید
۵	تراکنش، نسخه و idempotency	می‌دانید دو درخواست همزمان یا retry چه اثری دارند
۶	Messaging و outbox، سپس آزمون	فرق finalize، draft، notification، commit و release روشن است
۷	تست‌ها، عیب‌یابی، راهنمای توسعه	می‌توانید تستی بنویسید که رفتار واقعی مورد تغییر را ثابت کند

اجرای نخست را با `/health`، OpenAPI و تست‌ها شروع کنید. پایگاه تازه بعد از migration سازمان، عضویت یا کلاس آماده ندارد. ورود OTP یک حساب ایجاد می‌کند، اما نقش مدیر سازمان به آن نمی‌دهد. برای یادگیری جریان کامل سازمانی از fixture و تست‌ها استفاده کنید؛ داده fixture را با seed عمومی محصول اشتباه نگیرید.

واژه‌نامه مدل و شناسه‌ها

اصطلاح واقعی	معنی و محل	اشتباه رایج
`User`	حساب سراسری ورود در Identity	یک نقش سازمانی ثابت روی User وجود ندارد
`UserCredential`	اتصال روش ورود، فعلاً موبایل، به User	شماره موبایل شناسه پرونده دانش‌آموز نیست
`Person`	مشخصات شخص مانند نام؛ مستقل از سازمان	ممکن است پرونده آموزشی وجود داشته باشد ولی حساب ورود متصل هنوز نداشته باشد
`OrganizationPerson`	پرونده همان Person در یک Organization	شناسه آن با PersonId و UserId قابل جایگزینی نیست
`Membership`	رابطه User با سازمان	پرونده شخص و اجازه استفاده حساب دو رابطه متفاوت‌اند

اصطلاح واقعی	معنی و محل	اشتباه رایج
`RoleAssignment`	نقش معتبر یک Membership؛ مثل Student/Teacher/Parent/Manager	داشتن نقش در سازمان A مجوز سازمان B نیست
`UserSession`	نشست دارای hash توکن، زمان انقضا و workspace/child منتخب	Bearer این پروژه JWT خودبسنده نیست
`AccessContext`	نتیجه معتبر resolve نشست، سازمان، عضویت، نقش و subject	DTO context یا مقدار ذخیره‌شده در کلاینت اثبات مجوز درخواست بعدی نیست
`GuardianRelation`	رابطه زمان‌دار والد/سرپرست با پرونده فراگیر در سازمان	Parent بودن به‌تنهایی مجوز همه دانش‌آموزان نیست
`Course` / `AcademicPeriod`	دوره زمانی آموزشی / درس سازمان	هیچ‌کدام جای Class را نمی‌گیرند
`AcademicClass`	کلاس با درس، دوره، ظرفیت و وضعیت	enrollment به کلاس مشخص تعلق دارد
`Enrollment`	ثبت‌نام پرونده فراگیر در کلاس	UserId را به جای LearnerOrganizationPersonId نفرستید
`TeacherAssignment`	انتساب Membership معلم به کلاس	role Teacher به‌تنهایی دسترسی به همه کلاس‌ها نمی‌دهد
`Session`	جلسه آموزشی با StartUtc/EndUtc	با UserSession ورود اشتباه نشود
`ScheduleRule`	الگوی تکرار با ساعت محلی و TimeZoneId	زمان محلی الگو با زمان UTC رخداد یکی نیست
`AssignmentVersion` / `AssignmentRecipient`	نسخه تکلیف و گیرندگان تثبیت‌شده انتشار	تغییر enrollment بعدی، گیرنده تاریخی را خودکار بازنویسی نمی‌کند
`SubmissionAttempt`	پاسخ/تلاش تحویل تکلیف	جدول تلاش آزمون نیست
`GradeRelease` / `EvaluationRevision`	نسخه ارزیابی تکلیف / انتشار رسمی آن	draft ارزیابی نتیجه عمومی دانش‌آموز نیست
`QuestionVersion` / `ExamVersion`	تعریف نسخه‌دار آزمون و سؤال	نسخه منتشرشده را برای اصلاح پاسخ یک دانش‌آموز تغییر ندهید
`ExamAttemptQuestion` / `ExamAttempt`	تلاش دانش‌آموز و مجموعه/ترتیب سؤال تثبیت‌شده	تلاش با deadline و شمارنده نسخه خود کار می‌کند
`ExamFinalAnswer` / `AnswerRevision`	تاریخچه پاسخ پذیرفته‌شده / snapshot پاسخ نهایی	autosave با finalize یک عملیات نیست
`ExamGradeRelease` / `ExamGradeRevision`	نسخه تصحیح آزمون / انتشار رسمی آن	با جدول نمره تکلیف یکسان نیست
`FileAsset`	metadata و scope فایل؛ bytes در IFileStorage	FileId بدون مجوز منبع اجازه دانلود نمی‌دهد
`ConversationParticipant`	عضویت و نقش ارتباطی در گفتگو	Owner/Admin/Member گفتگو با نقش آموزشی سازمان متفاوت است

اصطلاح واقعی	معنی و محل	اشتباه رایج
`MessagingChangeEvent` / `RealtimeOutbox`	تغییر پایدار برای Delta / کار ارسال notification	دریافت SignalR شرط ثبت پیام نیست

مثال چندسازمانی

یک User می‌تواند در سازمان A نقش Teacher و در سازمان B نقش Parent داشته باشد. انتخاب workspace A/Teacher در نشست فقط عملیات مجاز Teacher در A را فعال می‌کند. برای مشاهده فرزند در B باید workspace B/Parent و child معتبر آن انتخاب شود. حتی اگر ClassId معتبر A در درخواست B ارسال شود، سرویس باید scope جاری و رابطه منبع را کنترل کند. FK مرکب از اتصال داده دو سازمان جلوگیری می‌کند؛ مجوز خواندن HTTP همچنان وظیفه سرویس است.

نقشه کد و مسئولیت لایه‌ها

پروژه‌های اصلی در ریشه solution هستند؛ پوشه `src` وجود ندارد. namespaceها و نام کلاس‌ها انگلیسی‌اند؛ توضیحات توسعه‌دهنده فارسی است. هر حوزه پوشه‌های متناظر خودش را دارد، اما میزبان و دیتابیس مشترک‌اند.

```
MakanApp.Api/Controllers          HTTP transport
MakanApp.Application/{Area}       Use cases + DTOs + ports + policies
MakanApp.Domain/{Area}            Entities + enums + rules
MakanApp.Infrastructure/{Area}    Adapters + EF stores
MakanApp.Infrastructure/Persistence/Configurations/{Area}
MakanApp.Infrastructure/Persistence/Migrations
tests/MakanApp.UnitTests
tests/MakanApp.IntegrationTests
tests/MakanApp.ArchitectureTests
```

از Controller به implementation

حوزه/عملیات	Application service	port و implementation اصلی
OTP، پروفایل، نشست	`IdentityService`	`IIdentityStore → EfIdentityStore`؛ `IIdentitySecurity → IdentityCryptography`؛ `IOtpDeliveryProvider`
workspace و دعوت	`OrganizationService`	`IOrganizationStore → EfOrganizationStore`
والد و انتخاب فرزند	`GuardianService`	`IGuardianStore → EfGuardianStore`
context مشترک	`GuardianAccessContextResolver`	`IAccessContextResolver`؛ ابتدا OrganizationService، سپس اعتبار child

حوزه/عملیات	Application service	port و implementation اصلی
دوره، درس، کلاس و ثبت‌نام	`AcademicService`	`IAcademicStore → EfAcademicStore`
جلسات، الگو و حضور	`AcademicSessionService`	`IAcademicSessionStore → EfAcademicSessionStore`
تألیف و انتشار تکلیف	`AssignmentService`	`IAssignmentStore → EfAssignmentStore`
پاسخ و تحویل تکلیف	`SubmissionService`	`ISubmissionStore → EfSubmissionStore`
تصحیح تکلیف	`EvaluationService`	`IEvaluationStore → EfEvaluationStore`
تألیف آزمون	`ExamService`	`IExamStore → EfExamStore`
شروع تلاش آزمون	`ExamAttemptService`	`IExamAttemptStore → EfExamAttemptStore`
lease و پاسخ آزمون	`ExamAnswerService`	`IExamAnswerStore → EfExamAnswerStore`
نهایی‌سازی آزمون	`ExamFinalizationService`	`IExamFinalizationStore → EfExamFinalizationStore`
تصحیح آزمون	`ExamGradingService`	`IExamGradingStore → EfExamGradingStore`
گفتگو و پیام	`MessagingService`	`IMessagingStore → EfMessagingStore`
گروه، کانال و مدیریت اعضا	`ConversationManagementService`	`IConversationManagementStore → EfMessagingStore`
Block و Report	`MessagingSafetyService`	`IMessagingSafetyStore → EfMessagingStore`
فایل	`StorageService`	`IFileAssetStore → EfFileAssetStore`؛ `IFileStorage`؛ `IFileAssetBoundAccessResolver`
داشبورد	`DashboardService`	`IDashboardReadService → EfDashboardReadService`
گزارش تفصیلی	`DetailedReportService`	`IDetailedReportReadService → EfDetailedReportReadService`

فایل‌های `*Abstractions.cs` قرارداد میان use case و ابزار فنی را نگه می‌دارند. فایل‌های `*Contracts.cs` شکل ورودی/خروجی‌اند؛ entity EF را مستقیم به کلاینت برنمی‌گردانیم. بعضی کلاس‌ها `partial` هستند: هنگام مطالعه یک service یا store همه فایل‌های هم‌نام آن را بررسی کنید؛ پیاده‌سازی ممکن است در فایل اصلی تمام نشود.

جای درست یک تغییر

تغییر موردنیاز	محل معمول	دلیل
قانون وضعیت، اعتبار ذاتی entity، محاسبه خالص	Domain	مستقل از HTTP و persistence قابل استفاده و تست باشد
ترتیب use case، مجوز منبع، DTO و اعتبارسنجی ترکیبی	Application	عملیات کسب‌وکاری و portهای لازم را تعیین می‌کند
SQL، EF mapping، locking، فایل و provider	Infrastructure	وابستگی ابزار فنی از لایه داخلی جدا می‌ماند
route، binding، status، header، authentication handler	Api	قرارداد انتقال و composition root است
schema جدید	Configuration + Migration	تغییر باید نسخه‌بندی‌شده و قابل بازتولید باشد

در این مخزن Event Sourcing، MediatR، GenericRepository یا سرویس‌های مستقل برای هر حوزه استفاده نشده‌اند. برای افزودن رفتار، ابتدا الگوی همان service و port موجود را دنبال کنید. interface تازه زمانی لازم است که یک وابستگی واقعی را جدا کند.

Dependency Injection، startup و مسیر هر درخواست

دو نقطه اصلی ترکیب سیستم Program.cs و DependencyInjection.cs هستند. Program سرویس‌های HTTP را اضافه می‌کند؛ AddInfrastructure اتصال port، SQLها، serviceها، options و adapterها را متصل می‌کند. Controller فقط interface سرویس را دریافت می‌کند؛ Controller از EF query استفاده نمی‌کند.

طول عمر سرویس‌ها

- `MakanDbContext` و بیشتر service/storeها Scoped هستند: در scope درخواست از context مشترک استفاده می‌کنند؛ context بین درخواست‌ها share نمی‌شود.
- `TimeProvider.System`، policy، optionsهای بدون state درخواست و بعضی adapterها Singleton هستند. user/context درخواست را داخل singleton ذخیره نکنید.
- `MeliPayamakOtpProvider` از HttpClient ثبت‌شده و lifetime فنی خودش استفاده می‌کند؛ HttpClient را برای هر درخواست دستی نسازید.
- `EfMessagingStore` یک‌بار در scope ثبت و برای سه interface resolve می‌شود. سه interface به معنی سه DbContext مستقل نیست.
- dispatcher پس‌زمینه Singleton میزبان است ولی برای هر دور `CreateAsyncScope` می‌سازد تا processor و DbContext Scoped را درست دریافت کند.

ترتیب pipeline

```
Request
-> Forwarded headers (only when configured)
-> CorrelationIdMiddleware
-> HttpRequestMetricsMiddleware
-> Exception handler / Security headers / technical endpoints
-> HTTPS redirection (outside Testing and when required)
-> Routing
-> CORS
-> Session authentication
-> Rate limiter
-> ASP.NET authorization
-> Controller -> Application -> Domain/port -> Infrastructure
-> DTO / HTTP result
```

`[Authorize]` فقط الزام ورود را در transport اعمال می‌کند. کنترل نقش، سازمان و رابطه کلاس/فرزند داخل Application ادامه دارد. CORS قبل از authentication است تا preflight مرورگر پردازش شود. محدودیت OTP یک bucket حافظه‌ای بر IP است؛ محدودیت شماره موبایل از persisted challengeها نیز بررسی می‌شود. این دو محدودیت یک شمارنده نیستند.

نشست این پروژه چگونه کار می‌کند؟

`SessionAuthenticationHandler` مقدار header `Authorization: Bearer ...` را به `IdentityService.AuthenticateAsync` می‌دهد. `IdentityCryptography` از توکن تصادفی ۳۲ بایتی، hash HMAC-SHA256 می‌سازد؛ فقط hash در `UserSessions` نگه‌داری می‌شود. احراز هویت، نشست فعال/منقضی/لغوشده و User مربوط را از سمت سرور بررسی می‌کند. claims شامل UserId و SessionId هستند؛ role سازمانی از claims ثابت استخراج نمی‌شود.

توکن جاری JWT نیست؛ decode کردن آن با ابزار JWT یا نگه‌داری role کلاینت مبنای تصمیم سرور نیست. refresh token جدا وجود ندارد. خروج نشست را revoke می‌کند؛ تغییر `Identity:SecurityKey` هم hash OTPها و هم نشست‌های قبلی را نامعتبر می‌کند.

مجوز به صورت مرحله‌ای

```
Bearer -> active UserSession -> UserId + SessionId
-> OrganizationService: workspace + organization + membership + active role
-> GuardianAccessContextResolver: selected child relation if present
-> use case: resource organization + ownership/enrollment/teacher assignment
-> authorized projection or mutation
```

`GuardianAccessContextResolver` در Application قرار دارد؛ به پیاده‌سازی SQL وابسته نیست. انتخاب workspace در `OrganizationService`، child قبلی نشست را پاک می‌کند. رابطه منتخب در درخواست بعدی دوباره بررسی می‌شود؛ revoke شدن رابطه نباید با cache قبلی دور زده شود.

خطا چگونه به HTTP تبدیل می‌شود؟

`GlobalExceptionHandler` exceptionهای حوزه را به ProblemDetails با `code` پایدار و status تبدیل می‌کند. خطای ناشناخته با `UNEXPECTED_ERROR` و پیام عمومی500 برمی‌گردد؛ stack trace پاسخ عمومی نیست. `traceId` به پاسخ اضافه می‌شود تا بتوان درخواست را در log پیدا کرد. خطای model binding قبل از رسیدن به use case ممکن است ValidationProblemDetails دارای `errors` بدهد؛ وجود `code` اختصاصی برای همه خطاهای binding تضمین نشده است.

`CancellationToken` از RequestAborted/پارامتر Controller تا I/O عبور می‌کند. لغو درخواست، retry موفق یا rollback همه side effectهای بیرونی را به‌تنهایی تضمین نمی‌کند؛ ارسال SMS و نوشتن فایل باید با قرارداد هر عملیات بررسی شوند.

راه‌اندازی محلی

پیش‌نیاز و بررسی اولیه

- .NET SDK سازگار با هدف `net9.0`؛ baseline پروژه .NET 9 است.
- SQL Server محلی یا یک پایگاه توسعه مجاز؛ تست‌های Integration مشخصاً به Windows و SQL Server LocalDB نیاز دارند.
- دسترسی به NuGet برای restore اولیه.
- برای کار با migration، ابزار محلی `dotnet-ef` نسخه `9.0.5` در `.config/dotnet-tools.json` تعریف شده است.

از ریشه مخزن اجرا کنید:

```
dotnet --list-sdks
dotnet restore MakanApp.sln
dotnet tool restore
dotnet build MakanApp.sln --configuration Debug --nologo
```

انتخاب دیتابیس توسعه

کلید اتصال `ConnectionStrings:MakanDatabase` است. نمونه زیر فقط از Windows Authentication و یک نام مشخص برای دیتابیس توسعه استفاده می‌کند:

```
$env:ConnectionStrings__MakanDatabase =
'Server=.;Database=MakanApp_Development;Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertifi
cate=True'
```

این متغیر فقط برای PowerShell فعلی و فرایندهای فرزند آن تنظیم می‌شود. برای ذخیره تنظیم runtime در توسعه می‌توان از User Secrets استفاده کرد:

```
dotnet user-secrets set 'ConnectionStrings:MakanDatabase' $env:ConnectionStrings__MakanDatabase --project
```

```
MakanApp.Api/MakanApp.Api.csproj
```

`MakanDbContextFactory` هنگام اجرای ابزار EF، متغیر محیطی بالا را می‌خواند و User Secrets پروژه API را نمی‌خواند. در نبود متغیر، به `Server=.;Database=MakanApp` برمی‌گردد. برای اینکه migration روی مقصد دیگری اعمال نشود، متغیر اتصال را در همان پنجره‌ای که فرمان EF اجرا می‌شود صریح تنظیم کنید.

اعمال migration و اجرا

پس از انتخاب دیتابیس توسعه مجاز، migrationها را با فرمان زیر اعمال کنید. این فرمان schema مقصد را تغییر می‌دهد؛ مقصد آن باید همان پایگاه توسعه انتخاب‌شده باشد.

```
dotnet tool run dotnet-ef database update --project MakanApp.Infrastructure/MakanApp.Infrastructure.csproj --
startup-project MakanApp.Infrastructure/MakanApp.Infrastructure.csproj
dotnet dev-certs https --trust
dotnet run --project MakanApp.Api/MakanApp.Api.csproj --configuration Debug --launch-profile https
```

برنامه migration خودکار در startup ندارد؛ آخرین migration `20261028202239_CompleteSchoolManagement` و جمع۵۲ است. پس از Checklist، حذف Photo، DOB، MFA و School ثبت شده‌اند. schema با دادهٔ onboarding فرق دارد؛ این اسناد روی DB عادی/سرور migration یا seed اجرا نمی‌کنند. fixture توسعه فقط روی پایگاه اختصاصی و با تأیید مقصد طبق راهنمای مستقل مجاز است.

نشانی محلی	کاربرد
`https://localhost:7063/health`	زنده‌بودن میزبان؛ بررسی اتصال SQL Server در آن ثبت نشده است
`https://localhost:7063/swagger`	همه محیطها جز Production/Staging منتشر می‌شود؛ در آن دو فقط با Swagger:Enabled=true
`https://localhost:7063/openapi/v1.json`	همان شرط انتشار Swagger؛ سند runtime است
`https://localhost:7063/api/v1`	پیشوند APIهای کاربردی
`https://localhost:7063/hubs/messaging`	SignalR پیام‌رسانی

پروفایل HTTP، HTTPS را هم روی `http://localhost:5077` باز می‌کند. خارج Testing، اگر Hosting:RequireHttps=true باشد redirection فعال است؛ برای نمونه‌های این سند HTTPS انتخاب شده. Development پایه Mock دارد؛ Real OTP را با جفت true/false صریح انتخاب کنید و از کانفیگ محیط مقابل استفاده نکنید.

تنظیمات

مقادیر زیر از `appsettings.json` و `DependencyInjection.cs` آمده‌اند. برای متغیر محیطی، `:` را با `__` جایگزین کنید؛ مثلاً `Storage__MaxFileSizeBytes`.

کلید	پیش‌فرض یا رفتار
`ConnectionStrings:MakanDatabase`	در تنظیم پایه خالی است؛ Development نمونه اتصال محلی دارد
`Identity:SecurityKey`	حداقل ۳۲ بایت UTF-8؛ خارج از Development/Testing و هنگام انتخاب `MeliPayamakOtp` در هر محیط الزامی است؛ فقط provider محلی توسعه می‌تواند کلید تصادفی process داشته باشد
`Identity:Otp:LifetimeSeconds`	`300` ، اعتبار پنج‌دقیقه‌ای OTP
`Identity:Otp:ResendDelaySeconds`	`30` ، فاصله ارسال مجدد؛ در Testing صفر
`Identity:Otp:RateLimitWindowSeconds`	`3600`
`Identity:Otp:MaxRequestsPerWindow`	`5` درخواست برای شماره در پنجره تعیین‌شده در تنظیم پایه
`Identity:Otp:MaxFailedAttempts`	`5` تلاش ناموفق
`Identity:Otp:MinimumCodeLength` / `MaximumCodeLength`	`4` / `64` ؛ کد provider به‌صورت string opaque اعتبارسنجی می‌شود
`Sms:Provider`	`Development` در توسعه، `MeliPayamakOtp` برای سرویس واقعی، یا `Unavailable`
`Sms:MeliPayamakOtp:BaseUrl`	`https://console.melipayamak.com`
`Sms:MeliPayamakOtp:ServiceToken`	secret الزامی برای provider واقعی؛ هرگز commit یا log نشود
`Sms:MeliPayamakOtp:TimeoutSeconds`	`10` ثانیه
`Identity:Session:AccessLifetimeSeconds`	`3600` ، برابر یک ساعت
`Identity:Session:RefreshLifetimeSeconds`	`2592000` ، برابر ۳۰ روز؛ refresh چرخشی
`Hosting:RequireHttps` / `AllowInsecureHttp`	`true` / `false` ؛ HTTP فقط با استثنای صریح Production
`Database:AllowUntrustedServerCertificate`	`false` ؛ استثنای مستقل تأیید گواهی SQL، نه اجازهٔ Encrypt=False
`Storage:RootPath`	در Development برابر `%LOCALAPPDATA%\MakanApp\Storage`
`Storage:MaxFileSizeBytes`	`10485760` ، برابر ۱۰ MiB
`Storage:UnattachedLifetimeHours`	`24` برای فایل هنوز متصل‌نشده
`Storage:AllowedContentTypes`	`application/pdf` ، `image/jpeg` ، `image/png` ، `text/plain`
`Messaging:MaximumTextLength`	`4000` ؛ بیشتر از سقف مدل قابل تنظیم نیست
`Messaging:DefaultHistoryLimit` / `MaximumHistoryLimit`	`50` / `100`
`Messaging:MaximumAttachmentsPerMessage`	`10`
`Messaging:MaximumMentionsPerMessage`	`50`

کلید	پیش‌فرض یا رفتار
`MaximumChangeLimit` / `Messaging:DefaultChangeLimit`	`100` / `200`
`Messaging:MaximumSearchQueryLength`	`200`
`MaximumSearchLimit` / `Messaging:DefaultSearchLimit`	`20` / `50`

مقادیر انتهای جدول که در JSON نوشته نشده‌اند، fallback کد DI هستند. محدودیت IP درخواست OTP هم در `Program.cs` برابر ۱۰ درخواست در دقیقه است؛ این محدودیت درون‌حافظه‌ای و برای یک instance است.

کلید امنیت یا ServiceToken را در Git یا سند مشترک ذخیره نکنید. اگر provider محلی توسعه بدون کلید ثابت اجرا شود، توکن نشست و challenge OTP قبلی بعد از restart معتبر نیست. انتخاب MeliPayamakOtp بدون کلید پایدار در startup رد می‌شود. پایه Sms:Provider=Unavailable دارد؛ فایل خصوصی Production این انتشار provider واقعی را تنظیم می‌کند. RootPath در Production باید مطلق و پایدار باشد؛ خالی بودن آن fail-fast می‌شود. فایل واقعی Production ignored و نمونهٔ عمومی بدون secret است؛ Host دیگر به key ring MFA نیاز ندارد؛ فایل‌های قدیمی سرور تا cleanup مستقل حفظ شوند.

قرارداد عملیاتی جاری: PH و انتشار دو محیط

تنظیم OTP و Host

تنظیم	قرارداد جاری
`Otp:Enabled` / `AllowMockAuthentication`	base/Development/Test فعلی false/true؛ Production نمونه true/false؛ Production/Staging هر ترکیب ناامن را رد می‌کنند
`Observability:OperationalMetrics:SnapshotIntervalSeconds`	پیش‌فرض ۳۰؛ ۵..۳۶۰۰ ثانیه؛ SQL در refresh محدود، نه callback gauge
`Hosting:ShutdownTimeoutSeconds`	override اختیاری ۵..۹۰ ثانیه؛ بدون آن budget نهایی Host حفظ می‌شود، پیش‌فرض .NET 9 برابر ۳۰s
`Storage:TerminalCleanup:Enabled`	پیش‌فرض false؛ فعال‌سازی به grace صریح و سیاست نگه‌داری نیاز دارد

بخش‌های صرفاً MFA یعنی `BackofficeSecurity` و `Security:DataProtection` حذف شده‌اند؛ `/health/ready` تنظیمات معتبر، SQL و Storage را می‌سنجد. کلیدهای قدیمی سرور در این مرحله حذف نمی‌شوند. قرارداد تنظیم جاری.

worker، تراکنش و بازیابی

dispatcherهای NotificationDelivery، Scheduled، MessagingRealtime، Outbox و ReportExport از claim/lease و بازبینی مالکیت جاری استفاده می‌کنند. Outbox عمومی اثر durable و processed را در تراکنش کوتاه واحد ثبت می‌کند؛ realtime پیش از notifier نسل claim/rowversion و lease را می‌سنجد. `WorkerCompletionBudget` برای ثبت محدود نتیجه حداکثر ۵s است؛ failure، cancellation تجاری ساختگی نمی‌سازد. retry پس از انقضای lease مجوز overwrite نتیجهٔ owner جدید نیست.

`ReportExportOutputStore` bytes هر تلاش را بیرون تراکنش با key مستقل می‌نویسد؛ سپس job/claim را قفل و metadata canonical را منتشر می‌کند. Ready immutable و `FileAsset.Id == ReportExportJob.Id` ثابت است؛ بعد از crash می‌توان Ready را reuse کرد. قطع ACK به معنی rollback قطعی نیست و catch عمومی حق حذف فایل احتمالاً منتشرشده را ندارد. orphan candidateها به reconciliation مستقل نیاز دارند؛ root sweep وجود ندارد.

durable idempotency با تحویل خارجی exactly-once فرق دارد: provider ممکن است بپذیرد و process پیش از ثبت ACK از بین برود، پس Push تکراری ممکن است. SignalR هم hint است و Delta مرجع است. workerها داخل API با stop/idle همان host می‌ایستند. نخستین rollout fencing جدید export باید process legacy را drain/stop کند؛ ایمنی رقابت new/new، هم‌پوشانی old/new را ثابت نمی‌کند. بازیابی و حدود تضمین.

retention و observability

`TerminalFileCleanupDispatcher` هشتمین BackgroundService و پیش‌فرض خاموش است. فقط bytes محلی Deleted/Rejected پس از GracePeriodHours مثبت، بدون reference و بدون retention مجازند؛ Ready/Pending و اتصال‌های Message/Submission/Moderation/Scheduled/ReportExport مستثنا هستند. هیچ ردیف SQL/rowversion حذف یا عوض نمی‌شود. حذف کوچک local زیر قفل کوتاه و بازبینی reference است؛ UNC، recursive sweep و عبور از symlink/reparse point مجاز نیست. سیاست کامل.

correlation امن در log scope، header و ProblemDetails یکسان است. request log فقط method/template/status/duration دارد، بدون path/query/body/header خصوصی؛ loggerهای framework/EF در محیط عملیاتی با رخداد ثابت امن کنترل می‌شوند. snapshot صف stale-aware است و شکست refresh backlog را صفر نمی‌کند. collector/exporter یا alert واقعی با وجود Meter ساخته‌شده فرض نمی‌شود؛ Observability.

قرارداد مشترک HTTP

به‌جز درخواست و تأیید OTP، تمام Endpointهای Controllerها نیازمند نشست معتبر هستند:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
Accept: application/json
```

توکن `accessToken` یک مقدار تصادفی opaque است و JWT نیست. از decode کردن آن برای استخراج نقش استفاده نکنید؛ context معتبر را از `/workspaces/current` بگیرید. Endpoint مستقل refresh token وجود ندارد؛ ورود مجدد از OTP انجام می‌شود.

- نام فیلدهای JSON به شکل `camelCase` است؛ Enumها را به شکل رشته‌های تعریف‌شده مثل `Student` یا `Text` ارسال کنید.
- شناسه‌ها `Guid` هستند. مقدارهای `<...>` در نمونه‌های JSON جایگزین‌شونده‌اند و باید از پاسخ واقعی پر شوند.
- زمان‌های لحظه‌ای را به UTC و با پسوند `Z`، تاریخ‌ها را `yyyy-MM-dd` و ساعت محلی برنامه را `HH:mm:ss` ارسال کنید.
- پاسخ عادی envelope عمومی مثل `data` ندارد؛ همان شیء یا آرایه قرارداد برمی‌گردد. دانلود فایل پاسخ باینری است.
- روش `PATCH` در این پروژه JSON Patch نیست؛ بدنه کامل Command مربوط را ارسال کنید.

- نسخه‌هایی مانند `rowVersion` رشته Base64 هستند. مقدار را همان‌طور که دریافت کرده‌اید در فیلد `expected...` بعدی بفرستید. این APIها قرارداد عمومی `If-Match` ندارند.
- برای نسخه یا cursor داخل Query از URL encoding استفاده کنید؛ مخصوصاً `+` و `/` در Base64 نباید خراب شوند.
- مجوز به انتخاب workspace، عضویت، نقش و رابطه با منبع وابسته است. ارسال `OrganizationId` یا `ClassId` مجوز ایجاد نمی‌کند.

استفاده در PowerShell و Postman

برای درخواست‌های JSON در PowerShell، این helper محلی را می‌توانید در همان نشست تعریف کنید. ارسال بایت UTF-8 متن فارسی را حفظ می‌کند:

```
$apiBase = 'https://localhost:7063/api/v1'

function Invoke-MakanApi {
    param(
        [string]$Method,
        [string]$Path,
        [object]$Body,
        [string]$Token
    )

    $requestOptions = @{
        Method = $Method
        Uri = "$apiBase$Path"
        Headers = @{ Accept = 'application/json' }
    }
    if ($Token) {
        $requestOptions.Headers.Authorization = "Bearer $Token"
    }
    if ($null -ne $Body) {
        $jsonBody = $Body | ConvertTo-Json -Depth 12 -Compress
        $requestOptions.ContentType = 'application/json; charset=utf-8'
        $requestOptions.Body = [System.Text.Encoding]::UTF8.GetBytes($jsonBody)
    }
    Invoke-RestMethod @requestOptions
}
```

در Postman، سند `/openapi/v1.json` را Import کنید و Authorization مجموعه را روی `Bearer Token` با مقدار توکن جاری بگذارید. در تنظیم فعلی OpenAPI، security scheme اختصاصی Bearer اضافه نشده است؛ وجود Swagger UI را به معنی آماده‌بودن دکمه Authorize برای نشست‌ها در نظر نگیرید. توکن‌های واقعی را در collection یا environment اشتراکی صادر نکنید.

ورود و پروفایل

دریافت OTP و نشست

```
$phoneNumber = Read-Host 'شماره موبایل حساب آزمایشی'
$challenge = Invoke-MakanApi -Method POST -Path '/auth/otp/challenges' -Body @{
    phoneNumber = $phoneNumber
}
```

پاسخ `202` شامل `challengeId` ، `expiresAtUtc` و `resendAvailableAtUtc` است؛ کد در پاسخ نیست.

انتخاب مسیر ورود: Development و Test در فایل‌های فعلی با Otp:Enabled=false و Otp:AllowMockAuthentication=true اجرا می‌شوند. در Mock پاسخ challenge فقط شکل قرارداد را حفظ می‌کند؛ SMS ارسال و challenge در SQL ذخیره نمی‌شود. verify فقط فرمت شماره و کد را کنترل می‌کند؛ مقدار code با OTP واقعی و challenge مقایسه نمی‌شود. برای Real هر دو flag را به true/false تغییر دهید؛ DevelopmentOtpProvider در حالت Real/Development کد تصادفی در حافظه می‌سازد و endpoint عمومی دریافت کد ندارد. Production از MeliPayamakOtp موجود استفاده می‌کند؛ کد provider فوراً HMAC می‌شود.

```
$otpCode = Read-Host 'کد دریافت‌شده برای همین challenge'
$login = Invoke-MakanApi -Method POST -Path '/auth/otp/verify' -Body @{
    challengeId = $challenge.challengeId
    phoneNumber = $phoneNumber
    code = $otpCode
}
$accessToken = $login.accessToken
$otpCode = $null
$me = Invoke-MakanApi -Method GET -Path '/me' -Token $accessToken
```

پاسخ 200 شامل accessToken/tokenType/expiresAtUtc/refreshToken/refreshExpiresAtUtc/user است. الزام تطبیق challenge، انقضا، supersede و OTP_ALREADY_USED مربوط به Real است. در هر دو حالت حساب معلق رد می‌شود، حساب جدید مطابق مسیر موجود می‌تواند ساخته شود و session/refresh/مجوزها عادی‌اند. کد ۴..۶۴ کاراکتر و بدون control character است و ارقام normalize می‌شوند. در Mock تکرار verify می‌تواند session جدید بسازد؛ تغییر flag نشست قبلی را revoke نمی‌کند. هیچ Role/Membership از Mock به‌دست نمی‌آید.

تکمیل پروفایل و خروج

```
$me = Invoke-MakanApi -Method PATCH -Path '/me/profile' -Token $accessToken -Body @{
    firstName = 'کاربر'
    lastName = 'آزمایشی'
    displayName = 'کاربر آزمایشی'
    username = 'demo.user'
}
```

نام و نام خانوادگی اجباری و هرکدام حداکثر ۱۰۰ کاراکترند. `displayName` اختیاری و حداکثر ۲۰۰ کاراکتر است. `username` باید ۳ تا ۳۲ کاراکتر و شامل حروف لاتین، رقم، نقطه یا زیرخط باشد؛ یکتایی آن نسبت به بزرگی و کوچکی حروف حساس نیست. مقدار نمونه را برای هر حساب تغییر دهید.

`POST /auth/logout` بدون بدنه، نشست جاری را revoke می‌کند و `204` می‌دهد. پس از موفقیت، توکن محلی را پاک کنید.

پروفایل شخصی، تاریخ تولد و عکس

حساب User مستقل از Membership/Role است؛ افزودن یا پایان نقش، Profile، عکس یا گفتگوهای شخصی را حذف نمی‌کند. DOB و Avatar شرط جدید تکمیل پروفایل یا تأیید سن نیستند؛ Personal Context به مدرسهٔ صوری نیاز ندارد. گزارش پروفایل و Messaging.

DOB خوداظهاری

`PATCH /api/v1/me/profile` همان نام‌ها و Username قرارداد قبلی را می‌گیرد و `dateOfBirth` اختیاری افزوده دارد؛ فرمت میلادی `YYYY-MM-DD`، date-only و بدون timezone است. تاریخ آینده با روز UTC سرور، 400 PROFILE_VALIDATION_FAILED است؛ minimum age تازه تعریف نشده. omission/null مقدار قبلی را حفظ می‌کند؛ clear با null تعریف نشده و PATCH تک‌فیلدی مستقل ساخته نشده است. `GET /api/v1/me` و پاسخ ورود، تاریخ self را دارند؛ People/Directory/AI تاریخ تولد تازه دریافت نمی‌کنند.

`Person.DateOfBirth` در `identity.Persons` و از نوع SQL date nullable است؛ با `CommunicationAgeVerification.VerifiedBirthDate` فرق دارد. تغییر self DOB، شاهد سن یا CommunicationAgeCategory/VerificationStatus را عوض نمی‌کند. Unknown همچنان protected است و DOB خوداظهاری شرط discovery/safety را دور نمی‌زند.

انتخاب، تعویض و برداشتن عکس

```
POST /api/v1/files (multipart File, Personal Workspace)
-> FileAsset Ready owned by current User, OrganizationId=null
-> PUT /api/v1/me/profile-photo {fileAssetId}
-> ProfileController -> ProfilePhotoService -> User.ProfilePhotoFileAssetId
-> ProfilePhotoResult {fileAssetId,contentType,downloadUrl}
```

فقط Ready شخصی خود کاربر و JPEG/PNG مجاز است؛ WebP/SVG در این نسخه پشتیبانی نشده‌اند. سقف از Storage:MaxFileSizeBytes موجود می‌آید، پایه۱۰MiB؛ تصویر از allowlist/extension/signature موجود عبور می‌کند. Guid دانسته‌شده، فایل سازمانی یا عکس دیگری مجوز اتصال نیست: FILE_NOT_FOUND=404؛ غیر:Ready FILE_NOT_READY=409؛ غیرتصویری: FILE_TYPE_NOT_ALLOWED=415. انتخاب مجدد همان فایل idempotent و تعویض تنها مرجع User را عوض می‌کند. `DELETE /api/v1/me/profile-photo` تکرارپذیر و204 است؛ User/Person/blob را حذف نمی‌کند. عکس فعال از حذف عمومی با FILE_IN_USE=409 و از cleanup محافظت می‌شود.

`profilePhoto` به nullable در SafeMessagingIdentityResult، CurrentUserResult و MessagingPublicProfileResult افزوده شده؛ ناموجودبودن با null نمایش داده می‌شود. People/Contacts/Direct/inbox/اعضا/فرستنده/mention هویت امن مشترک را مصرف می‌کنند. عکس در پیام snapshot نشده؛ خواندن دوبارهٔ پیام قدیمی عکس فعلی فرستنده را می‌گیرد. قرارداد کامل عکس و migration.

دانلود `GET /api/v1/files/{id}/content` Bearer و grant جاری لازم دارد؛ URL عمومی یا CDN ناشناس نیست. client باید با header مجاز دریافت و در صورت نیاز Object URL بسازد؛ token در query نگذارد. نمایش عکس از مجوز نمایش همان هویت پیروی می‌کند: discoverable identity، self یا هویت واقعاً قابل مشاهده در conversation/history/poll مجاز. Block به‌تنهایی لزوماً تاریخچهٔ مجاز را پنهان نمی‌کند؛ عکس نیز مجوز Send یا دیدن تلفن/چت دیگران نیست. جایگزینی/برداشتن عکس، فایل قبلی را کورکورانه حذف نمی‌کند؛ Ready-orphan reaper وجود ندارد.

Personal Messaging بدون نقش مدرسه

Personal Direct/Group/Channel بدون School Role از قبل وجود داشت؛ نشست، profile، workspace نمایشی، سن تأییدشده/رابطه، block، privacy و restriction جاری همچنان guard هستند. Profile ناقص در messaging-status فقط `ReadExistingConversations` را اعلام می‌کند و NextAction=ProfileCompletionRequired باقی است؛ این hint مجوز ایجاد Direct یا discovery تازه نیست. بعد از پایان نقش، client می‌تواند از workspace selection موجود به Personal برگردد؛ fallback پنهان یا role سراسری جدید ساخته نشده است.

فضای کاری و سرپرست

شناسه‌ها را با هم اشتباه نگیرید

شناسه	مفهوم و محل مصرف
`UserId`	حساب ورود و طرف گفت‌وگو
`PersonId`	شخص در مدل Identity؛ برابر UserId فرض نشود
`OrganizationPersonId`	رکورد شخص در سازمان؛ ثبت‌نام و انتخاب فرزند از این شناسه استفاده می‌کنند
`MembershipId`	عضویت کاربر در سازمان؛ انتخاب workspace و انتساب معلم
`EnrollmentId`	ثبت‌نام شخص در یک کلاس؛ حضور و غیاب
`SessionId`	در Auth نشست ورود، اما در `/academic/sessions` شناسه جلسه آموزشی است
`ExamAttemptId`	تلاش دانش‌آموز در آزمون
`AttemptQuestionId`	سؤال منجمدشده در یک تلاش؛ برای ذخیره پاسخ، با شناسه سؤال editor جایگزین نشود

انتخاب فضای شخصی یا سازمانی

```
$workspaces = Invoke-MakanApi -Method GET -Path '/workspaces/me' -Token $accessToken
$personalContext = Invoke-MakanApi -Method POST -Path '/workspaces/select' -Token $accessToken -Body @{
    workspaceType = 'Personal'
    membershipId = $null
    role = $null
}
```

برای فضای سازمانی، `membershipId` و `role` را از همان عضو فهرست `/workspaces/me` بگیرید:

```
{
  "workspaceType": "Organization",
  "membershipId": "<membershipId از فهرست فضاها>",
  "role": "Teacher"
}
```

نتیجه `AccessContext` شامل `actorId`، `userId`، `sessionId`، `workspaceType`، `organizationId`، `membershipId`، `activeRole` و `subjectOrganizationPersonId` است. انتخاب در نشست ذخیره می‌شود؛ تغییر آن در یک tab روی درخواست‌های tab دیگری که همان توکن را دارد هم اثر دارد. بعد از تغییر فضا، داده‌های context قبلی را دوباره از API مربوط دریافت کنید.

کاربر بدون عضویت، فضای شخصی دارد. ساخت حساب و تکمیل پروفایل به‌خودی‌خود نقش `Manager`، عضویت سازمانی، رابطه سرپرست یا مجوز تماس شخصی ایجاد نمی‌کند.

دعوت و فرزند

1. با `GET /invitations` دعوت‌های حساب جاری را بگیرید.
2. `POST /invitations/{invitationId}/accept` یا `/decline` را بدون بدنه بفرستید.
3. پس از پذیرش، فهرست workspaceها را تازه کنید و عضویت/نقش موردنظر را انتخاب کنید. پذیرش تکراری همان دعوت می‌تواند با `alreadyAccepted=true` نتیجه قبلی را بدهد.
4. برای والد، ابتدا فضای سازمانی با نقش دقیق `Parent` را انتخاب کنید؛ مقدار `Guardian` نام نقش API نیست.
5. `GET /guardian/children` را بخوانید و `learnerOrganizationPersonId` مجاز را به `POST /guardian/children/{organizationPersonId}/select` بدهید.
6. `GET /workspaces/current` اکنون context فرزند را نشان می‌دهد. رابطه فعال در درخواست‌های بعدی دوباره بررسی می‌شود.

ایجاد سازمان، عضویت اولیه، دعوت، شخص سازمانی و رابطه سرپرست API عمومی ندارند. برای اجرای سناریو سازمانی به داده از قبل آماده‌شده در محیط مجاز نیاز دارید. fixtureهای Integration الگوی داده آزمایشی‌اند و ابزار seed دیتابیس عادی نیستند.

آموزش و حضور و غیاب

مجوزهای اصلی

عملیات	بافت مجاز
ساخت دوره، درس، کلاس، ثبت‌نام و انتساب معلم	`Manager` سازمان انتخاب‌شده
فهرست کلاس‌ها	Manager سازمان، Teacher منتسب، Student ثبت‌نام‌شده؛ Parent در این Endpoint پشتیبانی نمی‌شود
مدیریت جلسه و برنامه، ثبت و اصلاح حضور و غیاب	Manager یا Teacher دارای انتساب فعال به کلاس
برنامه و جزئیات جلسه	نقش مجاز مرتبط با کلاس؛ Parent با فرزند انتخاب‌شده و رابطه معتبر نیز پشتیبانی می‌شود
مشاهده دفتر حضور و غیاب کلاس	Manager یا Teacher منتسب
مدیریت تکلیف و آزمون، ارزیابی و انتشار	Manager یا Teacher منتسب به کلاس
تحویل تکلیف و پاسخ آزمون	Student واجد شرایط و مالک تلاش
نتیجه والد	Parent و context فرزند مجاز؛ DTO مخصوص والد

همه نقش‌های جدول سازمانی‌اند و فقط در همان سازمان و روابط معتبر معنی دارند.

ساخت کلاس

با توکن مدیری که workspace سازمانی را انتخاب کرده است:

```
$period = Invoke-MakanApi -Method POST -Path '/academic/periods' -Token $accessToken -Body @{
    title = 'دوره آزمایشی'
    startDate = '2026-10-01'
    endDate = '2027-02-01'
}
$course = Invoke-MakanApi -Method POST -Path '/academic/courses' -Token $accessToken -Body @{
    title = 'ریاضی'
}
$class = Invoke-MakanApi -Method POST -Path '/academic/classes' -Token $accessToken -Body @{
    academicPeriodId = $period.id
    courseId = $course.id
    title = 'ریاضی مقدماتی'
    capacity = 20
    activateImmediately = $true
}
```

تاریخ‌های نمونه را متناسب با دوره خود انتخاب کنید. شناسه دوره و درس باید از همین سازمان باشند. ظرفیت مثبت است؛ رقابت برای آخرین ظرفیت در transaction SQL Server کنترل می‌شود. برای مسیر معمول استفاده، `activateImmediately=true` بفرستید؛ Endpoint عمومی مستقلی برای فعال‌کردن کلاس Draft وجود ندارد.

عملیات	بدنه
`POST /academic/classes/{classId}/enrollments`	`{"learnerOrganizationPersonId":"<شناسه شخص سازمانی>"}`
`POST /academic/classes/{classId}/teachers`	`{"teacherMembershipId":"<عضویت فعال معلم>"}`
`POST /academic/classes/{classId}/enrollments/{enrollmentId}/end`	`{"finalStatus":"Completed"}` یا `Withdrawn`
`POST /academic/classes/{classId}/teachers/{teacherAssignmentId}/end`	بدون بدنه

ثبت‌نام یا انتساب فعال تکراری خطا می‌دهد؛ این دو عملیات قرارداد عمومی `clientOperationId` ندارند. پایان ثبت‌نام/انتساب، تاریخچه را حذف نمی‌کند.

جلسه و برنامه تکرارشونده

برای `POST /academic/classes/{classId}/sessions`:

```
{
  "title": "جلسه اول",
  "startUtc": "2026-11-01T06:30:00Z",
  "endUtc": "2026-11-01T08:00:00Z",
  "timeZoneId": "Asia/Tehran",
  "meetingUrl": null
}
```

زمان جلسه جدید باید آینده باشد؛ هنگام اجرای مثال، تاریخ را متناسب با ساعت سرور تغییر دهید. پایان بعد از شروع است. تداخل زمانی کلاس و معلم بررسی می‌شود. پاسخ `SessionResult` دارای `rowVersion` است؛ برای ویرایش، علاوه بر فیلدهای بالا `expectedRowVersion` لازم است. لغو یا تکمیل فقط بدنه `{"expectedRowVersion":"<rowVersion>"}` می‌خواهند.

برای `POST /academic/classes/{classId}/schedule-rules`:

```
{
  "localDayOfWeek": "Sunday",
  "localStartTime": "10:00:00",
  "durationMinutes": 90,
  "timeZoneId": "Asia/Tehran",
  "effectiveFrom": "2026-11-01",
  "effectiveUntil": "2026-11-30",
  "sessionTitle": "جلسه هفتگی",
```

```
  "meetingUrl": null
}
```

قانون برنامه، روز و ساعت محلی را همراه timezone نگه می‌دارد و جلسه‌های دارای زمان UTC تولید می‌کند. برای ویرایش قانون، `applyFromDate` و `expectedRowVersion` نیز لازم‌اند. برنامه بازه را با `GET /academic/schedule?fromUtc=...&toUtc=...` بخوانید؛ هر دو زمان UTC، پایان بزرگ‌تر از شروع و بازه حداکثر یک سال است.

ثبت و اصلاح حضور و غیاب

ابتدا `GET /academic/sessions/{sessionId}/attendance` را بگیرید. برای `PUT` همان مسیر:

```
{
  "entries": [
    { "enrollmentId": "<enrollmentId از roster>", "status": "Present" }
  ]
}
```

مقادیر ثبت‌کردنی `Present` ، `Absent` ، `Late` و `Excused` هستند. `NotRecorded` وضعیت نمایش ثبت‌نشدن است. ثبت پیش از شروع جلسه و برای جلسه لغوشده مجاز نیست. برای اصلاح رکورد موجود از `PATCH` `/academic/sessions/{sessionId}/attendance/{attendanceId}` با `status` ، `correctionReason` و `expectedRowVersion` استفاده کنید؛ اصلاح تاریخچه جدا دارد.

مدیریت سازمان، کلاس، دعوت والد و جزوه

مرجع دقیق UI، نمونه request/response و خطاهای جدید قرارداد مدیریت مدرسه است؛ prefix همه مسیرهای این بخش api/v1/ است. شناسهٔ انتخابی client مدرک مجوز نیست.

Directory و roster

Manager در Workspace سازمانی جاری، `GET /organization/members` را با search/role/status/page/pageSize می‌خواند؛ OrganizationId ورودی ندارد. Directory روی Membership است، نه پروندهٔ OrganizationPerson؛ account ناقص ممکن است PersonId/Username و پروندهٔ فعال null داشته باشد. activeRoles از Assignment فعال می‌آیند؛ تلفن، DOB و secret در این projection نیستند. عکس فقط با grant هویت موجود نمایش داده می‌شود.

`GET /academic/classes/{classId}/learners` و `/teachers` roster صفحه‌بندی‌شده را به Manager یا Teacher با Assignment فعال همان کلاس می‌دهند؛ Student/Parent مجاز نیستند. learner تازه حتی پیش از Session/Assignment آموزشی دیده می‌شود. `GET /academic/classes` همان آرایه قبلی است؛ pagination یکسان به همه listها تحمیل نشده. ساخت Class فقط Manager است و Teacher با 403 MANAGER_ROLE_REQUIRED رد می‌شود؛ ساخت Session برای Teacher منتسب مجاز است.

Enrollment و TeacherAssignment همان writeهای موجود را دارند؛ rowversion پاسخ roster به معنی افزودن ExpectedVersion فرضی به End قبلی نیست. End و Enroll مقصد دو درخواست مستقل‌اند؛ transfer اتمیک جدید وجود ندارد. پایان Assignment یک مدرس، نقش سازمانی یا Assignment مدرس دیگر را پایان نمی‌دهد.

دعوت والد و رضایت صریح

```
Manager: POST /organization/people/{learnerId}/guardian-invitations
-> SchoolGuardianInvitationsController -> SchoolGuardianInvitationService
-> OrganizationService / existing Invitation store -> Pending
Parent: GET /invitations -> POST /invitations/{id}/accept
    {confirmGuardianRelation:true}
-> reuse/create Membership+Parent Role, Pending relation -> Active + audit
```

Request مدیر دقیقاً یکی از username یا expiresAtUtc، clientOperationId، phoneNumber و reason دارد. username exact و case-insensitive با @ اختیاری، یا شماره normalizeشده حساب موجود resolve می‌شود؛ ساخت حساب/credential، جست‌وجوی fuzzy و merge شخص نداریم. مقصد باید حساب فعال موجود باشد؛ expiry پس از اکنون و حداکثر۳۰روز و reason الزامی حداکثر۵۰۰ نویسه است. پاسخ مدیر تلفن/UserId/نام مقصد را افشا نمی‌کند. ایجاد201، replay200؛ همان clientOperationId با payload متفاوت409. دعوت Pending همان والد/کودک reuse می‌شود و کودک دوم مستقل است؛ rate limit ایجاد۱۰ درخواست در دقیقه برای actor است.

inbox فقط دعوت حساب جاری را می‌دهد؛ guardianLearnerOrganizationPersonId/guardianLearnerDisplayName/requiresGuardianConsent/nextAction افزایشی‌اند. organizationPersonId دعوت والد null است؛ آن selector برای اتصال حساب فراگیر است. accept بدون confirmGuardianRelation=true با400 GUARDIAN_INVITATION_CONSENT_REQUIRED و حساب مقصد نادرست404 است. profile کامل مقصد و پروندهٔ کودک فعال لازم‌اند. Invitation/Membership/Role/GuardianRelation/Audit در تراکنش هماهنگ‌اند؛ Enrollment/TeacherAssignment یا شمارهٔ تأییدشده ساخته نمی‌شود.

replay پذیرش همان receipt را می‌دهد و رابطهٔ Ended/Revoked یا Role پایان‌یافته را زنده نمی‌کند. revoke دعوت Pending با expectedRowVersion و reason از `/organization/guardian-invitations/{id}/revoke` است؛ رابطهٔ پذیرفته‌شده از lifecycle Guardian موجود پایان می‌یابد. statusهای Invitation برابر Pending/Accepted/Declined/Expired/Revoked هستند، نه statusهای SMS/Push. GuardianRelation تازه برای این flow ساخته نشده است.

Lesson / Handout

Method	مسیر	رفتار
GET/POST	academic/classes/{classId}/lessons/	list مجاز یا ساخت Draft با201/replay200
GET/PATCH	academic/lessons/{lessonId}/	خواندن مجاز یا جایگزینی محتوای Draft
POST	academic/lessons/{lessonId}/publish/	انتشار یا receipt انتشار قبلی

مسیر `ClassLessonsController → ClassLessonService → ClassLesson → IClassLessonStore/EfClassLessonStore → SQL Server` واقعی است. عنوان۱..۲۰۰، body plain text حداکثر۵۰۰۰۰، displayOrder=0..100000 و حداکثر۱۰ FileId غیرتکراری. Course از Class resolve می‌شود؛ ورودی CourseId جدا نداریم. Draft می‌تواند خالی باشد ولی publish حداقل متن یا فایل لازم دارد. body را HTML مورداعتماد رندر نکنید.

PATCH merge جزئی نیست؛ title/body/displayOrder/fileAssetIds/expectedVersion وضعیت کامل قابل‌ویرایش را تعیین می‌کنند. [] اتصال قبلی را برمی‌دارد و blob را حذف نمی‌کند؛ null body متن خالی است. Published immutable و ویرایش آن409 است؛ retry انتشار زمان و revision را دوباره نمی‌نویسد. optimistic concurrency از version واقعی response می‌آید.

Manager یا Teacher منتسب می‌نویسد؛ Student دارای Enrollment/پرونده فعال فقط Published را می‌بیند، Draft از list حذف و در Get404 می‌شود. Parent دسترسی Lesson مستقل ندارد. فایل تازه باید Ready همان سازمان و متعلق به actor باشد؛ مدرس مجاز دوم می‌تواند attachment موجود را حفظ کند. دانلود grant جاری منبع را بررسی می‌کند؛ uploader بودن پس از binding راه عبور از پایان Assignment نیست. آخرین اتصال معتبر می‌تواند از منبع مستقل دیگر باقی بماند. FILE_IN_USE از حذف مستقیم و cleanup از فایل متصل محافظت می‌کنند. Lesson نه AI Source است و نه Assignment/Exam/Grade جدید.

تکلیف و ارزیابی

```
پاسخ دانش‌آموز -> Draft -> انتشار و تعیین گیرندگان -> Draft ساخت
-> اتصال فایل / ذخیره متن -> Submit -> Draft ارزیابی -> Release -> نتیجه
```

ساخت و انتشار

با توکن Manager یا Teacher منتسب:

```
$assignment = Invoke-MakanApi -Method POST -Path "/academic/classes/$($class.id)/assignments" -Token
$accessToken -Body @{
    title = 'تمرین اول'
    description = 'پاسخ تمرین را ارسال کنید.'
    dueAtUtc = [DateTime]::UtcNow.AddDays(7).ToString('o')
    allowLateSubmission = $false
    maxAttempts = 2
    maxScore = 20
}
$published = Invoke-MakanApi -Method POST -Path "/academic/assignments/$($assignment.id)/publish" -Token
$accessToken -Body @{
    expectedAssignmentRowVersion = $assignment.assignmentRowVersion
    expectedVersionRowVersion = $assignment.versionRowVersion
}
```

پاسخ انتشار شامل `assignment` و `recipientCount` است. `MaxScore` متعلق به نسخه تکلیف است؛ نمره ارزیابی از این سقف تبعیت می‌کند. پیش‌نویس از مسیر `PATCH /academic/assignments/{assignmentId}` قابل ویرایش است و به هر دو نسخه نیاز دارد. انتشار گیرندگان را از ثبت‌نام‌های واجد شرایط همان زمان تعیین می‌کند؛ صرف عضویت Student برای پاسخ کافی نیست.

فهرست را با `GET /academic/classes/{classId}/assignments` یا `GET /academic/assignments?classId=...` بخوانید. نقش و رابطه با کلاس بر داده قابل مشاهده اثر دارند.

پاسخ و تحویل دانش‌آموز

`$studentToken` در مثال باید توکن نشست Student با workspace سازمانی انتخاب‌شده باشد:

```
$attempt = Invoke-MakanApi -Method POST -Path "/academic/assignments/$($assignment.id)/attempts" -Token
$studentToken
$attempt = Invoke-MakanApi -Method PATCH -Path "/academic/submission-attempts/$($attempt.id)/draft" -Token
$studentToken -Body @{
    answerText = 'پاسخ دانش‌آموز'
    expectedRowVersion = $attempt.rowVersion
}
$receipt = Invoke-MakanApi -Method POST -Path "/academic/submission-attempts/$($attempt.id)/submit" -Token
$studentToken -Body @{
    expectedRowVersion = $attempt.rowVersion
}
```

برای پیوست، ابتدا فایل را با همان حساب و scope مناسب آپلود کنید؛ سپس به `/academic/submission-attempts/{attemptId}/attachments` بدنه `fileAssetId` و `expectedRowVersion` بدهید. پس از هر ذخیره یا اتصال/حذف فایل، از `rowVersion` پاسخ جدید استفاده کنید. حذف پیوست، `expectedRowVersion` را در Query می‌گیرد.

ایجاد Draft، تلاش باز موجود را resume می‌کند. ذخیره Draft تحویل رسمی نیست؛ رسید `SubmissionReceipt` موفقیت Submit را با `submittedAtUtc`، `isLate` و وضعیت `Submitted` نشان می‌دهد. سرور مهلت، سیاست تأخیر، تعداد تلاش و آمادگی فایل را کنترل می‌کند. فیلد `contentVisible` را در پاسخ رعایت کنید؛ وجود metadata تلاش به معنی مجازبودن نمایش محتوای آن نیست.

ارزیابی و انتشار نمره

1. ارزیاب مجاز از `/academic/evaluations/queue`، با فیلترهای اختیاری `assignmentId`، `classId`، `reviewStatus` و `isLate`، صف را می‌خواند.
2. محتوای پاسخ نهایی از `/academic/submission-attempts/{attemptId}/evaluation/submission` خوانده می‌شود.
3. `PUT /academic/submission-attempts/{attemptId}/evaluation` یک Draft می‌سازد یا ویرایش می‌کند:

```
{
  "score": 18,
  "learnerFeedback": "پاسخ خوب است؛ محاسبه آخر را بررسی کن.",
  "guardianVisibleFeedback": "پیشرفت مناسبی داشته است.",
  "teacherPrivateNote": "یادداشت داخلی ارزیاب",
  "expectedRowVersion": null
}
```

در ایجاد اولیه، `expectedRowVersion=null` است؛ در ویرایش باید نسخه فعلی را بفرستید. نمره در بازه صفر تا `maxScore` نسخه تکلیف است. برای انتشار، به `/evaluation/release` مقدار `expectedRowVersion` پاسخ ارزیابی را بدهید.

دانش‌آموز `/academic/submission-attempts/{attemptId}/result` و والد `/guardian-result` را می‌خوانند. قبل از انتشار، نتیجه تکلیف می‌تواند `404 GRADE_NOT_RELEASED` بدهد. `teacherPrivateNote` به هیچ‌کدام داده نمی‌شود. اصلاح نتیجه منتشرشده از `/evaluation/corrections` با `correctionReason` و `expectedReleasedEvaluationRowVersion` یک Draft تازه می‌سازد؛ انتشار آن نیز جداگانه است.

آزمون و انتشار نمره

طراحی و نسخه‌بندی آزمون

با Manager یا Teacher مجاز، `POST /academic/classes/{classId}/exams`:

```
{
  "title": "آزمون ریاضی",
  "description": "آزمون کوتاه فصل اول",
  "availableFromUtc": "2026-11-01T06:30:00Z",
  "availableUntilUtc": "2026-11-01T08:30:00Z",
  "durationMinutes": 30,
  "maxAttempts": 1,
  "maxScore": 20,
  "randomizationPolicy": "QuestionOrder"
}
```

بازه نمونه را برای زمان اجرای خود تنظیم کنید. سیاست ترتیب سؤال `None` یا `QuestionOrder` است. پاسخ `ExamEditorDto` شامل `examRowVersion`، `versionRowVersion` و سؤال‌هاست. نمونه افزودن یک سؤال ۲۰ نمره‌ای به `POST /academic/exams/{examId}/questions`:

```
{
  "order": 1,
  "type": "ObjectiveSingleChoice",
  "prompt": "حاصل ۲ + ۲ کدام است؟",
  "score": 20,
  "options": [
    { "order": 1, "text": "۴", "isCorrect": true },
    { "order": 2, "text": "۵", "isCorrect": false }
  ],
  "expectedVersionRowVersion": "<versionRowVersion فعلی>"
}
```

سؤال تستی حداقل دو گزینه و دقیقاً یک گزینه درست می‌خواهد؛ ترتیب و متن گزینه‌ها نباید تکراری باشد. سؤال `Descriptive` گزینه ندارد. هنگام انتشار، آزمون باید حداقل یک سؤال داشته باشد و مجموع نمره سؤال‌ها دقیقاً برابر `maxScore` شود.

به `POST /academic/exams/{examId}/publish` دو فیلد `expectedExamRowVersion` و `expectedVersionRowVersion` را از آخرین پاسخ بدهید. نسخه منتشرشده درجا ویرایش نمی‌شود؛ `/versions` با `expectedExamRowVersion` و `expectedPublishedVersionRowVersion` نسخه بعدی می‌سازد. حذف سؤال، برخلاف بسیاری از DELETEها، بدنه JSON با

نسخه سؤال و نسخه آزمون دارد.

`GET /academic/exams` فهرست را بر اساس نقش/کلاس یا فرزند مجاز می‌دهد. `/editor` و `/teacher-preview` مخصوص ارزیاب مجازند و اطلاعات پاسخ صحیح دارند. `/student-preview` برای Teacher/Manager مجاز، نمایش امن بدون پاسخ صحیح می‌سازد؛ Student فقط با ثبت‌نام فعال و در بازه فعال آزمون می‌تواند آن را بگیرد. Parent اجازه دیدن محتوای سؤال از این مسیر ندارد.

شروع، ذخیره پاسخ و نهایی‌سازی

```
Student workspace -> Start -> دریافت سؤال‌های منجمد و سرور deadline
-> دریافت write lease -> Save Answer -> دریافت رسید هر ذخیره
-> دریافت رسید نهایی -> Finalize -> توقف ذخیره‌های در حال ارسال
```

1. به `POST /academic/exams/{examId}/attempts/start` بدنه `{"clientOperationId":"<Guid جدید>"}` بفرستید. برای retry همان عملیات، همین Guid را نگه دارید.
2. پاسخ دارای `examAttemptId`، `examVersionId`، `effectiveDeadlineUtc`، `serverNowUtc` و سؤال‌های دارای `attemptQuestionId` است. deadline از محدودترِ پایان بازه آزمون و مدت تلاش محاسبه می‌شود. ترتیب سؤال‌ها برای همان تلاش ثابت می‌ماند.
3. `POST /academic/exam-attempts/{attemptId}/write-lease` بدون بدنه بفرستید. `writeLeaseVersion` برگشتی را نگه دارید. حق نوشتن به نشست تعلق دارد؛ انتقال صریح از `/write-lease/transfer` نسخه را عوض می‌کند و نویسنده قبلی دیگر معتبر نیست.
4. برای هر سؤال، به `PUT /academic/exam-attempts/{attemptId}/answers/{attemptQuestionId}` بدنه زیر را بدهید:

```
{
  "clientOperationId": "<Guid مستقل این ذخیره>",
  "writeLeaseVersion": 1,
  "expectedRevisionNumber": null,
  "selectedOptionId": "<optionId همان سؤال در تلاش>",
  "textAnswer": null
}
```

`writeLeaseVersion=1` فقط نمونه است؛ مقدار واقعی پاسخ lease را استفاده کنید. در اولین پاسخ `expectedRevisionNumber=null` و در ویرایش، `revisionNumber` آخرین پاسخ پذیرفته‌شده لازم است. برای سؤال تشریحی `selectedOptionId=null` و `textAnswer` متن پاسخ است. رسید ذخیره، `revisionNumber`، `acceptedAtUtc` و `answerSetVersion` جدید را می‌دهد.

برای ادامه پس از reconnect یا تعارض، `GET /academic/exam-attempts/{attemptId}/answers` را بگیرید. این پاسخ مجموعه پاسخ پذیرفته‌شده، نسخه مجموعه و وضعیت write lease را نشان می‌دهد. ذخیره محلی کلاینت یا موفقیت HTTP ارسال‌نشده، اثبات پذیرش پاسخ در سرور نیست.

1. قبل از Finalize، تمام ذخیره‌های در حال ارسال را تمام کنید و نسخه نهایی مجموعه را بگیرید؛ سپس `POST /academic/exam-attempts/{attemptId}/finalize`:

```
{
  "clientOperationId": "<Guid مستقل نهایی‌سازی>",
  "expectedAnswerSetVersion": 1,
  "writeLeaseVersion": 1
}
```

هر دو نسخه نمونه‌اند و باید از پاسخ‌های جاری گرفته شوند. رسید شامل `finalizedAtUtc` ، `finalizedAnswerSetVersion` ، `answeredQuestionCount` ، `totalQuestionCount` و `status=Finalized` است. `GET /academic/exam-attempts/{attemptId}/receipt` آن را دوباره می‌دهد. رسید، نمره یا پاسخ صحیح ندارد.

پس از deadline، ذخیره یا Finalize جدید رد می‌شود و تلاش می‌تواند `Expired` شود. **auto-finalize یا scheduler پایان آزمون وجود ندارد**؛ وجود پاسخ ذخیره‌شده، تلاش Expired را به پاسخ تحویل‌شده تبدیل نمی‌کند. retry دقیق Finalize موفق با همان شناسه و payload، رسید قبلی را برمی‌گرداند؛ payload متفاوت با همان شناسه conflict است.

تصحیح و انتشار نمره آزمون

فقط تلاش `Finalized` وارد تصحیح می‌شود؛ منبع تصحیح `ExamFinalAnswers` و revisionهای منجمدشده است.

1. `GET /academic/exams/{examId}/grading` صف تصحیح را با فیلتر اختیاری `classId` و `status` می‌دهد.
2. `POST /academic/exam-attempts/{attemptId}/grading` بدون بدنه، Draft تصحیح را ایجاد یا بازیابی می‌کند. `GET` همان مسیر فقط Draft موجود را می‌خواند.
3. سؤال تستی بر مبنای کلید نسخه منجمد تصحیح می‌شود. سؤال تشریحی به بررسی دستی نیاز دارد. مسیر `PATCH /academic/exam-attempts/{attemptId}/grading/questions/{attemptQuestionId}` این بدنه را می‌گیرد:

```
{
  "awardedScore": 15,
  "learnerFeedback": "استدلال را کامل‌تر بنویس.",
  "evaluatorPrivateNote": "یادداشت خصوصی مصحح",
  "expectedGradeRowVersion": "<rowVersion فعلی تصحیح>"
}
```

نمره هر سؤال باید در بازه مجاز همان سؤال باشد. `expectedGradeRowVersion` نسخه کل Draft نمره است؛ پس از تغییر هر سؤال آن را از پاسخ تازه کنید.

1. به `/grading/complete` فیلدهای `learnerFeedback` ، `guardianVisibleFeedback` ، `evaluatorPrivateNote` و `expectedGradeRowVersion` بدهید؛ بررسی‌های لازم باید کامل شده باشند. وضعیت به `ReadyForRelease` می‌رسد.
2. به `/grade/release` بدنه `clientOperationId` و `expectedGradeRowVersion` بدهید. این مرحله انتشار رسمی است.
3. دانش‌آموز `/result` و والد `/guardian-result` را زیر `/academic/exam-attempts/{attemptId}` می‌خوانند. پیش از انتشار، برای تلاش نهایی‌شده پاسخ `200` با `releaseStatus=AwaitingGrading` یا `AwaitingRelease` و نمره `null` ممکن است؛ این با قرارداد نتیجه تکلیف متفاوت است.
4. `/grade/corrections` با `correctionReason` و `expectedReleasedGradeRowVersion` Draft اصلاحی می‌سازد؛ آن را دوباره کامل و منتشر کنید. تا انتشار اصلاحیه، نتیجه منتشرشده قبلی معتبر می‌ماند.

DTO دانش‌آموز فقط بازخورد دانش‌آموز و DTO والد فقط بازخورد والد را دارد؛ یادداشت خصوصی مصحح و کلید پاسخ در نتیجه عمومی قرار نمی‌گیرند.

فایل‌ها

بارگذاری فایل از `POST /files` با `multipart/form-data` انجام می‌شود؛ نام فیلد `File` است. در Postman بخش Body را روی form-data بگذارید، نوع `File` را برای آن فیلد انتخاب کنید و Content-Type دارای boundary را به خود ابزار بسپارید.

```
$uploadPath = Read-Host 'مسیر فایل PDF آزمایشی'
curl.exe --request POST "$apiBase/files" --header "Authorization: Bearer $accessToken" --form
"File=@$uploadPath;type=application/pdf"
```

پاسخ `201` شامل `id` ، `status` ، `contentType` ، `sizeBytes` ، `sha256Hash` ، `unattachedExpiresAtUtc` و `rowVersion` است. `id` را برای اتصال به تکلیف یا پیام نگه دارید؛ فقط فایل آماده و مجاز قابل اتصال است.

عملیات	مسیر و نکته
metadata	`GET /files/{fileId}`
دانلود	`GET /files/{fileId}/content` با Bearer؛ پاسخ stream است
حذف	`DELETE /files/{fileId}?expectedRowVersion=...` با نسخه URL-encoded

فایل لینک عمومی ندارد. مالکیت، scope و binding به پیام/پاسخ در مجوز دانلود اثر دارند. حذف فایل retained یا در حال استفاده می‌تواند `FILE_IN_USE` بدهد. مهلت فایل unattached به معنی وجود پاک‌سازی پس‌زمینه زمان‌بندی‌شده نیست.

در تنظیم پیش‌فرض فقط PNG، JPEG، PDF و متن مجازند. مدل Messaging نوع `Video` و `Voice` را می‌شناسد، اما برای آپلود آن‌ها باید MIME مجاز مناسب در تنظیم محیط وجود داشته باشد؛ تنظیم پایه این نوع‌ها را نمی‌پذیرد. اعتبارسنجی MIME و اندازه، جای اسکن بدافزار نیست؛ اسکن و object storage تولیدی در این نسخه ارائه نشده‌اند.

پیام‌رسانی و همگام‌سازی

افزودهٔ P1، فاز Peer Read: `GET /api/v1/conversations/{conversationId}/read-states` برای Direct رسید خواندن واقعی peer را با `peerReadThroughSequence/visibility/version` می‌دهد؛ Sent، Delivered و read خود فرستنده رسید peer نیستند. `ConversationChanged` و Delta اکنون `PeerReadStateChanged` با sequence nullable دارند؛ رویداد خصوصی read قبلی self-only باقی است. privacy، Block هر دو جهت، eligibility و مرز تاریخچه جاری اعمال می‌شوند؛ Group/Channel/Saved مقدار NotSupported دارند. قرارداد دقیق P1 بر توضیح تاریخی self-only پایین برای این افزوده مقدم است. Push واقعی هنوز از این فاز نتیجه نمی‌شود.

> **مرز قرارداد Messenger v2:** توضیحات این بخش رفتار runtime فعلی را مستند می‌کنند. ممیزی تاریخی ۱۲۷ requirement در STEP 14A، تصمیم‌های freezeشده در Messenger v2 Product Decisions و نتیجه re-audit نهایی در STEP 14E ثبت شده‌اند.

آغاز گفت‌وگو و مجوز ارتباط

تکمیل P0 حساب شخصی: OTP فقط مالکیت شماره را ثابت می‌کند و تکمیل Profile، تأیید سن نیست. `GET /api/v1/me/messaging-status` فقط وضعیت خود caller را با `verificationStatus`، `effectiveAgeCategory`، `allowedActions`، `nextAction` و `version` برمی‌گرداند. سن/Block/علت محدودیت شخص دیگر از این مسیر قابل دریافت نیست. فهرست action راهنمای وضعیت جاری است، نه مجوز مقصد؛ انتخاب workspace و سیاست جاری هر عملیات همچنان بررسی می‌شوند.

برای تأیید اولیه حساب شخصیِ بدون مدرسه، اپراتور Platform دارای `platform.users.manage` و نشست معتبر از `GET /api/admin/v1/users/{userId}/communication-age` نسخه جاری را می‌گیرد و سپس `POST` همان مسیر با پسوند `/verifications` می‌زند. بدنه: `verifiedBirthDate` با تاریخ معتبر، `expectedVersion` برابر نسخه دریافتی و `reason` به‌عنوان مرجع شاهد مجاز، بدون درج PII. پاسخ `CommunicationAgeResult` تاریخ تولد ندارد. شاهد classification، append-only و Audit در transaction واحد ثبت می‌شوند؛ هیچ Credential، Enrollment، Membership، OrganizationPerson یا حساب دوم ساخته نمی‌شود.

مسیر initial فقط `Unverified` را می‌پذیرد؛ `Verified/Conflict` با `409 AGE_VERIFICATION_CONFLICT` و نسخه کهنه با `412 CONCURRENCY_CONFLICT` رد می‌شوند. حل Conflict همچنان فقط `/communication-age/resolve` با UsersManage، Owner، نشست معتبر، Reason و نسخه جاری است؛ عامل دوم حذف شده، نه permission یا حریم خصوصی. denial معمول permission ممکن است فقط HTTP403 بدون body کاربردی داشته باشد. Migration منبع اولیه `20261016050425_AddPersonalCommunicationAgeVerification` حفظ شده است.

Adult فقط با `Verified`، Adult مؤثر است. Minor کمتر از ۱۸ و Unknown طبق GAP-05 محافظت می‌شوند. برای حساب محافظت‌شده، actionهای محدود به رابطه (`DiscoverRelatedUsers`، `CreateDirectWithAuthorizedRelationship`) با کشف عمومی/تماس غریبه یکسان نیستند؛ گروه خصوصی مجاز به معنی مجوز گروه عمومی یا کانال شخصی نیست. `version` نسخه وضعیت User است، نه permission token؛ تغییر هدف/نشست/زمان می‌تواند مجوز عملیاتی را مستقل تغییر دهد.

`POST /conversations/direct` :

```
{
  "targetUserId": "<UserId مخاطب مجاز>",
  "scope": "Organization"
}
```

Scope یکی از `Personal` یا `Organization` است. در Scope سازمانی، سازمان از workspace جاری به دست می‌آید. ایجاد گفت‌وگوی تازه `201` و بازیابی زوج موجود `200` با `alreadyExisted=true` دارد.

در فضای شخصی، Direct به‌صورت سراسری و مستقل از Organization قابل شروع است. مقصد باید بر اساس `DirectMessagesAudience` فعلی خود (`Everyone`، `Contacts` یا `Nobody`) درخواست‌کننده را مجاز کند و Block دوطرفه نیز برقرار نباشد. حالت `Contacts` یک‌طرفه و از دید مقصد است: مقصد باید درخواست‌کننده را در Contacts خود ذخیره کرده باشد. برای

دو Adult تأییدشده، رابطه مدرسه، Contact یا `PersonalCommunicationGrant` شرط نیست؛ grant قدیمی فقط برای سازگاری schema/history باقی مانده است. هرگاه یک طرف Minor/Unknown باشد، رابطه جاری مجاز GAP-05 نیز الزامی است؛ شناسه یا ذخیره یک‌طرفه توسط غریبه آن را دور نمی‌زند.

در فضای سازمانی، عضویت‌ها و نقش‌ها فعال‌اند و policy فعلی این روابط را کنترل می‌کند:

نقش آغازگر	مخاطب‌های مجاز با شرایط لازم
Manager	Parent، Teacher، Manager و Student با ثبت‌نام فعال
Teacher	Manager و Teacher؛ Student کلاس مرتبط؛ Parent با رابطه آموزشی معتبر
Student	با ثبت‌نام فعال: Manager و Teacher مرتبط
Parent	Manager و Teacher مرتبط با فرزند

شناسه مخاطب ناشناخته، Blockشده یا غیرمجاز خطای عمومی `DIRECT_RECIPIENT_NOT_AVAILABLE` می‌دهد و علت داخلی افشا نمی‌شود. Organization Direct و روابط سازمانی این جدول در STEP 14B تغییر نکرده‌اند.

Privacy، Contacts و Discovery

Method	Route	کاربرد
`GET`	`/contacts?q=&favoritesOnly=&page=&pageSize=`	فهرست خصوصی Contacts؛ SavedName فقط برای مالک
`GET`	`/contacts/recent?page=&pageSize=`	مخاطبان اخیر بر اساس فعالیت Personal Direct
`POST`	`/contacts`	ذخیره idempotent با `contactUserId` و `savedName`
`PATCH`	`/contacts/{contactUserId}`	تغییر SavedName/Favorite با `expectedVersion`
`DELETE`	`/contacts/{contactUserId}`	حذف Contact بدون حذف chat، history یا Block
`GET`	`/people/search?q=&page=&pageSize=`	جست‌وجوی محدود Username/DisplayName
`POST`	`/people/resolve`	resolve دقیق Username یا Phone Number در body
`GET`	`/people/{userId}`	پروفایل عمومی حداقلی و privacy-aware
`GET`	`/messaging/privacy`	مؤثر کاربر settings

Method	Route	کاربرد
`PUT`	`/messaging/privacy`	ایجاد/ویرایش settings با optimistic concurrency

مقادیر پیش‌فرض بدون ایجاد ردیف دیتابیس عبارت‌اند از Direct= `Everyone` ، نمایش شماره= `Nobody` ، کشف با شماره= `Nobody` و دعوت گروه شخصی= `Contacts` . کشف شماره و نمایش شماره دو مجوز مستقل‌اند. Phone resolve فقط exact match است، شماره در URL قرار نمی‌گیرد و نتیجه شماره مخفی با شماره ناموجود یکسان است:

```
{
  "isDiscoverable": false,
  "profile": null
}
```

People Search به query حداقل دوکاراکتری و صفحه حداکثر ۵۰تایی محدود است. endpointهای `/people` علاوه بر rate limit، authentication دارند. DTO عمومی فقط اطلاعات حداقلی مانند `userId` ، `username` ، `displayName` ، شماره مجازشده، `isSavedContact` ، `isFavorite` و `canStartDirect` را دارد؛ داده سازمانی، نقش، سن، grant و جزئیات Privacy افشا نمی‌شوند. قرارداد کامل در سند STEP 14B ثبت شده است.

گروه، کانال و پیام

برای ایجاد گروه یا کانال از `/conversations/groups` یا `/conversations/channels` استفاده کنید:

```
{
  "clientOperationId": "<Guid ایجاد گفت‌وگو>",
  "scope": "Organization",
  "title": "گروه آموزشی",
  "description": "هماهنگی کلاس",
  "initialParticipantUserIds": []
}
```

ایجاد گروه/کانال سازمانی به workspace معتبر با نقش Manager یا Teacher نیاز دارد؛ ایجاد شخصی در workspace برای حساب معتبر انجام می‌شود و safety policy حساب کودک همچنان release gate مستقل است. افزودن عضو توسط شخص دیگر policy دعوت مخاطب را طی می‌کند، اما self-join یک Public Group دعوت ناخواسته محسوب نمی‌شود.
نقش‌های داخل گفتگو `Owner` ، `Admin` و `Member` از نقش‌های سازمانی جدا هستند. Admin فقط permissionهای صریح خود را دارد و در Channel فقط Owner یا Admin دارای `PublishMessages` می‌تواند منتشر کند. مالک باید پیش از خروج، مالکیت را با جریان درخواست انتقال و پذیرش مقصد واگذار کند.

برای ارسال `POST /conversations/{conversationId}/messages` :

```
{
  "clientMessageId": "<Guid این پیام منطقی>",
  "kind": "Text",
  "text": "سلام، زمان جلسه تأیید شد.",
  "attachmentIds": [],
```

```
  "replyToMessageId": null,
  "mentionedUserIds": [],
  "contactUserId": null
}
```

رسید `200` دارای `messageId` ، `sequence` ، `sentAtUtc` ، `status=Sent` و `version` است. `Sent` یعنی commit موفق در SQL Server؛ دریافت یا خواندن توسط مخاطب را تضمین نمی‌کند. همان `clientMessageId` را برای retry همان پیام نگه دارید؛ تغییر payload با شناسه قبلی `MESSAGE_IDEMPOTENCY_CONFLICT` می‌دهد.

برای پیام رسانه‌ای `kind` مناسب و `attachmentIds` فایل‌های آماده لازم است. Reply با شناسه پیام همان گفت‌وگو، Mention با UserId عضو مجاز و Forward با `destinationConversationId` و یک `clientMessageId` جدید انجام می‌شود؛ دسترسی به هر دو طرف و محدودیت Scope کنترل می‌شود.

ویرایش پیام `text` و `expectedVersion` می‌گیرد و فقط برای فرستنده تا مرز دقیق `SentAtUtc + 1h` مجاز است. حذف عمومی صریح با `DELETE /conversations/{id}/messages/{messageId}/for-everyone?expectedVersion=...` انجام می‌شود؛ مسیر قدیمی حذف برای سازگاری همین معنی را دارد. فرستنده همان deadline را دارد، ولی Owner یا Admin دارای `DeleteMessages` برای moderation محدود به یک ساعت نیست. حذف یک tombstone می‌سازد و متن/رسانه/دانلود عادی را می‌بندد. `DELETE .../for-me` بدون deadline فقط برای همان کاربر پیام را از Pin، Reply، Media، Search، History و File authorization مخفی می‌کند و retry آن idempotent است. واکنش‌های موجود `Like` ، `Love` ، `Laugh` ، `Wow` و `Sad` هستند.

رفتار و state گفتگو در STEP 14C

بدنه ساخت Group می‌تواند `visibility` برابر `Private/Public` ، `groupHistoryPolicy` برابر `FullHistory/FromJoin` و flags مربوط به `memberCapabilities` را بفرستد. Channel فقط FullHistory و capability صفر می‌پذیرد. در boundary ،FromJoin عضو هنگام join snapshot می‌شود و تغییر بعدی setting تاریخچه اعضای قبلی را بازنویسی نمی‌کند.

Method	Route	کاربرد
`GET`	`/conversations?filter=All&page=1&pageSize=30`	فهرست bounded؛ فیلترهای Direct/Group/Channel/Unread/Archived نیز پذیرفته می‌شوند
`GET`	`/conversations/discover?type=Group&q=...&page=1&pageSize=30`	کشف Public در context شخصی یا همان سازمان
`POST`	`/conversations/{id}/join`	عضویت داوطلبانه در Public user-managed
`PATCH`	`/conversations/{id}/settings`	Visibility/History/Capabilities با `expectedVersion`
`GET`	`/conversations/{id}/members?page=1&pageSize=30`	member list صفحه‌بندی‌شده
`PATCH`	`/conversations/{id}/members/{userId}/permissions`	permissionهای Admin با `expectedVersion`
`POST/DELETE`	`/conversations/{id}/archive`	archive/unarchive فقط برای caller

Method	Route	کاربرد
`POST/DELETE`	`/conversations/{id}/state/pin`	pin/unpin گفتگو فقط برای caller
`PUT/DELETE`	`/conversations/{id}/mute`	mute دائم یا تا timestamp صریح / UTC unmute
`POST`	`/conversations/{id}/mark-unread`	unread نمایشی بدون کاهش read cursor

فهرست گفتگو state کاربر جاری (`isArchived` , `isMuted` , `mutedUntilUtc` , `isPinned` , `isManuallyUnread`) و badge امن سازمان (`organizationId` , `organizationName`) را برمی‌گرداند. پیام جدید archive را خودکار باز نمی‌کند. Mute جلوی ذخیره پیام، unread یا Delta/SignalR را نمی‌گیرد و فقط برای notification policy آینده است. جزئیات کامل در سند STEP 14C ثبت شده است.

تاریخچه، Delta و رسید خواندن

نیاز	درخواست یا پاسخ
فهرست گفت‌وگوها	`GET /conversations` ؛ دارای `unreadCount` و cursorهای Read/Delivered
تاریخچه	`GET /conversations/{id}/messages?limit=50`
صفحه قدیمی‌تر	مقدار `nextBeforeSequence` صفحه قبلی را در `beforeSequence` بفرستید
تغییرات	`GET /conversations/{id}/changes?afterCursor=...&limit=100`
صفحه بعد تغییرات	`nextCursor` را حفظ کنید و تا `hasMore=false` ادامه دهید
اعلام دریافت	`POST /conversations/{id}/delivered` با `{"upToMessageSequence":12}`
اعلام خواندن	`POST /conversations/{id}/read` با همان شکل بدنه

عدد `12` نمونه است؛ فقط آخرین Sequence واقعاً دریافت‌شده یا خوانده‌شده را ACK کنید. cursor خواندن/دریافت عقب نمی‌رود. `MessageSequence` ترتیب پیام‌ها و cursor تغییرات ترتیب mutationها را بیان می‌کنند؛ این دو قابل جایگزینی نیستند. cursorها را opaque نگه دارید و به گفت‌وگو/جست‌وجوی دیگری منتقل نکنید.

SignalR

Hub مسیر `/hubs/messaging` دارد. علاوه بر SubscribeConversation/UnsubscribeConversation، متدهای SetTyping، SubscribePresence، Heartbeat و UnsubscribePresence در P2 وجود دارند؛ قرارداد فعلی در بخش وضعیت Messenger همین سند و Handoff P2 آمده است. event سمت کلاینت `ConversationChanged` نام دارد و اطلاعات سبک مثل `conversationId` ، `type` ، `resourceId` ، `resourceVersion` ، `cursor` ، `occurredAtUtc` و `payloadVersion` را حمل می‌کند؛ متن کامل پیام نیست.

پس از اتصال یا reconnect، دسترسی و subscriptionها را برقرار کنید و تغییرات را از آخرین cursor پردازش‌شده با REST بگیرید. صرف دریافت یک اعلان یا cursor جدید، مجوز ردکردن تغییرات بین cursor قبلی و آن اعلان نیست. بعد از پردازش موفق صفحه cursor، Delta ذخیره‌شده را جلو ببرید. تحویل اعلان می‌تواند تکراری باشد؛ `changeId` را برای تشخیص تغییر تکراری نگه دارید.

محدودیت فعلی احراز هویت Hub: `SessionAuthenticationHandler` فقط `Authorization: Bearer ...` را می‌خواند و `access_token` در query را نمی‌خواند. CORS از allowlist صریح محیط استفاده می‌کند و AllowCredentials ندارد؛ تغییر transport پشتیبانی query-token WebSocket/SSE مرورگر را اضافه نمی‌کند. REST مرورگر از `credentials: "omit"` و SignalR LongPolling از `accessTokenFactory` و `withCredentials:false` استفاده کند؛ نمونه در قرارداد فرانت‌اند است. transport و reconnect در origin واقعی باید آزمایش شوند. در قطع REST، realtime و Delta مسیر بازیابی‌اند؛ backplane و cookie-based affinity از تنظیم فعلی قابل نتیجه‌گیری نیستند.

جست‌وجو، مسدودسازی و گزارش

`GET /messages/search?q=...` از فیلترهای اختیاری `conversationId`، `senderUserId`، `kind`، `fromUtc`، `toUtc`، `cursor` و `limit` پشتیبانی می‌کند. پاسخ شامل `items`، `totalCount` و `nextCursor` است. Query و cursor را URL-encode کنید؛ نتایج بر اساس دسترسی جاری و watermark خصوصی Clear History محدود می‌شوند. `GET /conversations?q=...` نیز title گروه/کانال، هویت امن طرف Direct و عنوان Saved Messages را جست‌وجو می‌کند.

- `POST /messaging/blocks/{userId}` حساب را مسدود می‌کند؛ `DELETE` همان مسیر رفع مسدودسازی است؛ `GET /messaging/blocks` فهرست خود کاربر را می‌دهد.
- `POST /conversations/{conversationId}/messages/{messageId}/reports` بدنه `clientReportId`، `reason` و `description` می‌گیرد.
- دلیل گزارش یکی از `Harassment`، `Spam`، `Threat`، `InappropriateContent`، `Impersonation` یا `Other` است.
- `GET /messaging/reports/{reportId}` رسید گزارش متعلق به خود کاربر را می‌دهد. این API پنل رسیدگی مدیر یا تضمین اقدام خودکار روی گزارش نیست.

تکمیل Messenger در STEP 14E

Method	Route	قرارداد
`GET`	`/conversations/saved`	Saved Conversation شخصی را lazy می‌سازد؛ بار اول `201` و دفعات بعد `200` با `alreadyExisted=true`
`GET`	`/conversations?filter=Saved&q=&page=&pageSize=`	Saved را در list/filter می‌آورد؛ هر summary دارای `unreadMentionCount` است
`POST`	`/conversations/{id}/clear-history`	watermark خصوصی را تا آخرین Sequence committed جلو می‌برد و `ConversationUserStateResult` برمی‌گرداند
`GET`	`/messages/search?...&senderUserId=`	فیلتر sender را همراه فیلترهای قبلی اعمال می‌کند

Saved Messages فقط در workspace شخصی قابل resolve است، در SQL Server برای هر User یک Conversation یکتا دارد، notification تولید نمی‌کند و فقط مالک می‌تواند آن را بخواند یا در آن پیام بفرستد. متن، فایل، Search و Shared Media از pipeline عادی استفاده می‌کنند. Forward به Saved یک Message مستقل می‌سازد و تغییر یا حذف آن منبع را mutate نمی‌کند.

Clear History حذف فیزیکی Message نیست. مقدار `historyHiddenThroughSequence` در state خصوصی کاربر ثبت می‌شود و preview، Delta، Forward، Reply، File، Shared Media، Search، History و unread را برای پیام‌های قدیمی همان کاربر محدود می‌کند. تکرار درخواست idempotent است و پیام‌های بعدی دیده می‌شوند. طرف دیگر Conversation هیچ تغییری نمی‌بیند.

برای ارسال Contact Card، `kind="ContactCard"` و `contactUserId` معتبر بدهید و `text` و `attachmentIds` را خالی نگه دارید. پاسخ پیام فقط snapshot امن `userId`، `username` و `displayName` را در `contactCard` دارد؛ Phone Number برنمی‌گردد و card به‌تنهایی Contact یا Direct Conversation ایجاد نمی‌کند.

هر Message committed در Direct/Group/Channel برای participantهای فعال به‌جز فرستنده یک intent اعلان durable می‌سازد. type مؤثر هر گیرنده با اولویت `MessageMention > MessageReply > MessageReceived` تعیین می‌شود. preview Push/SMS فقط متن عمومی «پیام جدیدی در ماکان دارید.» است. Mute موقت یا دائم فقط Push/SMS را suppress می‌کند؛ Notification داخلی، خود پیام، Delta، unread و SignalR باقی می‌مانند. provider خارجی داخل transaction ارسال پیام فراخوانی نمی‌شود و retry همان `clientMessageId` اعلان تکراری ایجاد نمی‌کند.

Contact Sync و Bio عمومی افزوده نشده‌اند؛ ProfilePhoto شخصی اکنون از هویت مجاز و دانلود Bearer استفاده می‌کند و URL عمومی نیست. Typing/Presence/LastSeen در P2 قرارداد واقعی دارای privacy و TTL دارند. مرجع جاری، Handoff P2 و قرارداد عکس است. policy دقیق ساخت/عضویت Personal Group/Channel release blocker باقی مانده است.

خطاها و تلاش مجدد

خطای کاربردی به شکل ProblemDetails برمی‌گردد؛ نمونه:

```
{
  "type": "urn:makan:problem:auth-required",
  "title": "Authentication required",
  "status": 401,
  "detail": "برای ادامه باید وارد شوید.",
  "code": "AUTH_REQUIRED",
  "traceId": "<شناسه پیگیری درخواست>"
}
```

کلاینت تصمیم ماشینی را از status و `code` بگیرد و متن `detail` را برای توضیح به کاربر استفاده کند. خطاهای model binding، JSON نامعتبر یا validation خود ASP.NET Core ممکن است `errors` داشته باشند و `code` کاربردی نداشته باشند؛ مسیر fallback لازم است.

status	نمونه و اقدام
`400`	ورودی نامعتبر، نوع پاسخ یا cursor اشتباه؛ درخواست را اصلاح کنید

status	نمونه و اقدام
`401`	`AUTH_REQUIRED` ؛ توکن منقضی، revoke یا با restart نامعتبر شده است
`403`	نقش، workspace، رابطه آموزشی یا مجوز منبع کافی نیست
`404`	منبع پیدا نشد یا برای جلوگیری از افشای وجود آن پنهان شده؛ شناسه و context را بررسی کنید
`409`	تعارض lifecycle، ظرفیت، مهلت، idempotency یا نسخه پاسخ آزمون؛ `code` را تفکیک کنید
`410`	OTP یا دعوت منقضی/غیرقابل استفاده شده است
`412`	`CONCURRENCY_CONFLICT` یا `MESSAGE_EDIT_CONFLICT` ؛ داده و نسخه جاری را دوباره بگیرید
`413`	حجم فایل بیش از سقف
`415`	MIME فایل مجاز نیست
`422`	`SMS_PROVIDER_REJECTED` ؛ provider درخواست ارسال را رد کرده؛ retry کور انجام نشود
`429`	محدودیت OTP یا تلاش‌ها؛ ارسال خودکار سریع انجام ندهید
`503`	`FILE_STORAGE_FAILED` ، provider پیامک در دسترس نیست یا credential سرویس رد شده؛ خطای نشست کاربر نیست
`500`	خطای غیرمنتظره؛ `traceId` را برای بررسی سرور نگه دارید

قراردادهای retry

عملیات	شناسه یا قاعده
پذیرش دعوت	همان دعوت؛ نتیجه قبلی با `alreadyAccepted` قابل بازیابی است
شروع Direct	زوج کاربران و Scope؛ گفت‌وگوی قبلی resolve می‌شود
ساخت Group/Channel	`clientOperationId`
ارسال یا Forward پیام	`clientMessageId` در محدوده فرستنده و گفت‌وگو
گزارش پیام	`clientReportId`
شروع آزمون	`clientOperationId`
ذخیره پاسخ آزمون	`clientOperationId` هر ذخیره، همراه lease و revision موردانتظار

عملیات	شناسه یا قاعده
Finalize آزمون	`clientOperationId` و payload یکسان
انتشار نمره آزمون	`clientOperationId` و payload یکسان
ویرایش‌های مبتنی بر rowversion	پس از conflict ابتدا داده را بخوانید و تغییر را بازبینی کنید

هدر عمومی `Idempotency-Key` پیاده‌سازی نشده است. برای عملیات جدید Guid جدید و برای retry همان عملیات همان Guid و payload را نگه دارید. همه POSTها retry-safe نیستند؛ ساخت دوره، درس، کلاس یا تکلیف را با فرض idempotency تکرار نکنید. کنترل نسخه نیز با idempotency یک مفهوم نیست.

داشبوردهای نقش‌محور

داشبوردهای STEP 11A از داده رسمی موجود در SQL Server ساخته می‌شوند و برای چهار مخاطب قرارداد جدا دارند:

مخاطب	Endpoint	context لازم
Student	`GET /api/v1/dashboards/student`	workspace سازمانی با نقش فعال `Student`
Parent	`GET /api/v1/dashboards/parent`	workspace نقش `Parent` و در صورت مشاهده فرزند، child context انتخاب‌شده
Teacher	`GET /api/v1/dashboards/teacher`	workspace نقش `Teacher` و `TeacherAssignment` فعال
Manager	`GET /api/v1/dashboards/manager`	workspace نقش `Manager` همان سازمان

درخواست LearnerId، OrganizationId، body یا Role ندارد. نقش و سازمان از نشست و `AccessContext` سمت سرور resolve می‌شوند. Parent باید از `GET /api/v1/guardian/children` و `POST` `/api/v1/guardian/children/{learnerOrganizationPersonId}/select` موجود استفاده کند؛ dashboard سیستم انتخاب موازی ایجاد نمی‌کند. در نبود child انتخاب‌شده پاسخ Parent موفق اما خالی است و پس از revoke شدن رابطه، درخواست بعدی با `CHILD_CONTEXT_NOT_ALLOWED` متوقف می‌شود.

همه پاسخ‌ها `dataAsOfUtc` ، `organizationId` و `audience` دارند. `dataAsOfUtc` زمان آغاز خواندن روی سرور است و به معنی realtime یا snapshot واحد میان تمام queryها نیست. چون timezone سازمان هنوز مدل نشده است، windowهای داشبورد بر UTC تکیه دارند.

قواعد مهم مصرف پاسخ:

- Student و Parent فقط نمره‌ای را می‌بینند که release رسمی متناظر دارد؛ Draft evaluation یا grade آماده انتشار score ندارد.
- Attendance از رکورد رسمی Attendance می‌آید؛ `notRecordedCount` را Absent نمایش ندهید.
- Exam summary فقط metadata امن دارد و سؤال، option صحیح، answer key و note خصوصی در آن نیست.

- Teacher فقط کلاس‌ها و backlogهای دارای assignment معتبر خودش را می‌بیند.
- Manager summary سازمانی می‌گیرد و محتوای پیام/پاسخ خصوصی در آن وجود ندارد.
- collectionها bounded هستند؛ این API جایگزین گزارش تفصیلی یا pagination مرحله STEP 11B نیست.

نمونه empty state دانش‌آموز:

```
{
  "dataAsOfUtc": "2026-10-11T09:30:00Z",
  "organizationId": "00000000-0000-0000-0000-000000000001",
  "audience": "Student",
  "nextSession": null,
  "nearestAssignmentDeadline": null,
  "recentReleasedFeedback": [],
  "upcomingExams": [],
  "attendanceSummary": {
    "eligibleSessionCount": 0,
    "recordedCount": 0,
    "presentCount": 0,
    "absentCount": 0,
    "lateCount": 0,
    "excusedCount": 0,
    "notRecordedCount": 0
  },
  "pendingAssignmentCount": 0
}
```

فیلد AI/Copilot در قرارداد ساخته نشده است. insightهای هوشمند به STEP 10 و widgetهای Plan/Quota/Subscription مدیر به STEP 13 Commerce موکول شده‌اند. معماری و جزئیات privacy در سند STEP 11A آمده است.

گزارش‌های تفصیلی نقش‌محور

STEP 11B در کنار داشبوردهای خلاصه، APIهای report خواندنی و صفحه‌بندی‌شده را ارائه می‌کند. Dashboard برای glance سریع است؛ report برای sort، filter، history و drill-down استفاده می‌شود. مسیرهای اصلی:

- Student: `GET /api/v1/reports/student/attendance|assignments|exams|grades`
- Parent: `GET /api/v1/reports/parent/attendance|assignments|exams`
- Teacher: `GET /api/v1/reports/teacher/classes/{classId}/attendance|assignments|exams` و `GET /api/v1/reports/teacher/classes/{classId}/learners/{learnerId}`
- Manager: `GET /api/v1/reports/manager/enrollments|attendance|assessments|teacher-workload` و `GET /api/v1/reports/manager/classes/{classId}`

فهرست‌ها `dataAsOfUtc`، `pageNumber`، `pageSize`، `totalCount` و `items` دارند. پیش‌فرض صفحه ۱ با اندازه ۲۰ و سقف اندازه ۱۰۰ است. `fromUtc` / `toUtc` باید UTC باشند. sort فقط از allowlist هر report پذیرفته می‌شود و مقدار arbitrary با `REPORT_SORT_INVALID` رد می‌شود.

سازمان، نقش Student/Parent/Teacher/Manager و child context از session و `AccessContext` سمت سرور می‌آیند. `classId`، `courseId` و `academicPeriodId` فقط selector هستند. Teacher باید assignment فعال همان کلاس داشته باشد و Parent باید child context معتبر انتخاب کرده باشد. Guid خارج محدوده ممکن است برای جلوگیری از افشای منبع `REPORT_RESOURCE_NOT_FOUND` بدهد.

در گزارش حضور، نبود رکورد رسمی `NotRecorded` است و به `Absent` تبدیل نمی‌شود. پاسخ attendance علاوه بر page، یک `summary` روی کل مجموعه فیلترشده با شمارنده‌های eligible/recorded/status/not-recorded دارد. نمره Student/Parent فقط پس از release رسمی نمایش داده می‌شود. Parent فقط guardian-visible feedback را می‌بیند. گزارش آزمون هیچ سؤال، گزینه صحیح، answer key، پاسخ learner یا note خصوصی ندارد. عملکرد Manager درصد نرمال‌شده `score / maxScore` را برای Assignment و Exam جدا محاسبه می‌کند و سقف سراسری ۲۰ یا GPA فرض نمی‌کند.

نمونه صفحه خالی معتبر:

```
{
  "dataAsOfUtc": "2026-10-11T09:30:00Z",
  "pageNumber": 3,
  "pageSize": 20,
  "totalCount": 0,
  "items": []
}
```

جزئیات endpointها، allowlist مرتب‌سازی و privacy در سند معماری STEP 11B آمده است. Export، گزارش زمان‌بندی‌شده، cache و AI در این مرحله وجود ندارند.

AI v1: جریان کد و وضعیت واقعی

۱۲ عملیات عمومی زیر `/api/v1/ai` موجودند؛ Generic Chat و یازده عملیات نقش‌محور برای Student/Parent/Teacher/Manager. route، DTO و پاسخ هرکدام در مرجع API آمده است. `AiService:Enabled` پیش‌فرض false و core آموزشی مستقل است؛ RAG ingestion/source lifecycle deferred باقی مانده.

```
AiController
-> AuthenticatedSession / IAccessContextResolver
-> AiChatService / AiRoleContextService
-> Role Context Builder -> EfAiScopeStore / authorized read services
-> minimum educational DTO
-> HmacAiServiceCredentialProvider -> AiServiceClient
-> strict v1 validation -> AiResponseGrounding -> public response
```

مجوز قبل از تماس خارجی بررسی می‌شود. Student از Person/Enrollment فعال، Parent از GuardianRelation و فرزند انتخابی، Teacher از assignment و Enrollment همان کلاس و Manager از سازمان جاری resolve می‌شوند. PlatformOwner مجوز خودکار Manager آموزشی ندارد. User Bearer، OTP، private message و EF graph وارد context AI نمی‌شوند.

JWT سرویس HS256 دارای audience=makan-ai، issuer/subject=makan-backend، kid و scope=makan.ai.invoke است؛ TTL پیش‌فرض ۱۲۰ و بازه ۳۰..۳۰۰ ثانیه. credential در محیط تأمین و user token forward نمی‌شود.
`requestId/correlationId` و `X-AI-Contract-Version=v1` باید دقیق برگردند؛ source/entity، unknown field خارج context، شناسهٔ نادرست یا MaxScore ناسازگار پاسخ معتبر نیست.

بودجه `AiService:TimeoutSeconds` پیش‌فرض ۳۵s، بازه ۱..۱۸۰ و شامل خواندن body است. caller cancellation با timeout داخلی فرق دارد. retry خودکار POST نداریم؛ circuit process-local پس از سه failure پیش‌فرض ۳۰s باز می‌شود.
429→AI_RATE_LIMITED، 503/connection/model unavailable→AI_SERVICE_UNAVAILABLE،
504→AI_SERVICE_TIMEOUT و malformed/grounding→AI_RESPONSE_INVALID؛ body خام Provider به مصرف‌کننده منتقل نمی‌شود. public Retry-After تضمین نیست. قرارداد خطا.

`AiServiceTelemetry` از ActivitySource و metricهای requests/duration/circuit.rejections با label محدود استفاده می‌کند؛ prompt/body/JWT/key لاگ نمی‌شود. ابزار رسمی opt-in در `tools/ai/MakanApp.AiIntegrationProbe` شواهد sanitizeشده می‌گیرد؛ در این بازبینی تماس واقعی AI اجرا نمی‌شود.

Service Auth واقعی در شواهد پیشین PASS شده، ولی Parent Child Summary هنوز Provider Timeout دارد و Full-12 واقعی Deferred است. Fake-AI/negative authorization/grounding محلی، این مانع را رفع‌شده اعلام نمی‌کند؛ وضعیت جاری و Consolidation مرجع تفکیک شواهد هستند.

نقشه persistence و مرز تراکنش‌ها

MakanDbContext و مدل طراحی فعلی 107 جدول دارند؛ 52 migration در مخزن است. برای CHECK، index، PK/FK و rowversion به فرهنگ جاری دیتابیس رجوع کنید؛ این شمارش تأیید schema یک سرور نیست.

schema	تعداد جدول	گروه داده
`academic`	9	قراردادهای حوزه؛ جزئیات در فرهنگ دیتابیس
`assessment`	18	قراردادهای حوزه؛ جزئیات در فرهنگ دیتابیس
`commerce`	10	قراردادهای حوزه؛ جزئیات در فرهنگ دیتابیس
`guardian`	1	قراردادهای حوزه؛ جزئیات در فرهنگ دیتابیس
`identity`	7	قراردادهای حوزه؛ جزئیات در فرهنگ دیتابیس
`messaging`	42	قراردادهای حوزه؛ جزئیات در فرهنگ دیتابیس
`notification`	4	قراردادهای حوزه؛ جزئیات در فرهنگ دیتابیس
`organization`	5	قراردادهای حوزه؛ جزئیات در فرهنگ دیتابیس
`platform`	9	قراردادهای حوزه؛ جزئیات در فرهنگ دیتابیس
`reports`	1	قراردادهای حوزه؛ جزئیات در فرهنگ دیتابیس

schema	تعداد جدول	گروه داده
`storage`	1	قراردادهای حوزه؛ جزئیات در فرهنگ دیتابیس

سه لایه تضمین درستی داده

1. Domain قوانین محلی و تغییر وضعیت entity را کنترل می‌کند؛ مثل معتبر بودن نمره و غیرقابل ویرایش بودن نسخه منتشرشده.
2. Application context و چند منبع را کنار هم بررسی می‌کند؛ مثل معلم مجاز کلاس، enrollment معتبر و موعد تحویل.
3. SQL Server با transaction، CHECK، unique/filtered index، FK و rowversion از invariant پایدار محافظت می‌کند. این لایه برای رقابت دو درخواست ضروری است؛ صرف بررسی سرویس کافی نیست.

FKهای مرکب مانند `(OrganizationId, ClassId)` تطابق بافت را الزام می‌کنند. این FKها حق خواندن یک endpoint را اثبات نمی‌کنند. filtered unique indexها معمولاً یکتایی رابطه فعال را نگه می‌دارند تا سابقه پایان‌یافته حفظ شود. CHECK SQL نیز فقط محدودیت تعریف‌شده خود را تضمین می‌کند؛ lifecycle کامل محصول یک CHECK واحد نیست.

سه چیز متفاوت‌اند: revision، rowversion و operationId

ابزار	سؤال پاسخ‌داده‌شده	رفتار مصرف‌کننده
SQL `rowversion`، در API به شکل Base64	آیا ردیف از آخرین خواندن من عوض شده؟	نسخه response جاری را برای write بعدی بفرستید؛ در412 دوباره بخوانید
`expectedRevisionNumber` / `expectedAnswerSetVersion`	آیا مجموعه/پاسخ آزمون همان revision مورد انتظار است؟	receipt آخرین ذخیره پذیرفته‌شده را نگه دارید؛ conflict را با merge/refetch حل کنید
`clientOperationId` / `clientMessageId` / `clientReportId`	آیا این همان عملیات قبلی است که پاسخ آن گم شده؟	در retry همان شناسه و همان payload؛ برای عملیات تازه شناسه تازه
`writeLeaseVersion`	آیا نشست جاری با نسخه درست حق نوشتن روی تلاش را دارد؟	acquire/transfer طبق API؛ داشتن Bearer جای lease را نمی‌گیرد

rowversion زمان رویداد نیست. پارامتر expected version در body/query همان endpoint قرار دارد؛ هدر عمومی If-Match یا Idempotency-Key برای همه APIها وجود ندارد. همه POSTها retry-safe نیستند.

نمونه‌های واقعی تراکنش و locking

عملیات	مرز واقعی	نکته
Verify OTP	`IdentityService` + `EfIdentityStore`، Serializable	مصرف challenge و ساخت account/session در یک عملیات تراکنشی؛ خطای verify نیز وضعیت تلاش ناموفق را ثبت می‌کند

عملیات	مرز واقعی	نکته
قبول دعوت	`OrganizationService` + `EfOrganizationStore`	Invitation برای حساب مقصد با `UPDLOCK/HOLDLOCK` خوانده و وضعیت مجدداً بررسی می‌شود
ثبت‌نام/پایان ثبت‌نام	`AcademicService` + `EfAcademicStore`، Serializable	قفل کلاس و بازخوانی ظرفیت/رابطه فعال؛ جلوگیری از مصرف آخرین ظرفیت توسط دو درخواست
save answer / finalize	`ExamAnswerService/ExamFinalizationSer` `vice` + store متناظر	قفل تلاش، مجوز مالک، session/lease/deadline و version پیش از write/commit
mutation پیام	`EfMessagingStore` و فایل‌های partial آن	داده پیام، change و outbox در مرز مربوط به mutation پایدار می‌شوند
dispatch اعلان	`MessagingRealtimeOutboxProcessor`	claim در transaction کوتاه؛ تماس notifier پس از commit claim انجام می‌شود

ارسال SMS در `RequestOtpAsync` قبل از ذخیره challenge است و در transaction طولانی SQL انجام نمی‌شود. اگر provider موفق شود ولی ذخیره SQL شکست بخورد، SMS رسیده الزاماً challenge قابل تأیید در backend ندارد. retry کور ارسال پیامک را پیشنهاد نکنید. storage فایل هم transaction مشترک اتمیک با SQL Server ندارد؛ مسیرهای جبران و وضعیت FileAsset در `StorageService` بررسی شوند.

migration و محیط توسعه

migration 52 تا `20261024184613_RemovePlatformMfa` مدل فعلی را می‌سازند؛ startup Migrate/EnsureCreated اجرا نمی‌کند. MakanDbContextFactory از ConnectionStrings__MakanDatabase می‌خواند. تغییر schema با principal جدا و artifact SQL بررسی‌شده انجام شود؛ tools/production/export-migration-sql.ps1 منطق batch لازم را نگه می‌دارد. Migration حذف عامل دوم تولید شده است؛ اجرای آن روی دیتابیس اصلی یا سرور مجوز جدا می‌خواهد و در این مرحله انجام نشده است.

برای تغییر schema ابتدا entity و configuration را کامل کنید، سپس migration بسازید و فایل‌های Up/Down و snapshot را بررسی کنید. فرمان زیر **الگوی توسعه** است؛ نام migration را متناسب تغییر واقعی انتخاب کنید و مقصد development را صریح تنظیم کرده باشید:

```
dotnet tool restore
dotnet tool run dotnet-ef migrations add DescribeActualSchemaChange --project
MakanApp.Infrastructure/MakanApp.Infrastructure.csproj --startup-project
MakanApp.Infrastructure/MakanApp.Infrastructure.csproj
```

نام نمونه بالا migration موجود نیست. این مرحله مستندسازی migration نساخته است. اجرای migration روی Production یک عملیات جدا با artifact بررسی‌شده و برنامه backup/rollback است؛ سند استقرار مسیر آن را توضیح می‌دهد.

شش مسیر واقعی برای مطالعه کد از ابتدا تا انتها

۱. درخواست و تأیید OTP

این مسیر مربوط به Real با true/false است؛ در Mock مجاز، ساخت/ذخیره verify challenge/ اجرا نمی‌شود ولی اعتبارسنجی حساب و صدور نشست عادی باقی است.

```
AuthController.RequestOtp
-> IdentityService.RequestOtpAsync
-> NormalizePhoneNumber + persisted rate/resend checks
-> IOtpDeliveryProvider.RequestOtpAsync
-> MeliPayamakOtpProvider OR DevelopmentOtpProvider
-> normalize returned code + IdentityCryptography.HashOtp
-> OtpChallenge + EfIdentityStore.SaveChangesAsync
-> RequestOtpResult (202)
```

provider مدیریت‌شده کد را تولید می‌کند؛ backend همان کد را hash می‌کند و کد موازی محلی نمی‌سازد. latest challenge پس از پاسخ قابل استفاده provider supersede می‌شود و زمان expiry از زمان بعد از تحویل provider محاسبه می‌شود. در verify، `OtpChallenge.Verify` مصرف/انقضا/تعداد تلاش را تعیین می‌کند؛ credential موجود User را resolve می‌کند یا User تازه ساخته می‌شود، سپس UserSession با hash توکن ساخته می‌شود. token خام فقط در response ساخت نشست تحویل داده می‌شود.

مسیر مطالعه: AuthController → IdentityService → OtpChallenge → IdentityAbstractions → EfIdentityStore → IdentityEndpointsTests.

۲. آخرین ظرفیت کلاس

```
ClassesController.EnrollLearner
-> AcademicService.EnrollLearnerAsync
-> resolve Organization / Manager
-> Serializable transaction -> lock Class
-> validate learner OrganizationPerson + existing active enrollment + capacity
-> Enrollment.CreateActive / store.Add
-> SaveChanges + Commit -> EnrollmentResult
```

هدف قفل کلاس این است که دو درخواست آخرین جای خالی را همزمان آزاد نبینند. یکتایی enrollment فعال در SQL نیز مکمل کنترل سرویس است. پایان enrollment سابقه را حذف نمی‌کند؛ status و زمان پایان ثبت می‌شوند. API ثبت‌نام LearnerOrganizationPersonId می‌گیرد، نه شناسه حساب User.

مسیر مطالعه: ClassesController → AcademicService → Enrollment → EfAcademicStore → EnrollmentConfiguration → AcademicEndpointsTests.

۳. تحویل تکلیف و انتشار نمره

انتشار تکلیف، نسخه و گیرندگان رسمی آن را مشخص می‌کند. Student draft خودش را می‌سازد/بازیابی می‌کند، متن و فایل مجاز را ذخیره می‌کند و FinalSubmit می‌زند. پاسخ موفق تحویل با حفظ clientOperationId قابل بازیابی است؛ ذخیره draft به معنی Submitted نیست. فایل ابتدا upload می‌شود و attach مجاز به submission مرحله جداست.

ارزیاب از `EvaluationService` یک `EvaluationRevision` draft می‌سازد، سپس release رسمی `GradeRelease` را ایجاد می‌کند. Student و Parent projection نتیجه منتشرشده خود را می‌گیرند؛ TeacherPrivateNote در خروجی عمومی قرار نمی‌گیرد. اصلاح نمره منتشرشده revision اصلاحی و دلیل می‌خواهد و سابقه رسمی را بی‌صدا overwrite نمی‌کند.

مسیر مطالعه: SubmissionsController → SubmissionService → SubmissionAttempt → EfSubmissionStore؛ سپس EvaluationService → EfEvaluationStore → EvaluationLifecycleEndpointsTests.

۴. پاسخ آزمون تا نتیجه رسمی

```
ExamService: published version
-> ExamAttemptService: freeze questions/order + server deadline
-> ExamAnswerService: lease + expected revision + accepted AnswerRevision
-> ExamFinalizationService: final snapshot + receipt
-> ExamGradingService: objective scoring + descriptive review
-> ReadyForRelease -> ExamGradeRelease
-> authorized Student/Guardian result
```

هر تلاش به نسخه و مجموعه سؤال خودش وابسته است. زمان دستگاه مرجع مهلت نیست. ذخیره پاسخ، revision جدید قابل پیگیری تولید می‌کند؛ finalize مجموعه پاسخ پذیرفته‌شده را تثبیت می‌کند و receipt دارد. حق نوشتن به نشست مربوط است تا دو دستگاه بی‌صدا پاسخ همدیگر را overwrite نکنند. تصحیح تستی و تکمیل ارزیابی با انتشار عمومی یک مرحله نیستند. وجود deadline به معنی worker نهایی‌سازی خودکار نیست؛ auto-finalize در این نسخه وجود ندارد.

مسیر مطالعه: ExamsController → ExamAttemptService → ExamAnswerService → ExamFinalizationService → ExamGradingService و ExamFinalizationEndpointsTests.

۵. ارسال پیام، Delta و SignalR

```
ConversationsController.SendMessage
-> MessagingService + communication/participant policy
-> EfMessagingStore: message + stable change event + outbox
-> commit -> MessageReceiptResult
-> MessagingRealtimeOutboxDispatcher
-> MessagingRealtimeOutboxProcessor -> SignalRMessagingRealtimeNotifier
-> ConversationChanged notification -> client fetches Delta
```

`clientMessageId` اثر retry را محدود می‌کند. receipt HTTP نتیجه commit است؛ SignalR می‌تواند دیرتر برسد یا اتصال قطع باشد. notification یک اشاره به تغییر است، نه payload کامل پیام خصوصی. Delta از `MessagingChangeEvent` مرجع بازیابی تغییرات بعد از cursor است. cursor delivered/read وضعیت خود کاربر است؛ از آن رسید خواندن همه مخاطبان نتیجه نگیرید.

در P0، `MessagingRealtimeAudienceResolver` در scope تازه، `MessagingService.GetConversationChangesAsync` با limit=1 را برای همان cursor/type/resource اجرا می‌کند؛ مجوز container به‌تنهایی کافی نیست. حساب معلق، نشست منقضی/لغوشده، نقش یا سازمان نامعتبر، DeleteForMe، Clear، FromJoin و رویداد خصوصی فیلتر می‌شوند. Client فقط پس از پردازش صفحهٔ Delta checkpoint را جلو ببرد، نه صرف دریافت SignalR. پاسخ 429 جست‌وجو Retry-After دارد و در CORS قابل خواندن است؛ quota یا provider OTP تغییر نکرده است.

processor در MessagingRealtimeOutboxDispatcher.cs تعریف شده است. claim با `UPDLOCK/READPAST/ROWLOCK`، batch حداکثر۵۰ و lease۳۰ثانیه‌ای انجام می‌شود؛ ارسال notifier خارج transaction claim است. از retry، lease و crash باید امکان ارسال تکراری notification را نتیجه گرفت؛ مصرف‌کننده با شناسه/نسخه و Delta باید تکرار را تحمل کند. backplane چند instance در این نسخه ثبت نشده است.

مسیر مطالعه: EfMessagingStore → MessagingService → ConversationsController و فایل‌های partial → MessagingSyncEndpointsTests → SignalRMessagingRealtimeNotifier → dispatcher/processor.

۶. یک گزارش نقش‌محور

`ReportsController` query را bind می‌کند. `DetailedReportService` context و نقش مسیر، sort allowlist، page size و بازه UTC را بررسی می‌کند. `EfDetailedReportReadService` query/projection را روی جداول رسمی همان scope اجرا می‌کند. تغییر وضعیت entity یا جدول گزارش تازه‌ای لازم نیست. این تفاوت یک read use case با mutation است.

Student/Parent فقط release رسمی نتیجه را می‌بینند. مقدار `dataAsOfUtc` زمان شروع خواندن است و مجموعه چند query را به snapshot اتمیک تبدیل نمی‌کند. برای teacher-workload بازه زمانی فقط completedSessionCount را محدود می‌کند؛ مراجعه به کارت دقیق endpoint برای معنی فیلترها ضروری است.

مسیر مطالعه: ReportsController → DetailedReportService → DetailedReportAbstractions → EfDetailedReportReadService → DetailedReportServiceTests.

روش توسعه و عیب‌یابی برای عضو جدید تیم

یک قابلیت کوچک را چطور اضافه یا اصلاح کنیم؟

1. مسئله، endpoint/رفتار، نقش مجاز و scope منبع را دقیق بنویسید. AGENTS و اسناد مرتبط را بخوانید؛ وضعیت Git و تغییرات از قبل موجود را بررسی کنید.
2. مسیر Controller → service → domain → port → store را پیدا کنید. برای یک قانون خالص، تست Unit؛ برای API/SQL/مجوز/رقابت، تست Integration انتخاب کنید.
3. قرارداد request/result و خطاها را تعیین کنید. قبل از افزودن فیلد actor/organization از کلاینت، بررسی کنید آیا باید از context معتبر سرور resolve شود.
4. قانون را در لایه مناسب تغییر دهید. Controller فقط transport را map کند. I/O و EF در Infrastructure؛ async و CancellationToken تا انتهای عملیات عبور کنند.

5. اگر persistence تغییر کرد، constraints، configuration و migration را کامل و بررسی کنید. برای concurrency، فقط check اولیه ننویسید؛ race condition و transaction/locking را مشخص کنید.

6. retry را بررسی کنید: payload یکسان و operationId یکسان باید چه پاسخ/اثر داشته باشند؟ همان شناسه با payload متفاوت چه خطایی می‌دهد؟ کار شبکه‌ای طولانی داخل transaction نباشد.

7. test، build، restore و validation نهایی را اجرا کنید؛ سپس diff، اسناد و artifactهای لازم را بررسی و فقط فایل‌های مرتبط را commit کنید. endpoint جدید را در مرجع و مسیر استفاده‌اش را در راهنما ثبت کنید.

تست‌ها چه چیزی را ثابت می‌کنند؟

مجموعه	نمونه فایل واقعی	مورد اثبات
Unit	`OtpChallengeTests`، `ExamAnswerModelTests`، `DetailedReportServiceTests`	قانون خالص، normalization، state و orchestration بدون ادعای رفتار SQL
Integration	`AcademicEndpointsTests`، `EvaluationConcurrencyEndpointsTests`، `ExamAnswerEndpointsTests`	HTTP واقعی، SQL Server، host، constraints، مجوز، نسخه، retry و transaction
Architecture	`OnionDependencyTests`، `LayerPlacementTests`	جهت وابستگی و محل درست لایه‌ها
Hosting	`ProductionHostingTests`	رفتار تنظیمات میزبان در Production در چارچوب تست؛ اثبات استقرار IIS واقعی نیست

`MakanAppWebApplicationFactory` از Windows LocalDB و نام اختصاصی `MakanApp_MessagingSafety_IntegrationTests_Step7E` استفاده می‌کند. `SqlServerCollection` fixture مشترک مجموعه را تعریف می‌کند. initialize دیتابیس تست را پاک و migrationها را اجرا می‌کند؛ dispose نیز همان پایگاه ایزوله را پاک می‌کند. از این نام برای داده شخصی/توسعه استفاده نکنید و همزمان دو اجرای مستقل کل suite را روی همان fixture شروع نکنید.

fixture زمان کنترل‌شده، storage موقت و fault injection برای SMS/file/realtime دارد. از همین ابزارها برای تست زمان/خطای provider استفاده کنید؛ ساعت سیستم را دستکاری یا SMS واقعی در تست ارسال نکنید. تست seedهای داخلی fixture را برای ساخت نقش‌ها و روابط استفاده می‌کند؛ این helperها endpoint محصول نیستند.

فرمان‌های متمرکز زیر فقط برای بازخورد سریع هنگام توسعه‌اند؛ قبل از تحویل، suite کامل قواعد مخزن اجرا شود:

```
dotnet test tests/MakanApp.UnitTests/MakanApp.UnitTests.csproj --configuration Debug --filter
FullyQualifiedName~ExamAnswerModelTests
dotnet test tests/MakanApp.IntegrationTests/MakanApp.IntegrationTests.csproj --configuration Debug --filter
FullyQualifiedName~AcademicEndpointsTests
dotnet test tests/MakanApp.ArchitectureTests/MakanApp.ArchitectureTests.csproj --configuration Debug
```

مسیر عیب‌یابی یک درخواست ناموفق

نشانه	نقطه شروع	اقدام بعدی
host بالا نمی‌آید	Program/DependencyInjection و setting name	خطای validation تنظیمات را بررسی کنید؛ secret را log نکنید
400 با errors	binding و request schema	نوع، فیلد، enum، UUID و format زمان را پیش از service کنترل کنید
401 AUTH_REQUIRED	SessionAuthenticationHandler/IdentityService	expiry، hash key، header و revoke نشست
403 پس از انتخاب role	use case و AccessContext	role، membership، workspace، انتساب کلاس یا رابطه child
404 برای شناسه‌ای که می‌دانید وجود دارد	query scope و سیاست پنهان‌سازی	جست‌وجوی وجود جهانی منبع را جای authorization نگذارید
409	code اختصاصی و state جاری	تضاد lifecycle، ظرفیت، deadline، lease یا operationId متفاوت را تفکیک کنید
412	expected rowversion و store	نسخه جاری را بخوانید؛ داده ورودی کاربر را برای بازبینی حفظ کنید
503 در OTP یا upload	adapter/options	اعتبار سرویس، timeout، مسیر مطلق و permission؛ با500 عمومی یکی نیست
پیام ثبت شده ولی event نرسیده	outbox، notifier و اتصال Hub	receipt/Delta را مرجع قرار دهید و backlog/ خطا را بررسی کنید
تست SQL شکست می‌خورد	fixture/LocalDB/restore	مشکل محیط را از regression کد جدا گزارش کنید

برای خطای ناشناخته، traceId پاسخ را با log متناظر دنبال کنید. OTP، token، connection string و URL حاوی ServiceToken را در گزارش مشترک قرار ندهید. در EF store مربوط SQL exception، query scope و تغییرات tracked را بررسی کنید؛ نسخه و شناسه مصنوعی مثال‌های این کتاب، داده واقعی پایگاه شما نیستند.

مرز میزبانی و موارد لازم برای تحویل عملیاتی

میزبان migration خودکار ندارد؛ /health و /health/live فقط process و /health/ready شامل config/SQL/Storage است، نه key ring حذف‌شده یا providerهای اختیاری. CORS از AllowAnyOrigin می‌آید؛ true با wildcard بدون credentials و false با allowlist معتبر. Production/Staging برای Swagger فعال‌سازی صریح می‌خواهند. SignalR مرورگر LongPolling با Authorization header و withCredentials:false است؛ query-token پشتیبانی نمی‌شود.

UseForwardedHeaders با KnownProxies صریح و ForwardLimit=1 وجود دارد؛ اعتماد عمومی به proxy مجاز نیست. runtime SQL principal و migration principal جدا باشند. Backup دیتابیس بدون Storage و custody کلید Identity و Secretهای فعال کافی نیست. هیچ موفقیت تست یا دستور این کتاب اثبات IIS واقعی، تحویل recovery، SMS محیط هدف یا آماده‌بودن چند instance نیست. HTTP این انتشار استثنای صریح مالک است و رمزگذاری ندارد.

تست و عیب‌یابی

اجرای validation

```
dotnet restore MakanApp.sln
dotnet build MakanApp.sln --configuration Debug --nologo
dotnet test MakanApp.sln --configuration Debug --nologo
```

برای تمرکز بر یک مجموعه:

```
dotnet test tests/MakanApp.UnitTests/MakanApp.UnitTests.csproj --configuration Debug --nologo
dotnet test tests/MakanApp.ArchitectureTests/MakanApp.ArchitectureTests.csproj --configuration Debug --nologo
dotnet test tests/MakanApp.IntegrationTests/MakanApp.IntegrationTests.csproj --configuration Debug --nologo
```

Integration از SQL Server LocalDB و پایگاه ثابت `MakanApp_MessagingSafety_IntegrationTests_Step7E` استفاده می‌کند؛ fixture آن را در شروع حذف و با migration بازسازی می‌کند و در پایان حذف می‌کند. این نام فقط برای تست است؛ داده شخصی/توسعه را در آن نگذارید و دو اجرای همزمان این suite را روی یک LocalDB شروع نکنید. test fixture اتصال runtime را برای تست override می‌کند. فایل‌ها هم در پوشه موقت مختص تست نوشته می‌شوند.

Unit قوانین مدل و سرویس‌های قابل تست را بررسی می‌کند. Integration رفتار migration، SQL Server، HTTP، مجوز، همزمانی، idempotency و SignalR را پوشش می‌دهد. Architecture جهت وابستگی و عدم ورود EF Core/ASP.NET Core به لایه‌های داخلی را کنترل می‌کند.

عیب‌یابی سریع

نشانه	بررسی
برنامه با خطای connection string شروع نمی‌شود	تنظیم `ConnectionStrings:MakanDatabase` در محیط واقعی اجرا
`/health` سالم ولی API داده خطا می‌دهد	health فعلی probe دیتابیس نیست؛ اتصال و migration را بررسی کنید
`Invalid object name`	مقصد migration با مقصد runtime یکسان باشد و migrationهای جاری اعمال شده باشند
Swagger در دسترس نیست	در Production پیش‌فرض خاموش است؛ 404 می‌تواند رفتار صحیح باشد
OTP نمی‌رسد	Provider واقعی/secret، Unavailable/Development، محدودیت ارسال و خطای امن provider را بررسی کنید؛ تست خودکار SMS پولی نمی‌فرستد
پس از restart پاسخ `401` می‌گیرید	کلید موقت توسعه عوض شده؛ دوباره وارد شوید یا کلید توسعه را امن و پایدار تنظیم کنید

نشانه	بررسی
فضای سازمانی در فهرست نیست	عضویت، سازمان و نقش باید فعال باشند؛ ایجاد حساب جای provisioning را نمی‌گیرد
والد کلاس‌ها را نمی‌بیند	`/academic/classes` برای Parent ارائه نشده؛ برنامه/آزمون/نتیجه از مسیرهای مجاز با context فرزند مصرف شود
نمره بعد از Save دیده نمی‌شود	Draft و Release جدا هستند
پاسخ آزمون ذخیره شده ولی رسید نهایی نیست	Finalize مستقل است و باید پیش از deadline موفق شود
ویدئو یا صدا `415` می‌دهد	`Storage:AllowedContentTypes` پیش‌فرض این MIMEها را ندارد
SignalR مرورگر `401` می‌دهد	محدودیت هدر در مقابل `access_token` Query و transport مورد استفاده
مرورگر خطای CORS دارد	AllowAnyOrigin و overrideهای واقعی را بررسی کنید؛ در فهرست دقیق origin/redirect/preflight لازم است. credentials:omit و LongPolling بدون credentials؛ Postman شاهد مرورگر نیست
Integration شکست می‌خورد	دسترس‌پذیری LocalDB، اجرای موازی suite و مجوز ایجاد دیتابیس تست

محدودیت‌ها و مسیر مطالعه

مرز قابلیت موجود

مرز نسخه جاری: Peer Read مجاز Web Push/FCM، Direct و Typing/Presence در کد وجود دارند. آزمون واقعی scale-out، Push و رسانه P3 gate مستقل دارند. پاراگراف تاریخی زیر قابلیت جاری را تعیین نمی‌کند؛ کاتالوگ، وضعیت بالای سند و Handoffهای P0/P1/P2 مبنا هستند.

قابلیت جاری شامل onboarding، Identity/refresh/session سازمان و Guardian با رضایت، Class Lesson، Directory/roster و attachment، آموزش/Backoffice، Commerce lifecycle، Notification، export، Messenger P0/P1/P2، Assessment و دوازده API AI اختیاری است. DOB و عکس self موجود و MFA/step-up حذف شده‌اند. اشتراک فردی، Learning Profile، Badge، AI/Source quota و Educational Source/RAG lifecycle در مدل V2 همچنان پیشنهاد یا deferred هستند. real Payment provider، Push دستگاه، AV/CDR، scale-out، P3 media کامل و پذیرش عملیاتی مستقل‌اند؛ auto-finalize آزمون و Ready-orphan reaper ساخته نشده‌اند.

تست Integration داده لازم سناریوها را خودش می‌سازد. برای آزمودن دستی کل جریان سازمانی روی دیتابیس تازه، آماده‌سازی داده اولیه یک کار جدا و نیازمند روش مصوب است؛ این راهنما seed یا SQL تولیدی ارائه نمی‌کند.

دیتابیس و منابع کد

Persistence با `MakanDbContext` و schemaهای `identity`، `organization`، `guardian`، `academic`، `assessment`، `storage` و `messaging` سازمان‌دهی شده است. روابط هم‌سازمانی با FKهای ترکیبی، یکتایی وضعیت فعال با filtered unique index و تغییرات حساس با `rowversion` و transaction محافظت می‌شوند. Outbox پیام‌رسانی اعلان را پس از commit کسب‌وکاری ارسال می‌کند.

برای جزئیات مدل، راهنمای ساختار دیتابیس را کنار migrationها بخوانید؛ بازبینی جاری آن 107 جدول و 52 migration را از مدل طراحی EF پوشش می‌دهد. تطابق schema سرور باید جدا بررسی شود.

مسیر مطالعه یک جریان واقعی آزمون:

1. ExamsController و ExamGradingsController برای قرارداد HTTP.
2. ExamFinalizationService، ExamAnswerService، ExamAttemptService، ExamService و ExamGradingService برای use case.
3. مدل‌های Assessment برای lifecycle و قوانین دامنه.
4. ExamGradingAbstractions برای مرز persistence.
5. پیاده‌سازی Assessment و EF configurationها.
6. Migrationها برای ساختار SQL Server.
7. ExamGradingEndpointsTests و سایر تست‌های آزمون برای نمونه درخواست و سناریوهای مجاز/غیرمجاز.

برای قرارداد دقیق هر عملیات به مرجع API و برای قواعد کلی مخزن به AGENTS.md مراجعه کنید.

نگهداری همین راهنما و مرجع

sync_repository_docs.py به‌طور پیش‌فرض فقط کاتالوگ جاری و inventory را بازتولید می‌کند و برچسب/نتایج اسناد تاریخی را عوض نمی‌کند. --refresh-historical-notices فقط برای بازبینی صریح برچسب‌هاست؛ runbookهای جاری operations/dev از آن مستثنا هستند. تغییر رفتار AI، worker، OTP و مجوزها باید در قالب قرارداد فرانت و راهنمای دستی هم اعمال شود؛ OpenAPI به‌تنهایی policy محیط را نشان نمی‌دهد.

راهنمای آموزشی Markdown دستی نگه‌داری می‌شود؛ مرجع API، مدل دیتابیس و README قرارداد مصرف از شواهد قابل بازتولید تولید می‌شوند. HTML/PDF هر چهار سند خروجی جداگانه دارند؛ خروجی‌های دیتابیس محلی/ignored هستند. تغییر DTO، route، مجوز، option، DI یا Migration باید در متن و metadata منعکس و خروجی‌ها دوباره ساخته شوند؛ رندر تنها جای استخراج از کد را نمی‌گیرد.

برای تولید خروجی‌ها ابزار مستندسازی مستقل از build محصول است. از ریشه مخزن با Python و مرورگر Edge اجرا کنید؛ وابستگی‌های Python فقط ابزار Markdown/PDF هستند:

```
python -m pip install --target .artifacts/docs-python -r tools/docs/requirements.txt
dotnet run --project tools/docs/MakanApp.DocumentationSnapshot/MakanApp.DocumentationSnapshot.csproj --
configuration Debug -- .artifacts/documentation-snapshot
python tools/docs/sync_repository_docs.py --snapshot .artifacts/documentation-snapshot --reviewed-date 2026-10-
```

```
18
python -X utf8 tools/docs/build_backend_docs.py --deps .artifacts/docs-python --pdf
```

ابزار Markdown را به HTML مستقل RTL تبدیل می‌کند، هر HTML را با Edge مخفی چاپ می‌کند و PDF دارای شماره صفحه/نشانک می‌سازد. ورودی و خروجی‌ها هر دو در `docs` هستند؛ فایل‌های موقت، گزارش audit و تصاویر کنترل در `.artifacts/backend-handbook` می‌مانند. برای HTML تنها، `--pdf` را حذف کنید. ابزار به SQL Server وصل نمی‌شود و schema یا داده محصول را تغییر نمی‌دهد.

پس از تولید، فهرست، لینک داخلی، فارسی، جدول بلند، نمونه کد و صفحات نماینده PDF را بررسی کنید. اگر سورس محصول تغییر کرده، علاوه بر renderer باید محتوای Markdown با Controller/DTO/سرویس و OpenAPI تازه تطبیق داده شود؛ چاپ دوباره به‌تنهایی سند را به‌روز نمی‌کند.

شناسنامه تاریخی تحویل ۲۰۲۶-۱۰-۱۲ و نتیجه همان اجرا

این تحویل در `2026-10-12` بر مبنای سورس `main@f0ff3be` انجام شده است. تغییرات این مرحله مستندات و ابزار ساخت خروجی‌اند؛ کد محصول، frontend و schema تغییر نکرده‌اند. هیچ migration تازه ساخته یا روی دیتابیس عادی/Production اعمال نشده است.

restore موفق بود؛ build اولیه و نهایی هر دو صفر warning و صفر error داشتند. در هر دو دور ۶۶۵ suite، تست اجرا و موفق شدند: ۲۳۴ Integration، ۴۱۷ Unit و ۱۴ Architecture؛ شکست و skipped صفر. Integration از LocalDB ایزوله fixture استفاده کرد؛ SMS واقعی یا استقرار روی سرور انجام نشد.

شاهد تاریخی همین تحویل ۲۰۲۶-۱۰-۱۲: ۱۴۱ عملیات، ۱۸۲ schema JSON و ۱۹۰ نمونهٔ همان snapshot کنترل شدند. این اعداد کاتالوگ امروز نیستند. بازتولید جاری از 332 عملیات، 570 schema و مدل طراحی 107 جدول استفاده می‌کند؛ metadata با ApiExplorer و ارجاع schemaها کنترل می‌شود. نتیجهٔ اجرای جدید ابزارها و تست‌ها در گزارش پایان همگام‌سازی ثبت می‌شود؛ validation قدیمی به‌جای اجرای تازه یا smoke سرور اعلام نمی‌شود.

این بررسی، قرارداد نسخه مشخص مخزن را تأیید می‌کند. آمادگی داده اولیه سازمان، مجوز حساب‌های واقعی، اعتبار ServiceToken، تحویل storage، SMS پایدار و تنظیم proxy باید در محیط هدف جدا آزمایش شوند.

خروجی غیرهمزمان گزارش‌ها — STEP 11C

گزارش‌های جدولی STEP 11B اکنون با یک job پایدار SQL Server به خروجی خصوصی CSV تبدیل می‌شوند. فرمت‌های PDF و Excel پیاده‌سازی نشده‌اند، چون سند محصول فرمت اجباری تعیین نکرده و CSV UTF-8 بدون وابستگی تازه نیاز فعلی را پوشش می‌دهد.

عملیات	مسیر	نتیجه
ایجاد	`POST /api/v1/reports/exports`	اعتبارسنجی/authorization، ثبت `Queued` و پاسخ `202`
وضعیت	`GET /api/v1/reports/exports/{jobId}`	وضعیت امن فقط برای مالک و scope فعلی

عملیات	مسیر	نتیجه
دانلود	`GET /api/v1/reports/exports/{jobId}/download`	stream خصوصی فقط در `Completed`

body ایجاد شامل `reportType` بسته، `format=Csv` و `filters` typed متناظر گزارش است. سقف پیش‌فرض ۱۰هزار سطر و ۲۰MiB است. jobهای `Queued` پس از restart از SQL Server دوباره دیده می‌شوند؛ `Processing` دارای lease منقضی recover می‌شود و retry حداکثر سه بار است. claim با lockهای سازگار SQL Server انجام می‌شود و wake signal حافظه‌ای منبع حقیقت نیست.

در ایجاد، پردازش و دانلود دسترسی دوباره کنترل می‌شود. Parent به GuardianRelation و فرزند snapshotشده، Teacher به TeacherAssignment همان کلاس، Student به شخص حساب و Manager به همان سازمان محدود است. تغییر دسترسی می‌تواند تولید را با `REPORT_DATA_NO_LONGER_AUTHORIZED` متوقف یا دانلود را رد کند. فایل از `storage.FileAssets` و storage provider موجود استفاده می‌کند و physical path برنمی‌گرداند.

CSV quote/newline/comma را escape می‌کند و متن user-controlled با پیشوندهای `= + - @` را با apostrophe خنثی می‌کند تا spreadsheet formula injection رخ ندهد. قرارداد کامل type، lifecycleها، indexها، migrationها، retry و محدودیت‌ها در سند معماری STEP 11C آمده است.

STEP 12A — اعلان‌های داخلی

Notification Center از رکوردهای پایدار schema `notification` می‌خواند. اعلان فقط reference مقصد است و مجوز دسترسی محسوب نمی‌شود؛ endpoint تکلیف، آزمون یا نتیجه باید Enrollment، Membership و GuardianRelation جاری را طبق policy خودش دوباره بررسی کند.

عملیات	مسیر	ورودی/نتیجه
فهرست	`GET /api/v1/notifications`	`page` ، `pageSize<=100` ، `unreadOnly` ، `category` و `organizationId` ؛ پاسخ `NotificationPage`
شمارش نخوانده	`GET /api/v1/notifications/unread-count`	`organizationId` اختیاری؛ پاسخ `NotificationUnreadCount`
خواندن	`POST /api/v1/notifications/{notificationId}/read`	idempotent؛ پاسخ `NotificationItem`

هر سه endpoint به Bearer session معتبر نیاز دارند و فقط رکوردهای recipient جاری را می‌بینند. شناسه کاربر از client گرفته نمی‌شود و شناسه اعلان متعلق به دیگری با `notification_not_found` و 404 پنهان می‌شود. inbox می‌تواند notificationهای چند سازمان را همراه `OrganizationId` نشان دهد و با organization فیلتر شود.

انتشار تکلیف/آزمون، انتشار رسمی نتیجه و پیام‌های Direct/Group/Channel event نسخه‌دار Outbox را در همان transaction کسب‌وکاری ثبت می‌کنند. worker پس از commit اعلان مجاز را materialize می‌کند. متن نتیجه answer، score و note خصوصی ندارد و اعلان بیرونی پیام نیز متن خام، نام فایل یا Phone Number ندارد. Saved Messages اعلان تولید نمی‌کند.

در محدوده SMS، Push، STEP 12A عمومی، Quiet Hours، preferences، Email و realtime اعلان وجود نداشتند. OTP ملی‌پیامک کانال جداگانه Identity است. جزئیات entity، قفل migration، retry، deduplication، SQL Server و privacy والد در سند معماری STEP 12A آمده است.

تنظیمات و کانال‌های ارسال اعلان — STEP 12B

اعلان داخلی STEP 12A همچنان منبع حقیقت durable است. پس از materialize شدن `Notification`، سرویس `NotificationDeliveryCoordinator` با توجه به scope شخصی/سازمانی، preference، category، ساعات سکوت، عضویت جاری، device فعال و موبایل تأییدشده، تلاش‌های Push/SMS را در SQL Server ثبت می‌کند. provider بیرونی داخل transaction کسب‌وکاری فراخوانی نمی‌شود.

عملیات	مسیر	ورودی/نتیجه
دریافت تنظیمات	`GET /api/v1/notification-preferences`	`organizationId` اختیاری؛ هفت دسته effective با mandatory، default/explicit و rowversion
تغییر یک دسته	`PUT /api/v1/notification-preferences/{category}`	کانال‌ها، Quiet Hours، `TimeZoneId` و `ExpectedRowVersion`؛ فقط کاربر جاری
ثبت دستگاه	`POST /api/v1/notifications/devices`	`platform` و `pushToken`؛ پاسخ فاقد token
لغو دستگاه	`DELETE /api/v1/notifications/devices/{registrationId}`	فقط device متعلق به کاربر جاری؛ پاسخ 204

Scope سازمانی فقط با Membership فعال قابل مشاهده/تغییر است. ردیف scope شخصی و هر سازمان مستقل است. In-app خاموش‌شدنی نیست. Default کانال اختیاری `Push=true` و `Sms=false` است. category امنیت حساب mandatory است و server خاموش‌کردن Push/SMS آن را رد می‌کند؛ producer امنیتی تازه‌ای در این مرحله افزوده نشده است.

Quiet Hours فقط با timezone صریح همان preference اجرا می‌شود، زیرا User/Organization هنوز timezone مرجع ندارند. بازه روزانه و عبوری از نیمه‌شب پشتیبانی می‌شود؛ اعلان داخلی حتی در ساعات سکوت فوراً ایجاد می‌شود و delivery خارجی تا پایان بازه `Deferred` می‌ماند.

در تحویل تاریخی STEP 12B فقط providerهای `Development/Unavailable` موجود بودند؛ P1 اکنون گزینهٔ `WebPushAndFcm` را افزوده است و default/config قبلی را تغییر نمی‌دهد. allowlist، credential و secure context لازم‌اند؛ دریافت خارجی هنوز pending است. رجیستری device از session token جدا است؛ token در API/log برنمی‌گردد و تلاش ارسال فقط شناسه device را ذخیره می‌کند. logout، انقضای session یا revoke دستگاه مانع resolve شدن آن برای Push آینده می‌شود. قرارداد جاری P1.

`INotificationSmsProvider` کاملاً از OTP جدا است. شماره موبایل از credential تأییدشده Identity resolve می‌شود و body دلخواه frontend پذیرفته نمی‌شود. `MeliPayamakOtpProvider` و endpoint اختصاصی OTP تغییر نکرده‌اند؛ adapter ملی‌پیامک برای SMS عمومی تا دریافت قرارداد رسمی provider عمداً وجود ندارد.

`NotificationDeliveryDispatcher` تلاش‌های `Pending/Deferred` را با lease و قفل SQL Server claim می‌کند. timeout، transient retry محدود، permanent failure و suppression علت‌های جدا دارند. unique index بر `(NotificationId, Channel, DestinationReference)` از ساخت effect تکراری جلوگیری می‌کند. preview خارجی privacy-safe و deep link فقط navigation hint است؛ authorization مقصد باید دوباره اجرا شود.

index، schemaها، lifecycle، تنظیمات provider و محدودیت‌های دقیق در سند معماری STEP 12B آمده است.

اشتراک، دسترسی تجاری و ظرفیت — STEP 13A

Commerce وضعیت جاری اشتراک سازمان، قابلیت‌های فعال و ظرفیت‌ها را از PlanVersion منتشرشده resolve می‌کند. نام نمایشی Plan در authorization استفاده نمی‌شود و Subscriptionهای قدیمی با انتشار نسخه جدید خودکار تغییر نمی‌کنند.

عملیات	مسیر	نتیجه
خلاصه اشتراک مدیر	`GET /api/v1/commerce/summary`	Plan/Version، وضعیت اشتراک، Entitlementها، remaining، usage، quota و over-quota

این endpoint به Bearer session معتبر، workspace سازمانی، Membership فعال و نقش جاری `Manager` نیاز دارد. OrganizationId از frontend دریافت نمی‌شود و از `AccessContext` resolve می‌شود. نبود اشتراک خطای سرور نیست و با `state=NoSubscription` و HTTP 200 نمایش داده می‌شود؛ اشتراک خارج از بازه یا لغوشده `Inactive` است.

کلیدهای Entitlement شامل Reports، Messaging، Exams، Assignments، Academic و Storage هستند. quotaهای فعلی `ActiveLearners`، `ActiveClasses`، `StorageBytes` و `ReportExports` هستند. سه مورد نخست مصرف را از ماژول مالک می‌خوانند؛ فقط `ReportExports` در UsageLedger metered است. `null` برای Limit به معنی unlimited صریح است و نبود definition به معنی unlimited نیست.

برای مصرف metered، `SourceOperationId` و unique index retry را idempotent می‌کنند. Transaction سریال‌پذیر و `UPDLOCK/HOLDLOCK` روی Subscription مانع مصرف هم‌زمان بیش از آخرین واحد ظرفیت می‌شود. `CheckCapacityAsync` به‌تنهایی رزرو نیست؛ write pathهای authoritative آینده باید lock quota و write ماژول مالک را در یک transaction کوتاه ترکیب کنند.

Downgrade داده‌های آموزشی یا فایل را حذف نمی‌کند. مصرف بالاتر از سقف با `isOverQuota=true` دیده می‌شود و عملیات ظرفیت‌ساز آینده باید مسدود شود؛ workflow از قبل شروع‌شده کورکورانه قطع نمی‌شود.

هیچ endpoint مدیر برای ایجاد Plan، انتشار PlanVersion، تغییر quota یا فعال‌سازی Subscription وجود ندارد. Manager سازمانی Platform Admin نیست. Contacts، Backoffice، callback، gateway، Payment، Order و AI integration در این مرحله ساخته نشده‌اند. جزئیات مدل، concurrency، schema، خطاها و مرزهای آینده در سند معماری STEP 13A آمده است.

سفارش و پرداخت — STEP 13B

قیمت قابل خرید یک جزء immutable از `PlanVersion` منتشرشده است. Order مبلغ و currency آن نسخه را در لحظه ایجاد freeze می‌کند؛ body ایجاد Order هیچ Amount یا Currency معتبری از client نمی‌گیرد. واحد canonical فعلی `IRR` و precision مبلغ `decimal(18,2)` است. PlanVersionهای قدیمی که قیمت ندارند قابل خرید نیستند و backfill حدسی نشده‌اند.

عملیات	مسیر	ورودی/نتیجه
ایجاد سفارش	`POST /api/v1/commerce/orders`	`planVersionId` , `clientOperationId`؛ پاسخ 201 یا retry برابر 200
آغاز پرداخت	`POST /api/v1/commerce/orders/{orderId}/payments`	`clientOperationId`؛ PaymentAttempt، وضعیت، reference/URL امن
جزئیات	`GET /api/v1/commerce/orders/{orderId}`	Order، تاریخچه attemptها و fulfillment state
فهرست	`GET /api/v1/commerce/orders?limit=20`	حداکثر ۱۰۰ مورد برای مدیر و سازمان جاری

تمام مسیرها به workspace، Bearer session، Membership سازمانی، فعال و `Manager` جاری نیاز دارند. OrganizationId از `AccessContext` می‌آید. Order سازمان دیگر 404 است؛ Manager نمی‌تواند amount/currency را عوض کند، پرداخت را موفق اعلام کند یا Subscription را مستقیم فعال کند. POSTها rate limit مستقل دارند و idempotency همچنان با ClientOperationId و payload hash enforce می‌شود.

تماس `IPaymentGateway` بیرون SQL transaction انجام می‌شود. timeout به `Unknown` می‌رود و Failed فرض نمی‌شود. Verification server-to-server مبلغ، IRR و reference را تطبیق می‌دهد؛ settlement موفق Paid، Payment، PaymentAttempt و پیام activation Outbox را اتمیک ثبت می‌کند. worker Outbox Subscription قبلی را بدون حذف تاریخچه لغو و نسخه خریداری‌شده را دقیقاً یک بار فعال می‌کند. بنابراین Order پرداخت‌شده ممکن است موقتاً `SubscriptionActivationPending` باشد.

Provider واقعی و قرارداد callback هنوز تأیید نشده‌اند. runtime فقط `Payments:Provider=Unavailable` دارد و endpoint callback عمومی ساخته نشده است؛ provider تست فقط در Integration Test ثبت شده و تماس واقعی ندارد. Refund، provisioning، billing period، Order expiration، reconciliation/Finance، frontend، Backoffice و Notification سازمان جدید، پرداخت خارج از این مرحله‌اند. جزئیات error code، locking، schema، lifecycleها و محدودیت‌ها در سند معماری STEP 13B آمده است.

تعدیل پیام و ایمنی کودک — STEP 14D

ثبت report قبلی حفظ شده است، اما هر گزارش اکنون یک `ModerationCase` و action اولیه دارد. گزارش عادی Group/Channel کاربرمدیریت‌شده در صف بازبین همان گفتگو قرار می‌گیرد؛ Personal Direct، گفتگوی SystemManagedAcademic و هر `ChildSafetyConcern` به صف داخلی `PlatformSafety` می‌روند. API PlatformSafety تا STEP 15 ارائه نشده است.

عملیات	مسیر	ورودی/نتیجه
فهرست پرونده گفتگو	`GET /api/v1/conversations/{conversationId}/moderation/cases`	filter اختیاری `status` / `priority` و صفحه‌بندی حداکثر ۱۰۰
جزئیات	`GET /api/v1/conversations/{conversationId}/moderation/cases/{caseId}`	فقط snapshot همان پیام و safe identity
شروع بررسی	`POST .../{caseId}/start-review`	`expectedVersion` ؛ claim اتمیک
حل	`POST .../{caseId}/resolve`	`expectedVersion` و `internalNote` اختیاری
رد	`POST .../{caseId}/dismiss`	`expectedVersion` و `internalNote` اختیاری
فایل شاهد	`GET .../{caseId}/evidence/files/{fileAssetId}`	re-authorization در هر درخواست

Owner مجوز ضمنی review دارد. Admin فقط با flag مستقل `ReviewReports` مجاز است؛ `DeleteMessages` معادل آن نیست. Member، Manager سازمان بدون participant، participant/permission لغوشده و کاربر گزارش‌شده دسترسی ندارند. note داخلی در پاسخ API یا receipt گزارش‌دهنده نیست. status گزارش‌دهنده از مسیر موجود `GET /api/v1/messaging/reports/{reportId}` همگام می‌شود.

ارسال محتوای user-generated در گفتگویی که participant فعال Minor یا Unknown دارد پیش از persistence از `IContentSafetyProvider` عبور می‌کند. Adult-only provider را صدا نمی‌زند. `Allowed` مسیر عادی Message را ادامه می‌دهد؛ `Blocked` و `ReviewRequired` Message نمی‌سازند و `SafetyIncident` و پرونده Critical داخلی می‌سازند. خطا/timeout/پاسخ نامعتبر provider با `MESSAGE_SAFETY_UNAVAILABLE` و 503 fail-closed است، اما incident تخلف کاذب ایجاد نمی‌کند.

مسیرهای Send و Forward در هر گفتگوی child-sensitive هر دو مشمول این gate هستند؛ در Direct دو طرف و در Group/Channel participantهای فعال بررسی می‌شوند. بررسی دوباره پیش از commit، تغییر همزمان participantها را پوشش می‌دهد.

تصمیم جاری Internal Makan Content Safety، GAP-05 قاعده‌محور واقعی و محدود متن است؛ DevelopmentAllow فقط در محیط توسعه/تست است. unsupported media به ReviewRequired و unavailable در مسیر محافظت‌شده به fail-closed می‌رود. این provider از AI/RAG جداست و ادعای تحلیل معنایی کامل رسانه ندارد؛ قرارداد مصوب GAP-05 بر گزارش تاریخی STEP 14D مقدم است.

Backoffice پلتفرم — STEP 15E

Backoffice زیر مسیر مستقل `/api/admin/v1` قرار دارد. `OrganizationRole.Manager` هیچ مجوزی برای این مسیر ایجاد نمی‌کند؛ هر درخواست به Bearer session فعال و permission جاری Platform از SQL Server نیاز دارد. revoke نقش بدون انتظار برای انقضای token مؤثر می‌شود. جزئیات ماتریس نقش‌ها و نتیجه پذیرش در سند STEP 15E و قرارداد UX در تحویل

Frontend ثبت شده است.

قرارداد IAM و Audit

Method و Route	Permission	Request	Response
`GET /api/admin/v1/me`	`platform.access`	بدون body	`PlatformMeResult` با هویت امن، نقش‌ها و permissionهای مؤثر
`GET /api/admin/v1/roles`	`platform.roles.read`	بدون body	مجموعه `PlatformRoleCatalogItem`
`GET /api/admin/v1/role-assignments`	`platform.roles.read`	`userId` ، `roleKey` ، `activeOnly` ، `page` ، `pageSize`	`PlatformRoleAssignmentPage`
`POST /api/admin/v1/role-assignments`	`platform.roles.manage`	`GrantPlatformRoleCommand`	`PlatformRoleMutationResult` ؛ ایجاد 201 و retry همارز 200
`DELETE /api/admin/v1/role-assignments/{assignmentId}`	`platform.roles.manage`	`EndPlatformRoleAssignmentCommand` در body	`PlatformRoleMutationResult`
`GET /api/admin/v1/audit-logs`	`platform.audit.read`	filterهای actor/action/target/result/time و page ، sort allowlist	`AdminAuditLogPage`

grant/revoke نقش به `Reason` و step-up نیاز دارد. آخرین `PlatformOwner` با transaction و SQL Server lock قابل پایان‌دادن نیست. bootstrap مالک اول یک ابزار عملیاتی خارج HTTP است و پس از ایجاد اولین مالک بسته می‌شود. Audit append-only است و هیچ API برای update/delete ندارد.

Organization، User و Provisioning

Method و Route	Permission	Request	Response
`GET /api/admin/v1/organizations`	`platform.organizations.read`	search/status/page/pageSize و sort allowlist	`PlatformOrganizationPage`
`GET /api/admin/v1/organizations/{organizationId}`	`platform.organizations.read`	شناسه مسیر	`PlatformOrganizationDetail`
`GET /api/admin/v1/organizations/{organizationId}/suspension-impact`	`platform.organizations.read`	شناسه مسیر	`PlatformOrganizationImpactPreview`

Method و Route	Permission	Request	Response
`POST /api/admin/v1/organizations`	`platform.organizations.manage`	`CreatePlatformOrganizationCommand`	`PlatformProvisioningResult` ؛ 201 یا replay برابر 200
`POST /api/admin/v1/organizations/{organizationId}/suspend`	`platform.organizations.manage`	`PlatformOrganizationStateCommand`	`PlatformOrganizationMutationResult`
`POST /api/admin/v1/organizations/{organizationId}/reactivate`	`platform.organizations.manage`	`PlatformOrganizationStateCommand`	`PlatformOrganizationMutationResult`
`GET /api/admin/v1/organizations/{organizationId}/memberships`	`platform.organizations.read`	page/pageSize , `activeOnly`	`PlatformMembershipPage`
`POST /api/admin/v1/organizations/{organizationId}/invitations`	`platform.organizations.manage`	`CreatePlatformInvitationCommand`	`PlatformInvitationResult` ؛ 201 یا replay برابر 200
`PUT /api/admin/v1/organizations/{organizationId}/memberships/{membershipId}/role`	`platform.organizations.manage`	`ChangePlatformMembershipRoleCommand`	`PlatformMembershipMutationResult`
`DELETE /api/admin/v1/organizations/{organizationId}/memberships/{membershipId}`	`platform.organizations.manage`	`EndPlatformMembershipCommand` در body	`PlatformMembershipMutationResult`
`GET /api/admin/v1/provisioning`	`platform.provisioning.read`	status و page/pageSize	`PlatformProvisioningPage`
`POST /api/admin/v1/provisioning`	`platform.provisioning.manage`	`ProvisionOrganizationCommand`	`PlatformProvisioningResult` ؛ 201 یا replay برابر 200
`GET /api/admin/v1/users`	`platform.users.read`	search/status/page/pageSize و sort allowlist	`PlatformUserPage` با phone mask‌شده
`GET /api/admin/v1/users/{userId}`	`platform.users.read`	شناسه مسیر	`PlatformUserDetail` با PII محدود
`POST /api/admin/v1/users/{userId}/suspend`	`platform.users.manage`	`PlatformUserStateCommand`	`PlatformUserMutationResult`

Route و Method	Permission	Request	Response
`POST /api/admin/v1/users/{userId}/reactivate`	`platform.users.manage`	`PlatformUserStateCommand`	`PlatformUserMutationResult`
`POST /api/admin/v1/users/{userId}/sessions/revoke`	`platform.sessions.manage`	`RevokePlatformSessionsCommand`	`PlatformSessionRevocationResult`
`POST /api/admin/v1/users/{userId}/sessions/{sessionId}/revoke`	`platform.sessions.manage`	`RevokePlatformSessionsCommand`	`PlatformSessionRevocationResult`

تعلیق/فعال‌سازی Organization یا User به step-up نیاز دارد. provisioning حاوی `PlanVersionId` نیز step-up می‌خواهد؛ ساخت صرف Organization بدون subscription چنین الزامی ندارد. mutationها `Reason` می‌گیرند و audit می‌شوند. تعلیق User نشست‌های فعال او را revoke می‌کند. قاعده آخرین Manager و استقلال membershipهای چند سازمان در لایه persistence و transaction حفظ می‌شود.

Education، Messaging Metadata و PlatformSafety

Route و Method	Permission	Request	Response
`GET /api/admin/v1/education/classes`	`platform.education.read`	organization/status/search/page	`PlatformClassPage`
`GET /api/admin/v1/education/classes/{classId}`	`platform.education.read`	شناسه مسیر	`PlatformClassDetail`
`GET /api/admin/v1/education/teachers`	`platform.education.read`	organization/class/search/page	`PlatformTeacherAssignmentPage`
`GET /api/admin/v1/education/enrollments`	`platform.education.read`	organization/class/search/page	`PlatformEnrollmentPage`
`GET /api/admin/v1/education/assignments`	`platform.education.read`	organization/class/search/page	`PlatformAssignmentPage`
`GET /api/admin/v1/education/exams`	`platform.education.read`	organization/class/search/page	`PlatformExamPage`

Method و Route	Permission	Request	Response
`GET /api/admin/v1/messaging/conversations`	`platform.messaging.read_metadata`	organization/type/scope/visibility/status/page	`PlatformConversationMetadataPage` بدون message body
`GET /api/admin/v1/moderation/cases`	`platform.moderation.review`	status/priority/page	`PlatformModerationCasePage`
`GET /api/admin/v1/moderation/cases/{caseId}`	`platform.moderation.review`	شناسه مسیر	`PlatformModerationCaseDetail` با evidence همان پرونده
`POST /api/admin/v1/moderation/cases/{caseId}/start-review`	`platform.moderation.review`	`PlatformModerationVersionCommand`	`PlatformModerationCaseDetail`
`POST /api/admin/v1/moderation/cases/{caseId}/resolve`	`platform.moderation.review`	`PlatformCompleteModerationCommand`	`PlatformModerationCaseDetail`
`POST /api/admin/v1/moderation/cases/{caseId}/dismiss`	`platform.moderation.review`	`PlatformCompleteModerationCommand`	`PlatformModerationCaseDetail`
`POST /api/admin/v1/moderation/cases/{caseId}/enforce-content-removal`	هر دو `platform.moderation.review` و `platform.moderation.enforce`	`PlatformModerationVersionCommand`	`PlatformModerationCaseDetail`
`GET /api/admin/v1/moderation/cases/{caseId}/evidence/files/{fileAssetId}`	`platform.moderation.review`	شناسه پرونده و فایل	stream فقط اگر فایل به همان پرونده متصل باشد

Platform permission مجوز خواندن دلخواه Contacts ،Direct یا فایل را ایجاد نمی‌کند. metadata پیام‌رسان فاقد متن پیام است و evidence فقط از Case مجاز resolve می‌شود. mutationهای moderation optimistic concurrency دارند و هم AdminAudit و هم سابقه دامنه‌ای مربوط را حفظ می‌کنند.

Commerce، Finance و Operations

Method و Route	Permission	Request	Response
`GET /api/admin/v1/commerce/plans`	`platform.commerce.read`	search/page/pageSize	`PlatformPlanPage`

Method و Route	Permission	Request	Response
`GET /api/admin/v1/commerce/plans/{planId}`	`platform.commerce.read`	شناسه مسیر	`PlatformPlanDetail` با version history
`POST /api/admin/v1/commerce/plans`	`platform.commerce.manage`	`CreatePlatformPlanCommand`	`PlatformPlanDetail` با 201
`POST /api/admin/v1/commerce/plans/{planId}/versions`	`platform.commerce.manage`	`CreatePlatformPlanVersionCommand`	`PlatformPlanVersionItem` با 201
`POST /api/admin/v1/commerce/plan-versions/{planVersionId}/publish`	`platform.commerce.manage`	`PublishPlatformPlanVersionCommand`	`PlatformPlanVersionItem`
`GET /api/admin/v1/commerce/subscriptions`	`platform.commerce.read`	organization/status/page/pageSize	`PlatformSubscriptionPage`
`POST /api/admin/v1/commerce/organizations/{organizationId}/subscription-override`	`platform.commerce.manage`	`OverridePlatformSubscriptionCommand`	`PlatformSubscriptionOverrideResult`
`GET /api/admin/v1/finance/orders`	`platform.finance.read`	organization/status/page/pageSize	`PlatformOrderPage` با referenceهای maskشده
`GET /api/admin/v1/finance/orders/{orderId}`	`platform.finance.read`	شناسه مسیر	`PlatformOrderItem` و تاریخچه امن attempt/payment
`GET /api/admin/v1/operations/notifications`	`platform.notifications.read`	بدون body	`PlatformNotificationOperations` فقط counter/state/provider availability
`GET /api/admin/v1/operations/jobs`	`platform.operations.read`	بدون body	`PlatformJobOperations`
`GET /api/admin/v1/operations/health`	`platform.operations.read`	بدون body	`PlatformSystemHealth` بدون secret/config خام

انتشار PlanVersion و override اشتراک به permission مخصوص و Reason نیاز دارند. نسخه منتشرشده immutable است و override تاریخچه را بازنویسی یا داده آموزشی را حذف نمی‌کند. Finance فقط read است؛ API پرداخت موفق دستی، refund یا reconciliation ساختگی وجود ندارد. عملیات مدیریتی notification/job نیز read-only و بدون token، شماره کامل یا متن اعلان‌اند. دوازده API مستقل Backend AI وجود دارند، اما منوی مدیریتی/محتوای AI عمومی از نقش Platform نتیجه نمی‌شود؛ Service Auth قبلی PASS و Full-12، Parent Provider Timeout و RAG lifecycle deferred هستند.

صفحه‌بندی، خطا و رفتار حساس

- listها server-side هستند؛ مقدار پیش‌فرض اغلب `page=1&pageSize=30` است، membership/audit پیش‌فرض ۵۰ دارند و سقف معتبر در Application enforce می‌شود. sort فقط در مسیرهایی که صریحاً ارائه شده و فقط از allowlist پذیرفته می‌شود.
- پاسخ خطا `ProblemDetails` با کد پایدار دارد. `401` یعنی Bearer session نامعتبر/منقضی، `403` یعنی permission ناکافی، `404` برای resource غیرقابل مشاهده/ناموجود، `409` برای conflict یا last-owner/last-manager، `412` برای concurrency/version mismatch، `422` برای validation، `429` برای rate limit و `503` برای dependency fail-closed است.
- در runtime جاری عامل دوم وجود ندارد؛ ورود OTP، نشست معتبر، Platform permission و بازبینی جاری User/Session در عملیات حساس برقرارند. هیچ bypass یا مجوز جدیدی برای نقش مدرسه ساخته نشده است.
- تمام شناسه‌های client فقط selector هستند. membership، scope، permission، اتصال evidence و state جاری از SQL Server resolve می‌شوند.
- OpenAPI جاری در `/openapi/v1.json` قرارداد transport را منتشر می‌کند؛ نام DTOهای جدول بالا مرجع schemaهای دقیق است.

تکمیل Messenger P1 و رجیستری Push

سخت‌سازی core Messaging در P1: خروجی زمان UTC در state، search/media، history، receipt/retry و realtime با `Z` حفظ می‌شود؛ ساعت ذخیره‌شده تغییر نکرده است. متن UTF-8 سالم که نویسه‌اش روی مرز نمونهٔ ۵۱۲ بایت قطع می‌شود دیگر اشتباهاً 415 نمی‌گیرد. Send/Edit سقف مشترک ۳۰۰، Reaction مشترک ۳۰۰ و Forward برابر ۱۲۰ درخواست در دقیقه/کاربر/process دارند؛ Upload=20، Search=60 و device=60 حفظ شده‌اند. پاسخ 429 در write/upload هم Retry-After دارد؛ OTP/AI تغییر نکرده‌اند. جزئیات رسانه، Reply/Edit/Delete/Forward/Pin، cursor و خطاها در Handoff P1 است. AV/CDR و codecهای افزوده پیاده نشده‌اند.

علاوه بر POST/DELETE دستگاه موجود، `GET /api/v1/notifications/devices` فهرست paginated و خصوصی metadata و `POST /api/v1/notifications/devices/web-push` ورودی استاندارد `endpoint/p256dh/auth/expectedVersion` را ارائه می‌کنند. تمام مسیرها authenticated و owner-scoped هستند؛ token و کلیدها هیچ‌گاه در پاسخ list/register نیستند. Web به allowlist دقیق HTTPS در `Notifications:WebPush:AllowedEndpointHosts` نیاز دارد؛ نبود آن `503` `notification_push_unavailable` است. تغییر کلید endpoint موجود version جاری می‌خواهد؛ conflict برابر ۴۰۹، ورودی نامعتبر ۴۰۰ و throttling mutation برابر ۴۲۹ با Retry-After است. سقف ثبت/لغو ۶۰ درخواست در دقیقه به‌ازای کاربر است. قرارداد و migration. ثبت subscription به‌تنهایی اثبات ارسال/دریافت واقعی Push نیست.

`GET /api/v1/notifications/devices/web-push/configuration` فقط config عمومی authenticated یعنی `isConfigured/vapidPublicKey` می‌دهد. provider واقعی `Notifications:PushProvider=WebPushAndFcm` ، Web را با Web Push/VAPID و Android/Ios را با FCM HTTP v1 می‌فرستد؛ کلید خصوصی و فایل service-account خارجی‌اند. payload پیام عمومی و دارای notificationId/conversationId/type/deepLink/tag است، نه متن خصوصی یا سن/مدرسه/رابطه. مقصد معنایی پیام `makan://conversations/{id}` است و client پس از ورود باید REST مجاز را بخواند. Delivered داخلی یعنی پذیرش provider، نه نمایش یا read. تست adapter با HTTP کنترل‌شده و SQL محلی وجود دارد؛ دریافت روی دستگاه واقعی هنوز **P1_PUSH_RUNTIME_PENDING** است. تنظیم‌ها، HTTPS مرورگر، APNs iOS، خطاها و محدودیت‌های runtime در Handoff بالا آمده‌اند؛ تنظیم OTP ملی‌پیامک تغییر نکرده است.

سخت‌سازی Production — STEP 16

نشست و refresh token

ورود OTP اکنون access token opaque کوتاه‌عمر و refresh token چرخشی برمی‌گرداند. raw refresh token فقط در پاسخ صدور/rotation دیده می‌شود و در SQL Server فقط HMAC/hash ذخیره می‌شود. هر access token ،refresh قبلی و refresh token قبلی را نامعتبر می‌کند؛ replay توکن مصرف‌شده کل نشست را revoke می‌کند.

Method و Route	Authentication	Request	Response
`POST /api/v1/auth/refresh`	بدون access token	`RefreshSessionCommand`	`RefreshSessionResult` با access/refresh جدید
`GET /api/v1/auth/sessions`	Bearer	بدون body	نشست‌های فعال خود کاربر، بدون token/hash
`DELETE /api/v1/auth/sessions/{sessionId}`	Bearer	شناسه نشست متعلق به خود کاربر	`204`
`POST /api/v1/auth/sessions/revoke-others`	Bearer	بدون body	`204` و حفظ نشست جاری

`VerifyOtpCommand` فیلد اختیاری `DeviceName` دارد. access پیش‌فرض ۳۶۰۰ ثانیه و absolute/refresh پیش‌فرض ۲۵۹۲۰۰۰ ثانیه است و از configuration خوانده می‌شود. OTP ورود نشست اولیه را می‌سازد؛ MFA و Step-Up در کد فعال وجود ندارند.

Health و رفتار میزبانی

Route	کاربرد	جزئیات عمومی
`GET /health`	alias سازگار liveness	فقط Healthy/Unhealthy
`GET /health/live`	زنده‌بودن process	dependency را probe نمی‌کند

Route	کاربرد	جزئیات عمومی
`GET /health/ready`	آمادگی دریافت traffic	config معتبر، SQL Server و persistent storage

Production در startup اتصال امن Identity key، SQL پایدار، Storage root مطلق و تنظیم معتبر میزبان/proxy می‌خواهد. CORS/Host طبق Configuration هستند: پایه و نمونه Test/Production فعلی AllowAnyOrigin=true و AllowedHosts=*؛ حالت محدود allowlist، false معتبر لازم دارد. cookie credentials نداریم. MFA/key ring قبلی شرط startup/ready نیستند؛ HTTP/SQL exceptionهای مستقل همچنان صریح‌اند. Swagger پیش‌فرض Production خاموش و X-Correlation-ID/ProblemDetails حفظ‌اند.

آپلود فقط با content type مجاز، پسوند سازگار و signature حداقلی معتبر پذیرفته می‌شود. mismatch با `FILE_CONTENT_INVALID` و `415` برمی‌گردد. دانلود همچنان authorization منبع را resolve می‌کند و content disposition از نوع attachment است.

در backoffice، `PlatformPlanDetail.Versions` به ۱۰۰ مورد و افراد فعال `PlatformClassDetail` به ۲۰۰ teacher و ۲۰۰ learner محدود شده‌اند. flagهای `VersionsTruncated`، `ActiveTeachersTruncated` و `ActiveLearnersTruncated` نشان می‌دهند client باید برای داده‌ی کامل از listهای paginated متناظر استفاده کند.

جزئیات blocker، deploymentها و ماتریس محیط در Production Readiness و IIS Deployment ثبت شده‌اند.

افزودهٔ Messenger P2 — Typing و Presence

قراردادهای واقعی `SetTyping`، `Heartbeat`، `SubscribePresence` / `UnsubscribePresence` و eventهای گذرا، همچنین `GET /api/v1/people/{userId}/presence` و دو audience جدید privacy در Handoff P2 ثبت شده‌اند. پیش‌فرض مصوب Online/Last Seen برابر Contacts از دفترچهٔ مقصد است؛ رابطهٔ جاری، Block و GAP-05 همچنان guard هستند. این بخش ادعای تحویل بقیهٔ P2 یا real Push نیست.

افزودهٔ Messenger P2 — سازمان‌دهی شخصی

`GET/PUT /api/v1/conversations/saved/messages/{messageId}/labels`، `savedLabel` در CRUD، Search مالک‌محور `/api/v1/messaging/folders` و selector اختیاری `folderId` در فهرست گفتگو اضافه شده‌اند. Saved همان canonical قبلی است؛ پوشه فقط query مجاز جاری را پیش از pagination فیلتر می‌کند. قرارداد label normalization، version، filter/override/pin و خطاهای 400/404/412 در همان Handoff P2 آمده‌اند؛ هیچ مجوز گفتگو از پوشه یا برچسب به دست نمی‌آید.

افزودهٔ Messenger P2 — مدیریت Group/Channel و عضویت

لینک دعوت hash-only، درخواست permission، approvalهای مستقل `ManageInvites` / `BanMembers` / `ChangeInfo`، مشخصات و محدودیت فردی نسخه‌دار اضافه شدند. endpointهای واقعی و قرارداد token یک‌بارقابل‌دریافت، expiry/revoke/max-use، 400/401/403/404/412 و current-state در Handoff P2 آمده‌اند. token در body redeem است، نه URL API یا audit؛ Ban عضویت را پایان می‌دهد و Unban آن را زنده نمی‌کند. queue خصوصی است و عضویت مدرسه/Enrollment را خودکار ایجاد نمی‌کند. این API مدیریت دستی کلاس رسمی یا اختیار مرور private Direct نیست.

افزودهٔ Messenger P2 — Topics

GET/POST `/api/v1/conversations/{id}/topics` و PUT نسخه‌دار `.../topics/{topicId}` برای Group، `topicId` در Send/history و `destinationTopicId` در Forward اضافه شدند. ManageTopics=4096 مستقل است؛ All جاری=8191 و مجوزهای قبلی خودکار افزایش نمی‌یابند. موضوع Open/Closed دارد؛ Close حذف تاریخچه نیست و Send تازه را با 409 رد می‌کند. FK مرکب از اتصال پیام به موضوع گروه دیگر جلوگیری می‌کند. قرارداد کامل permission معلم کلاس، FromJoin، نسخه، safety عنوان و `TopicChanged=13` در Handoff P2 ثبت است.

افزودهٔ Messenger P2 — Scheduled / Silent

POST/GET `/api/v1/conversations/{id}/scheduled-messages` و POST نسخه‌دار `.../{scheduleId}/cancel` قصد پایدار خصوصی را نگه می‌دارند؛ تا commit پیام واقعی، Sent نیست. current authorization، SQL lease/backoff و گیت Safety در dispatch، رزرو ClientMessageId و atomic Message/receipt/Outbox پیاده‌اند؛ تنظیم اجباری جدید یا migration خودکار startup ندارند. edit موعد API ندارد و cancel/recreate شناسهٔ تازه می‌خواهد. `isSilent` در Send/Forward/history، پیام/Unread/SignalR/اعلان داخلی را حفظ و external delivery را suppress می‌کند؛ Mute یا OTP نیست. جزئیات وضعیت، retry، دو نشست، خطا و migration در Handoff P2 و شواهد واقعی validation در ممیزی آمده‌اند.

افزودهٔ Messenger P2 — Search و current-state hardening

Search از `hasAttachment` ، `hasLinks` و `topicId` در همان محدودهٔ مجاز و پیش از pagination پشتیبانی می‌کند؛ count نامرئی افشا نمی‌شود و هویت senderها batch است. GET `/api/v1/conversations/{id}/me/restriction` فقط وضعیت خود caller بدون reason مدیریتی را می‌دهد. Contact Card در Send/Forward/dispatch، کشف پروفایل محافظت‌شده را دور نمی‌زند؛ global UserId مجوز نیست. mutation پیام پس از Safety، نشست و مجوز را داخل transaction دوباره می‌سنجد. عنوان/توضیح جدید گفتگو نیز audience محافظت‌شده و Safety جاری دارد؛ retry دقیق_Topic ثبت‌شده Message یا provider-call تازه نمی‌سازد. قرارداد nullable version، خطاها، حدود بدنه/normalization و خواندن coherent Folder/Label در Handoff P2 آمده‌اند؛ migration یا appsettings تازه‌ای برای این افزوده لازم نیست.

افزودهٔ Messenger P2 — Poll / Quiz و پذیرش

POST `.../conversations/{id}/polls` با privacy صریح، GET `.../messages/{messageId}/poll` و زیرمسیرهای POST vote/close و GET voters اضافه شده‌اند. Poll رأی نسخه‌دار و Anonymous/NonAnonymous دارد؛ Quiz فقط یک پاسخ نهایی و کلید محافظت‌شده پس از پاسخ خود کاربر دارد و Exam/Grade نمی‌سازد. Close نهایی و تراکنشی است؛ Forward فقط متن مستقل پرسش/گزینه‌هاست. رویداد PollChanged همان invalidation کمینه است؛ رأی‌دهنده/کلید در Delta افشا نمی‌شوند. پیام عمومی فقط پرسش را نگه می‌دارد؛ جزئیات گزینه‌ها از GET مجاز خوانده می‌شود. Migration `20261016211112_AddConversationPollsAndQuiz` فقط در fixture ایزوله آزمایش می‌شود؛ تنظیم SMS/AI/appsettings تغییر نکرده است. Handoff P2 مرجع DTO، مثال، rowversion مستقل، خطاها و محدودیت‌های واقعی است؛ ممیزی P2 شواهد آزمون/Git و نگاشت T01 تا T50 را ثبت می‌کند.

افزودهٔ Messenger P2 — Checklist v1

قرارداد مستقل Checklist مصوب شده است؛ نوع رشته‌ای `Checklist` و Modeهای `Personal` / `Collaborative` در API و OpenAPI وجود دارند. POST `/api/v1/conversations/{conversationId}/checklists` با `clientMessageId/title/mode/items[{itemId,text}]/topicId?/isSilent?` ، receipt پیام موجود را می‌دهد. GET `/api/v1/conversations/{conversationId}/messages/{messageId}/checklist` جزئیات caller را برمی‌گرداند؛ PUT همان مسیر با `title/mode/items/expectedVersion` تعریف را ویرایش می‌کند. PUT زیرمسیر `/items/{itemId}` با `isChecked/expectedDefinitionVersion/expectedStateVersion?` تیک را تغییر می‌دهد. `ConversationChecklistResult` شامل `messageId/conversationId/title/mode/items[{itemId,text,isChecked,stateVersion}]/isCompleted/version/messageVersion` است. نسخهٔ تعریف، پیام و state قابل جایگزینی با هم نیستند؛ نسخهٔ اولیه Collaborative ،Personal=null از ابتدا موجود است.

Personal فقط وضعیت خود حساب و همهٔ دستگاه‌های همان حساب را دارد؛ Collaborative مشترک است. `isCompleted` مشتق‌شده و بدون Grade/Assessment است. equivalent retry اثر تازه ندارد؛ stale متضاد 412 است. Creator با Send جاری و پنجرهٔ یک‌ساعته، یا Owner/Admin غیرسازنده با DeleteMessages و Send جاری مجاز به تعریف‌اند؛ کانال PublishMessages هم می‌خواهد. Mode و متن ItemId منتشرشده تغییر نمی‌کنند؛ برای جایگزینی، soft-retire و Guid تازه، برای Mode متفاوت Checklist تازه لازم است. هر تغییر ساختاری CAS دارد و state قدیمی حفظ می‌شود.

`ChecklistChanged=15` فقط invalidation روی Delta/SignalR موجود است؛ Personal به همان Collaborative ،User/تعریف به audience فعلی. تیک Message/Notification/unread تازه نمی‌سازد. generic Send/Schedule/Edit نمی‌توانند تعریف Checklist را دور بزنند. Forward فقط متن مستقل عنوان و آیتم‌های فعال؛ DeleteForMe/Everyone و Reply/Search/history فعلی حفظاند. جستجو روی عنوان است، نه state شخصی. GAP-05، Content Safety همهٔ متن‌ها، current ،Block membership/session و moderation پرونده‌محور پابرجا هستند. خطاهای ،Checklist: Invalid=400 NotFound=404، StructureImmutable=409، Conflict/StateConflict=412؛ خطاهای auth/Safety همان قبل‌اند. Handoff P2 مثال، حدود ۱۵۰/۲۰/۳۰۰، DTO ،body=64KiB و retry دقیق را دارد. Migration مستقل `20261017033804_AddConversationChecklists` است؛ appsettings، ملی‌پیامک، AI یا provider تازه لازم نیست.